



BLUESTAR PLM®
PLM inside Dynamics 365

USER GUIDE

Bluestar Configurator

Table of Contents

Introduction	4
File	5
Content	6
Product structure	6
Items	7
Routes	14
Switch	21
If	24
Select	27
For	29
Mate rules	31
Script	32
Model call	34
View	36
Domains	37
Constraints	40
Rules	44
Decision tables	48
Spreadsheet	50
Types	56
Functions	58
Assemblies	63
Manage mate rules	65
Define mate rules in Blueview	65
Import Spreadsheet	68
Operations	69
Solving	69
Testing	69
Settings	71
Debugging	72
Configurator programming language	73
Operators	73
Arrays	76
Boolean operations	78

Logical operators	78
Boolean relations	79
Built-in functions.....	80
Statements.....	89
Execution of Configurator model	101
Releasing Configurator model	104
Bluestar Sales configurator	110

Introduction

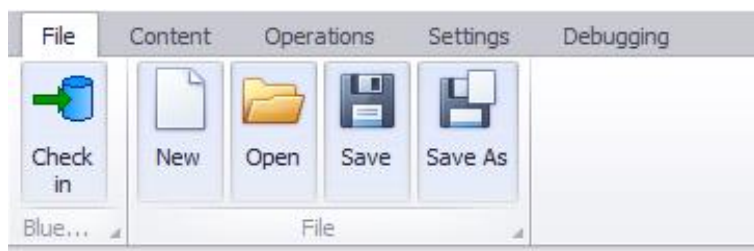
Bluestar PLM provides the Dynamics 365 for Finance and Operations & AX (365/AX) based product configuration platform that lets you manage the entire configuration process from quotation through engineering and production in a single system. As a result, you can eliminate redundant configuration efforts, achieve fast quotations, and automatically generate related CAD documentation.

The main features in Bluestar are:

- Dynamics 365 for Finance and Operations & AX Configurator Integration
- CAD Automation
- 3D Visualization
- Configure Complex Engineered-To-Order Products
- Third Party Configurator Integration.

File

The “File” tab includes two groups: “Bluestar” and “File”:



- **“Bluestar”** – consists of a “Check in” button. This button allows the user to check the Configurator model into Bluestar.
- **“File”** – consists of a standard set of buttons for work with files: “New”, “Open”, “Save”, and “Save as”.

Content

Product structure

The product structure tree is one of the main parts of the Configurator. It defines the set of items with which the Configurator can perform operations in CAD. The product structure tree also defines parameters of these items and that which of them will be loaded in the CAD model.

To open the “Product structure” window, the user must choose the “Content” tab in the menu bar and click on the “Product structure” button in the toolbar.

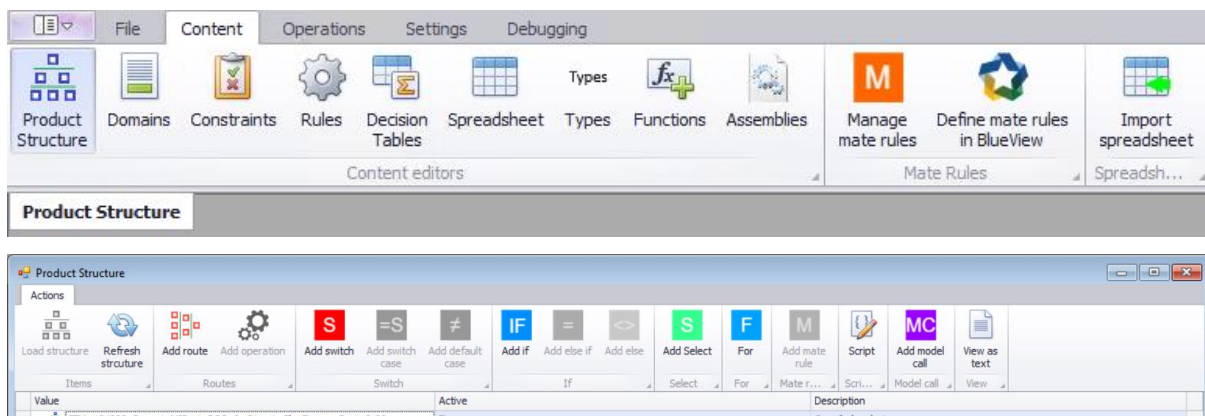


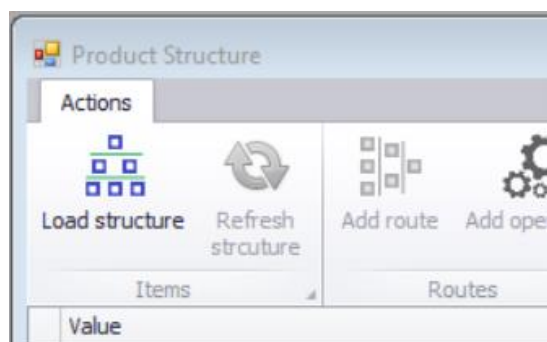
Figure 1: The "Production structure" window's toolbar

The “Product structure” toolbar consists of 10 groups:

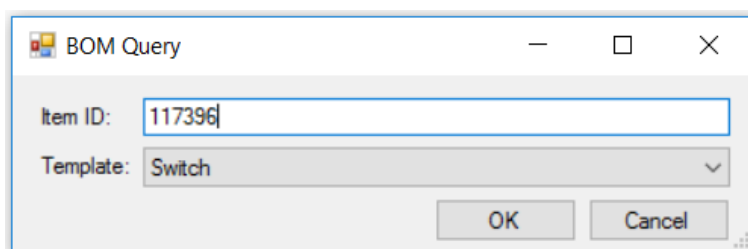
- Items
- Routes
- Switch
- If
- Select
- For
- Mate rules
- Script
- Model call
- View.

Items

The “Items” group has two buttons: “Load structure” and “Refresh structure”.



After creating a new Configurator model, only the “Load structure” button is available. To load a new structure, the user should click on the “Load structure” button, then input the Item ID from Bluestar and click on “OK”. The template defines how the default alternative decisions should be set up.



After clicking on the “OK” button, the product structure is loaded in, and the list of available items are represented in the “Product structure” window.

Each row in the “Product structure” has three cells: “Value”, “Active” and “Description”.

- **“Value”** – can contain an item, a BOM (Bill Of Material) or operations from the toolbar. This cell defines which action is carried out by the row.
- **“Active”** – defines, if the action of the row will be carried out or not. This field can contain True or False values or a Boolean expression (expression, that returns True or False values).
- **“Description”** – can contain text information and it can be used to comment the rows of the list.

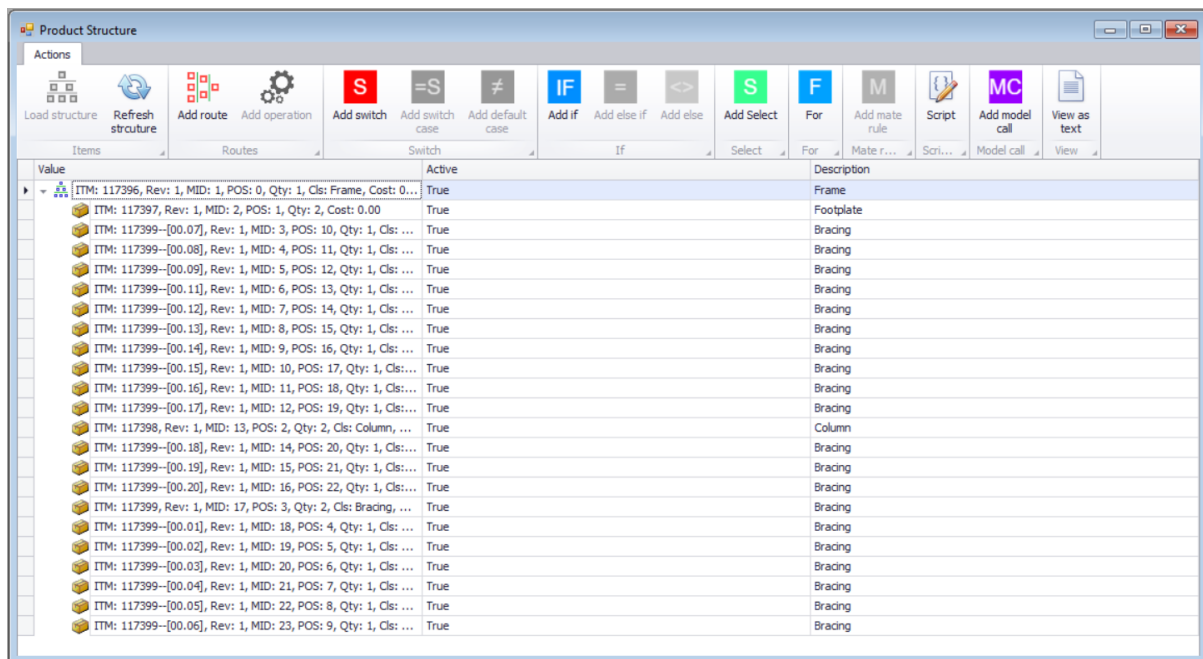


Figure 2: "Product structure" window

The first row in the given list (with a blue structure tree icon) is the BOM, that contains the items and can be rolled up by clicking on the downwards arrow, left to the BOM icon. The BOM contains global parameters, that have an influence on the items' parameters in the BOM and on their relative positions.

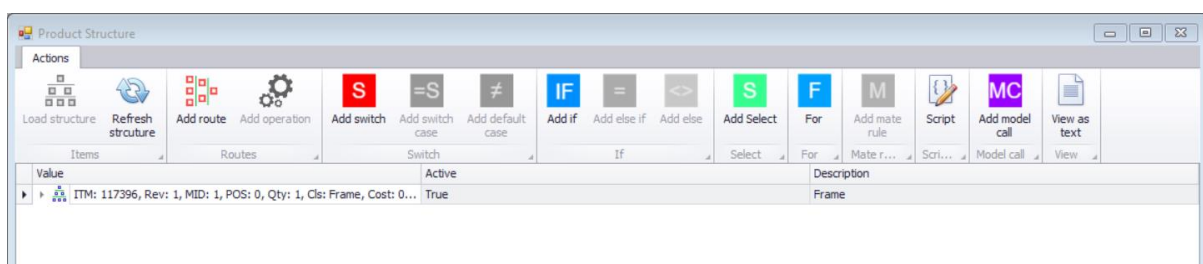


Figure 3: Rolled up BOM

Under the BOM row, one can find the items (with the yellow box icon). These are parts of the CAD models. Each of them can have its parameters, that defines the appearance and properties in CAD.

Each element in the list can be repositioned by dragging it up or down, but items can't be dragged out of their BOM.

To edit the parameters of an item, the user should double click on it in the list. This brings up a window with three tabs: "Item", "Attributes", and "BOM Line".

In the "Item" tab, one can see the fields with information of the BOM in Dynamics AX. In the "Attribute" tab is the list of the attributes of the model, that influence the CAD model. The "BOM Line" tab allows the setup of the generated BOM line.

Top-level items don't have a "BOM Line" tab, as they are not part of any BOMs.

Figure 4: Parameters window

The "Item" tab has twelve cells, which are:

- **"ModelID"** – the defined ID for the operation in the Configurator model.
- **"Active"** – defines if the item is active or not.
- **"ObjectID"** – Bluestar Object ID.
- **"Variant id"** – allows the user to control the ID of the configured object.
- **"Object Type"** – Bluestar Object type.
- **"Object name"** – Bluestar Object name.
- **"Variant name"** – The items variant name.
- **"Price"** – the item's price.
- **"Weight"** – the weight of the item.
- **"Description"** – the user can comment on the item.

Out of these, five cells ("ObjectID", "Object type", "Variant id", "Revision", and "Variant name") are filled automatically.

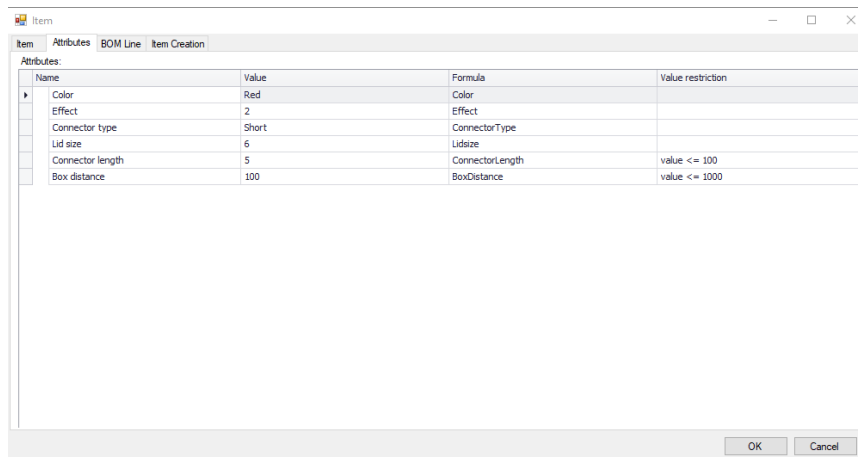


Figure 5: Attributes window

The “Attributes” tab has four columns: “Name”, “Value”, “Formula” and “Value restriction”.

- **“Name”** – defines the name of the parameter.
- **“Value”** – displays the value, that is used for building the model.
- **“Formula”** – contains an expression, that defines the attribute's value. It can be a mathematical expression, function, domain or a combination of them.
- **“Value restriction”** – can contain rules that forbid some values.

Attributes:			
Name	Value	Formula	Value restriction
Length	966.8	FrameDepth - 100	value > 0

Figure 6: Formula with domain and mathematical operation

Attributes:			
Name	Value	Formula	Value restriction
Length	1287.32367336269	diag(FrameDepth, Panel3Height)	value > 0

Figure 7: Formula with domain and function

NOTE – Press the CTRL and SPACE key combination to get help with building the formulas.

The screenshot shows a software window titled 'Item'. It has four tabs: 'Item', 'Attributes', 'BOM Line', and 'Item Creation'. The 'BOM Line' tab is active. Inside this tab, there are six input fields arranged in two columns. The left column has 'Position:' with value '1', 'Height:' with value '0', and 'Depth:' with value '0'. The right column has 'Constant:' with value '1', 'Width:' with value '0', and 'Phantom:' with value 'False'. At the bottom right of the window are 'OK' and 'Cancel' buttons.

Figure 8: BOM line window

The “BOM line” tab has 6 columns:

- **“Position”** – the position of the BOM line that will be created.
- **“Height”** – the height component of the BOM line’s consumption.
- **“Depth”** – the depth component of the BOM line’s consumption.
- **“Constant”** – the constant component of the BOM line’s consumption.
- **“Width”** – the width component of the BOM line’s consumption.
- **“Phantom”** – if the created item should be a phantom

The screenshot shows the same 'Item' window, but with the 'Item Creation' tab selected. The 'Create:' dropdown menu is set to 'Variant'. There is an empty 'Config id:' input field. Below these, there are two checkboxes: 'Attribute based reuse:' (unchecked) and 'Reused item must have same object ID:' (unchecked). 'OK' and 'Cancel' buttons are at the bottom right.

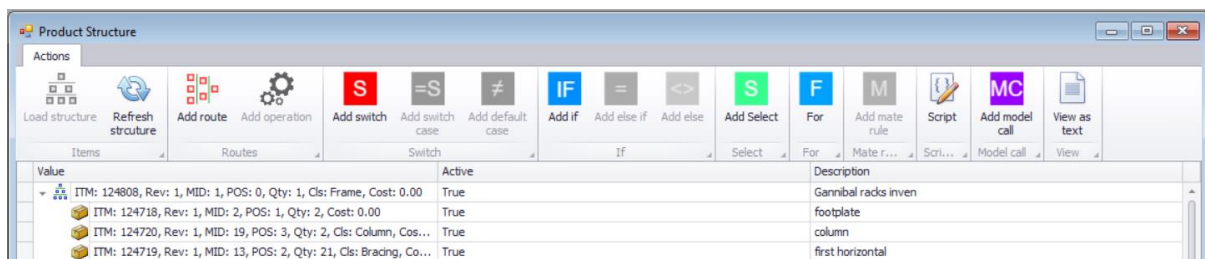
Figure 9: Item creation window

The “BOM line” tab has 4 columns:

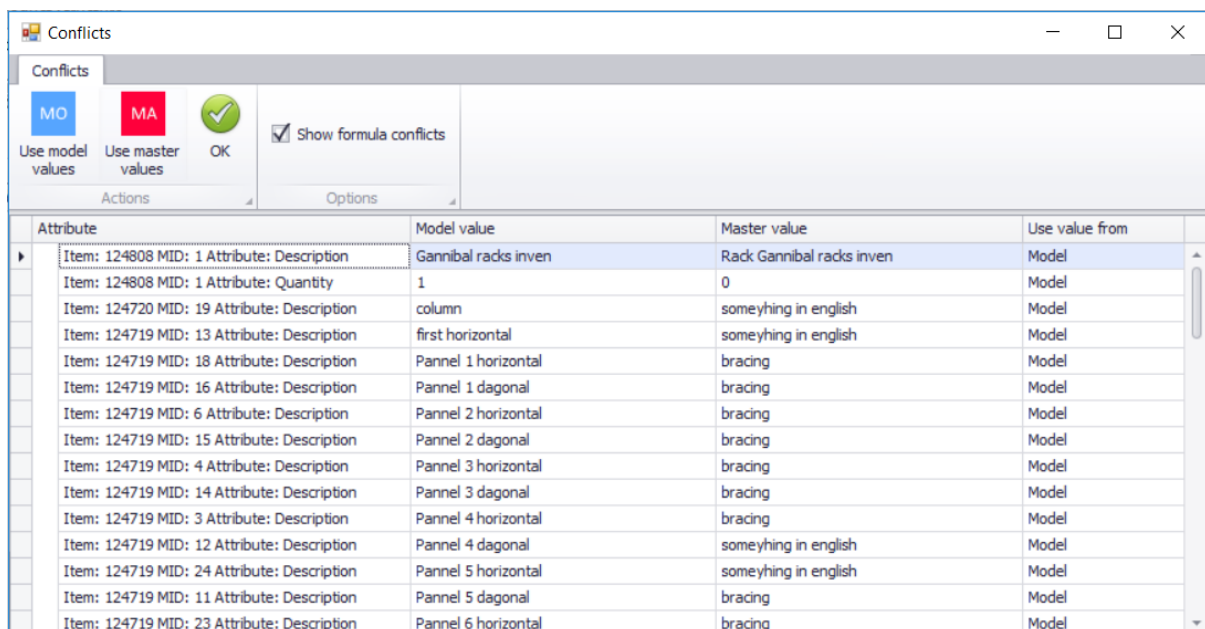
- **“Create”** – defines what type of item should be created if a new item is created based on this master item. It can be either “Variant”, or “UniqueObject”.

- **“Config id”** – allows the user to control the ID of the configured object. If an item with a matching id can be found then this item is reused.
- **“Attribute based reuse”** – If set then the configurator will try to find an item with the matching class and attributes and reuse that item
- **“Reused item must have same object ID”** – If set then the configurator will only reuse items that have the same objectid as the master item. They can still be a variant of the master.

After loading the structure, all other buttons in the toolbar become available.



The “Refresh structure” button refreshes the structure’s condition after changes were made in it outside the Configurator (for example changing the structure in Dynamics AX). It opens the window “Conflicts”, that shows all conflicts between values from the Configurator model and from the master model (the default model that the Configurator uses as a base for building models).



- **“Use model values”** – assigns values from the Configurator model to all conflicts in the list.
- **“Use master values”** – assigns values from the master model to all conflicts in the list.
- **“OK”** – accepts all changes and closes the window.

- **“Show formula conflicts”** – adds conflicts between formulas and master model values to the list.

Each row in the list of conflicts has four cells:

- **“Attribute”** – shows the item and model IDs and the conflicting attribute of the item of the item that has conflict.
- **“Model value”** – represents the conflicting value from the Configurator model.
- **“Master value”** – represents the conflicting value from the master model.
- **“Use value from”** – defines which value will be used in the Configurator model.

If one wants to use a mix of values from the Configurator model and master model, the user can define which value to use by clicking on the “Use value from” cell and choosing a variant from the drop-list. The “Use model values” and “Use master values” buttons are not suitable for this task, as they change all values in the list at once.

Routes

The “Routes” group includes two buttons: “Add route” and “Add operation”.

This group is used for creating substructures in the “Product structure” list, that will not affect the CAD model but allows the user to describe the production process, the production time and the product price. To create a route, the user should click on the “Add route” button in the toolbar.

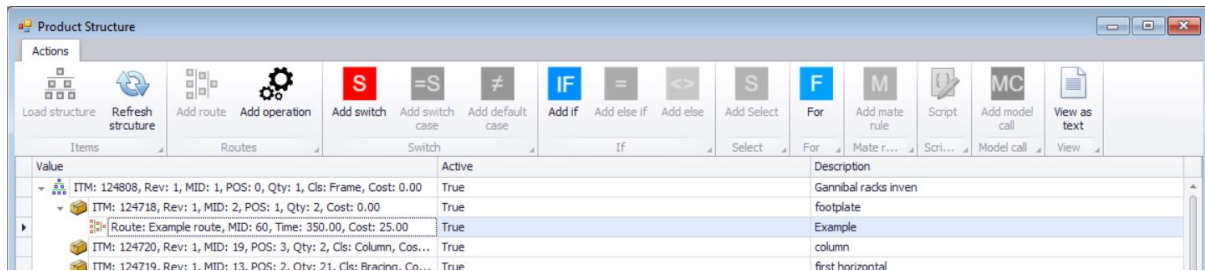
Mandatory fields:

- **“Model ID”** – can only contain numbers. This field is the defined ID for the operation in the Configurator model.
- **“Name”** – defines the name of the route.
- **“Time”** – filled out automatically based on the time required for the operations of the route.
- **“Site ID”** – the worksite where the route is used.
- **“Production Location”** – can be either internal or external.
- **“Item Group”** – the item group that the route is used for.
- **“From Quantity”** – the quantity required for the route.
- **“Active”** – defines if the given route and operations are carried out or not. This field can contain True or False values or a Boolean expression.

Optional fields:

- **“Description”** – the user can comment on the route.
- **“Cost”** – defines the cost of the route.

After clicking on the “OK” button, the route appears in the “Product structure” list.



Operation

Operations can be created only in the route substructure, by clicking on the “Add operation” button. This opens the “Operation” window, that has 5 tabs:

- General
- Setup
- Times
- Resources
- Note.

General

The screenshot shows the 'Operation' dialog box with the following fields and values:

Section	Field	Value
Model	ModelID	3
	Description	
	Active	True
	Total time	161
	Cost factor	0
	Cost	0
Identification	Operation name	Assembly
	Priority	Primary
	Route code	Route
Route	Route group	Proc
	Route type	Standard
	Property	

Mandatory fields:

- **“ModelID”** – can only contain numbers. This is the defined ID for the operation in the Configurator model.
- **“Operation name”** – can contain any symbols. This is the name of the operation.
- **“Active”** – defines, if the operation is carried out or not. This field must contain True or False values or a Boolean expression.
- **“Priority”** – priority of the operation.
- **“Route code”** – AX route code of the operation.
- **“Route group”** – AX route group of the operation.
- **“Route type”** – AX route type of the operation.

Optional fields:

- **“Description”** – is used for commenting on the operation in the “Product structure” list.
- **“Total time”** – contains the sum of all time fields and fills out automatically.
- **“Cost”** – defines the cost of the operation.
- **“Cost factor”** – defines the conversion between the time and cost. If it is 1, the cost of production equals the production time.
- **“Property”** – AX route type of the property.

Setup

The screenshot shows the 'Operation' dialog box with the 'Setup' tab selected. The 'Setup' section contains the following fields:

- Formula: Standard (dropdown)
- Scrap Percentage: 5 (text input)
- Factor: 1 (text input)
- Costing Resource: 1111 (text input)

The 'Cost Categories' section contains the following fields:

- Setup Category: Cutting (text input)
- Run Time Category: Car Audio (text input)
- Quantity Category: Assembly (text input)

At the bottom right, there are 'OK' and 'Cancel' buttons.

Mandatory fields:

- **“Formula”** – how the operation cost is calculated.
- **“Scrap Percentage”** – how much of the material that the operation uses is scrap.
- **“Factor”** – AX factor of the property.

Optional fields:

- **“Costing Resource”** – AX costing resource of the property.
- **“Setup Category”** – AX setup category of the property.
- **“Run Time Category”** – AX run time category of the property.
- **“Quantity Category”** – AX quantity category of the property.

Times

The screenshot shows the 'Operation' dialog box with the 'Times' tab selected. The dialog contains two main sections for time-related settings. The left section lists various time components with their current values: Queue time before (50), Setup time (50), Run time (50), Process quantity (4), Transit time (5), Queue time after (6), and Hours/Time (7). The right section has a single field for Transfer batch (8). The dialog also includes 'OK' and 'Cancel' buttons at the bottom right.

Field	Value
Queue time before	50
Setup time	50
Run time	50
Process quantity	4
Transit time	5
Queue time after	6
Hours/Time	7
Transfer batch	8

Optional fields:

- **“Queue time before”** – defines waiting time before the start of the operation.
- **“Setup time”** – defines the required preparation time for the operation.
- **“Run time”** – defines the time required for the operation.
- **“Process quantity”** – how much quantity can one process in one time unit.
- **“Transit time”** – defines the time it takes to move the product during production.
- **“Queue time after”** – defines the waiting time after the finish of the operation.
- **“Hours/Time”** – how many hours goes into one time unit.
- **“Transfer batch”** – how many can be transferred in one batch.

Resources

The 'Operation' dialog box has tabs for General, Setup, Times, Resources, and Note. The Resources tab is active, showing a 'Requirements' section with 'Quantity' set to 5 and 'Load' set to 25. Below these are buttons for Add, Edit, Copy, and Delete. A table lists various requirement types and their corresponding requirement values.

Requirement type	Requirement	Description
Resource	3115	
ResourceGroup	1120	
Capability	Assembly	
Skill	Accounting	
Course	00003	
Certificate	CGFM	
PersonTitle	Account Manager	
ResourceType	Machine	

At the bottom right are OK and Cancel buttons.

Fields:

- **“Quantity”** – the number of required resources.
- **“Load”** – load percentage for each resource.

Buttons:

- **“Add”** – creates a new row in the list and opens the “Resource requirement” window.

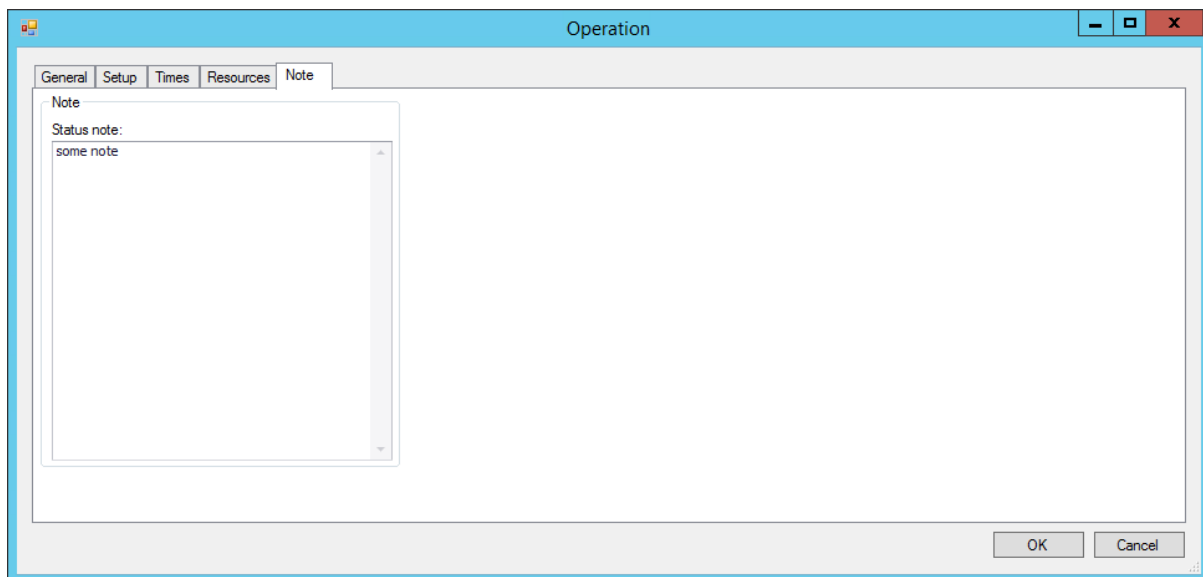
The 'Resource requirement' dialog box has a title bar with a minus, maximize, and close button. It contains the following fields and controls:

- Requirement type:** A dropdown menu currently showing 'Resource'.
- Requirement:** A text input field.
- Description:** A text input field.
- Minimum level needed:** A label with no input field.
- Operations scheduling:** A checkbox, currently unchecked.
- Job scheduling:** A checkbox, currently unchecked.

At the bottom right are OK and Cancel buttons.

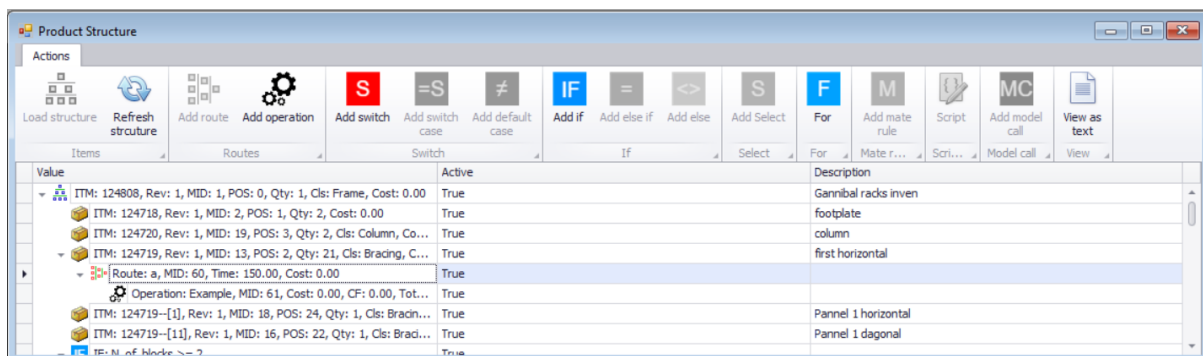
- **“Requirement type”** – the user can choose the type of the required resource.
- **“Requirement”** – the user can input the resource name and amount.
- **“Description”** – the user can comment on the requirement.
- **“Operation scheduling”** – use resource for operation scheduling.
- **“Job scheduling”** – use resource for job scheduling.
- **“Edit”** – allows the user to make changes in the row.
- **“Copy”** – creates a new row based on an existing one.
- **“Delete”** – deletes the row from the list with every information it contains.

Note



- **“Status note”** – used for writing notes connected to the given operation.

After clicking on the “OK” button, an operation appears in the “Product structure” list.

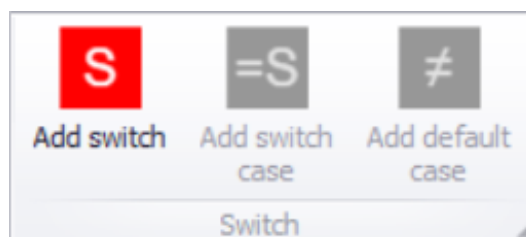


Routes can be moved in the “Product structure” in any way.

Operations can be moved inside routes and between the routes but can’t be placed outside the routes.

Switch

The “Switch” group includes three buttons which are “Add switch”, “Add switch case” and “Add default case”. This group is used to create “switch-case” constructions in the “Product structure”. “Switch” is a type of selection control mechanism, that is used for allowing the value of a variable or expression to change the control flow of program execution via a multiway branch.



To create a new switch structure, the user should click on the “Add switch” button. This button opens the “Switch” window.

The “Switch” window has three fields:

- **“Formula”** – must contain the key of the “Switch” construction. It can be a variable or an expression. The value of the variable or the expression will be compared with the “Cases”.
- **“Active”** – defines if the “Switch” structure is active, or not. It can contain True, False values, or a Boolean expression.
- **“Description”** – can contain comments about the structure.

After clicking on the “OK” button, the given “Switch” appears in the “Product structure” list.

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 0, Cls: Frame, Cost: 0.00	True	Rack Gannibal racks inven
Switch on: N_of_blocks	True	
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, Cos...	True	something in english
ITM: 124719, Rev: 1, MID: 13, POS: 2, Qty: 21, Cls: Reaction, Co...	True	something in english

To create cases, the user should click on the “Add switch case” button. Cases can be only created inside the “Switch” construction. The “Add switch case” button opens the “Switch case” window.

This window has two fields:

- **“Condition”** – contains the value with which the switch key will be compared. If the switch key equals to the condition value, the case instructions will be carried out.
- **“Description”** – can contain comments or description of the “Case”.

After creation, the “Case” will appear in the “Switch” substructure.

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 0, Cls: Frame, Cost: 0.00	True	Rack Gannibal racks inven
Switch on: N_of_blocks	True	
Condition: 2	True	Rack height is 2 blocks
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate

The “Case” can contain items and other constructions.

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 0, Cls: Frame, Cost: 0.00	True	Rack Gannibal racks inven
Switch on: N_of_blocks	True	
Condition: 2	True	Rack height is 2 blocks
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, ...	True	someyhing in english
Route: Example, MID: 81, Time: 0.00, Cost: 0.00	True	
ITM: 124719, Rev: 1, MID: 13, POS: 2, Qty: 21, Cls: Bracing, Cost: ...	True	someyhing in english
ITM: 124719--11, Rev: 1, MID: 18, POS: 24, Qty: 1, Cls: Bracing, C...	True	bracing

The “Add default case” button allows the user to create a “Case” that will be used if none one of the other cases match the supplied value. This button opens the “Switch default case” window, that only has a “Description” field for comments and descriptions.

The default case appears in the “Switch” substructure. The “Switch” structure can contain any number of cases, but only one default case.

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 0, Cls: Frame, Cost: 0.00	True	Rack Gannibal racks inven
Switch on: N_of_blocks	True	
Condition: 2	True	Rack height is 2 blocks
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate
Condition: 3	True	
Route: Example, MID: 81, Time: 0.00, Cost: 0.00	True	
Condition: 4	True	
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, ...	True	something in english
Default case:	True	if nothing approaches
ITM: 124719, Rev: 1, MID: 13, POS: 2, Qty: 21, Cls: Bracing, Cost: ...	True	bracing
ITM: 124719-[1], Rev: 1, MID: 18, POS: 24, Qty: 1, Cls: Bracing, C...	True	bracing

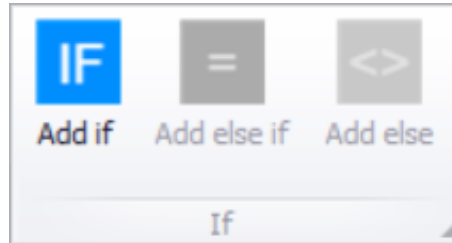
Example of a working “switch case” construction:

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 0, Cls: Frame, Cost: 0.00	True	Rack Gannibal racks inven
Switch on: N_of_blocks	True	
Condition: 2	True	Rack height is 2 blocks
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate
Condition: 3	True	
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, ...	True	something in english
Route: Example, MID: 81, Time: 0.00, Cost: 0.00	True	
Default case:	True	if nothing approaches
ITM: 124719-[1], Rev: 1, MID: 18, POS: 24, Qty: 1, Cls: Bra...	True	bracing
ITM: 124719-[11], Rev: 1, MID: 16, POS: 22, Qty: 1, Cls: Br...	True	bracing
ITM: 124719-[6], Rev: 1, MID: 23, POS: 8, Qty: 1, Cls: Brad...	True	bracing

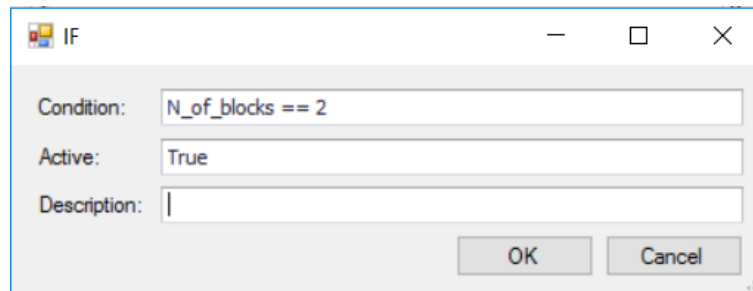
If the “N_of_blocks” domain has a value of “2”, the item from the case with the condition number “2” is used, but if the “N_of_blocks” domain has a value of “3”, then the item and the calculated route from the case with the condition number “3” are used. For every other “N_of_blocks” value, it uses the three items from the “Default case”, because neither of the two other cases match.

If

The “If” group includes “Add if”, “Add else if” and “Add else” buttons. This group is used to create “If-else” constructions in the “Product structure” list, that defines, if the given row in the list is active or not.



To create a new “If” structure, the user should click on the “Add if” button. This button opens the “If” window.



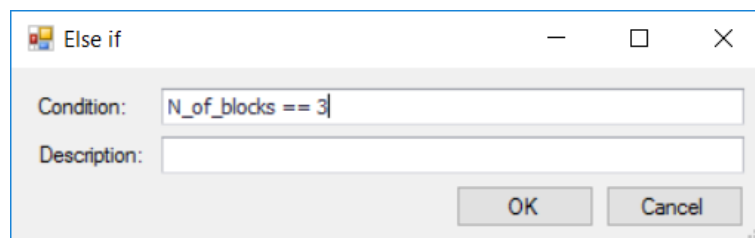
The “If” window has three fields:

- **“Condition”** – must contain a Boolean expression. This field defines the condition, where the rows in the “If” are active.
- **“Active”** – defines if the “If” structure is active or not. It can contain True or False values, or a Boolean expression.
- **“Description”** – can contain comments about the structure.

After creation, the “If” construction appears in the “Product structure” list. By default, “If” structure already has an “Else” substructure. It can be deleted and added again using the “Add else” button. The window for creating “Else” substructure has only a “Description” field.

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 0, Cls: Frame, Cost: 0.00	True	Rack Gannibal racks inven
IF: N_of_blocks == 2	True	
Then:	True	
Else:	True	
ITM: 124719-[5], Rev: 1, MID: 63, POS: 9, Qty: 1, Cls: Bracing, C...	True	something in english
ITM: 124719-[4], Rev: 1, MID: 64, POS: 10, Qty: 1, Cls: Bracing, C...	True	bracing

To create an “Else if” substructure, the user should click on the “Add else if” button. The “Add else if” button opens the “Else if” window.



The “Else if” dialog window contains two input fields: “Condition:” with the text “N_of_blocks == 3” and “Description:” which is empty. At the bottom right are “OK” and “Cancel” buttons.

This window has two fields:

- **“Condition”** – must contain a Boolean expression. This field defines the condition where the rows in the “Else if” are active.
- **“Description”** – can contain comments about the substructure.

After creation, the “Else if” substructure appears in the “If” structure.

Value	Active	Description
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate
IF: N_of_blocks == 2	True	
Then:	True	
Else if: N_of_blocks == 3	True	
Else:	True	
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, Cost: 0.00	True	column

“If” and its substructures can contain items and other constructions.

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 1, Cls: Frame, Cost: 0,00	True	Gannibal racks inven
IF: N_of_blocks == 2	True	
Then:	True	
Switch on: FrameDepth	True	
Condition: 32	True	
ITM: 124719-[11], Rev: 1, MID: 16, POS: 22, Qty: 1, Cls:...	True	Pannel 1 dagonal
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0,00	True	footplate
Else if: N_of_blocks == 3	True	
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, Cost:...	True	column
Else:	True	
ITM: 124719, Rev: 1, MID: 13, POS: 2, Qty: 21, Cls: Bracing, Cos...	True	first horizontal

Example of a working “If else” construction:

Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 1, Cls: Frame, Cost: 0,00	True	Gannibal racks inven
IF: N_of_blocks == 2	True	
Then:	True	
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0,00	True	footplate
Route: Example, MID: 86, Time: 0,00, Cost: 0,00	True	
Operation: Eaxmple, MID: 87, Cost: 0,00, CF: 0,00, Total ...	True	
Else if: N_of_blocks == 3	True	
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, Cost:...	True	column
Else:	True	
ITM: 124719, Rev: 1, MID: 13, POS: 2, Qty: 21, Cls: Bracing, Cos...	True	first horizontal
ITM: 124719-[1], Rev: 1, MID: 18, POS: 24, Qty: 1, Cls: Bracing,...	True	Pannel 1 horizontal

If the “N_of_blocks” domain has a value of “2”, the item and the calculated route from the “Then” substructure are used, but if the “N_of_blocks” domain has a value of “3”, then the item from the “Else-if” substructure is used. For every other “N_of_blocks” value, it uses the two items from the “Else” substructure.

Select

The “Select” group consists of an “Add select” button. This group is used for searching items with certain parameters. Items in the “Select” structure are only active, when the value of the searched attribute meets the search criteria. The “Add select” button opens the “Select” window.

Name	Type	Value	Formula
	string		

- **“Active”** – defines if the “Select” structure in the “Product structure” is active, or not. It can contain True or False values or a Boolean expression.
- **“Description”** – can contain comments about the structure.
- **“Attributes”** – includes “Add” and “Remove” buttons, and a list of attributes.
- **“Select”** – the user must input at least one attribute.

To create an attribute, the user should click on the “Add” button. After this, a new row appears in the list of attributes:

- **“Name”** – must contain the name of the searched attribute.
- **“Type”** – the user must choose the type out of the attribute’s value from the drop list.
- **“Value”** – the user must input the value that the item’s attribute must have.
- **“Formula”** – allows to fill the “Value” cell using an expression. It makes it possible to make a dynamic input in the “Select” tool. The “Formula” cell can contain an expression, that returns value with the type defined in “Type” cell.
- **“Remove”** – deletes the attribute from the list with all its information.

Example:

Active: ☒ True

Description:

Attributes

Name	Type	Value	Formula
I Length	real	966.8	FrameDepth-100

This “Select” construction will only make items active, that have the parameter “Length”, and it equals to the expression “FrameDepth - 100”. In this case, the value of this parameter must be 966.8.

After clicking on the “OK” button, the given “Select” row appears in the “Product structure” list. The “Select” construction only operates items placed in its substructure.

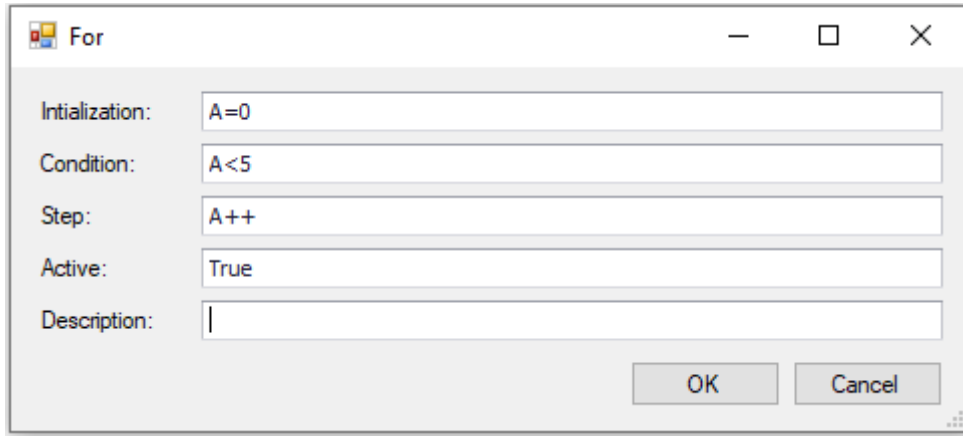
Value	Active	Description
ITM: 124808, Rev: 1, MID: 1, POS: 0, Qty: 1, Cls: Frame, Cost: 0.00	True	Gannibal racks inven
ITM: 124718, Rev: 1, MID: 2, POS: 1, Qty: 2, Cost: 0.00	True	footplate
ITM: 124720, Rev: 1, MID: 19, POS: 3, Qty: 2, Cls: Column, Cost...	True	column
Select: (Length)	True	
ITM: 124719, Rev: 1, MID: 13, POS: 2, Qty: 21, Cls: Bracing...	True	first horizontal
ITM: 124719-[1], Rev: 1, MID: 18, POS: 24, Qty: 1, Cls: Bra...	True	Pannel 1 horizontal
ITM: 124719-[11], Rev: 1, MID: 16, POS: 22, Qty: 1, Cls: Br...	True	Pannel 1 diagonal
ITM: 124719-[9], Rev: 1, MID: 60, POS: 5, Qty: 1, Cls: Brad...	True	bracing
ITM: 124719-[8], Rev: 1, MID: 61, POS: 6, Qty: 1, Cls: Brad...	True	something in english
ITM: 124719-[7], Rev: 1, MID: 62, POS: 7, Qty: 1, Cls: Bracing, ...	True	bracing
ITM: 124719-[6], Rev: 1, MID: 63, POS: 8, Qty: 1, Cls: Bracing, ...	True	bracing
ITM: 124719-[5], Rev: 1, MID: 64, POS: 9, Qty: 1, Cls: Bracin...	True	something in english

In this case, the “Select” construction checks the “Length” parameter of only the five items placed in its substructure.

For

The “For” group consists of one button. This construction is used for repeating operations in the “Product structure” list. It can be used to load more than one copy of an item to the CAD model.

To create a new “For” construction, the user should click on the “For” button. It opens the “For” window, that has five fields: “Initialization”, “Condition”, “Step”, “Active” and “Description”.



The screenshot shows a standard Windows-style dialog box titled "For". It contains five labeled text input fields stacked vertically. The first field, "Initialization:", contains the text "A=0". The second field, "Condition:", contains "A<5". The third field, "Step:", contains "A++". The fourth field, "Active:", contains "True". The fifth field, "Description:", is empty. At the bottom right of the dialog are two buttons: "OK" and "Cancel".

- **“Initialization”** – defines default value of the variable-counter (its value in the first cycle). This field can contain an expression with an integer or real type variable
- **“Condition”** – must contain a Boolean expression. The “For” cycle will be repeated until the condition is true. “Condition” usually contains a variable-counter and compares its value with other values every cycle.
- **“Step”** – defines an operation with variable-counter after each cycle.
- **“Active”** – defines if the “For” structure is active or not. It can contain True, False or Boolean expression.
- **“Description”** – can contain comments about the structure.

NOTE: "Initialization" uses an existing variable from the model, such as a Domain.

After clicking on the “OK” button, the “For” structure appears in the “Product structure” list. The “For” construction only operates items placed in its substructure.

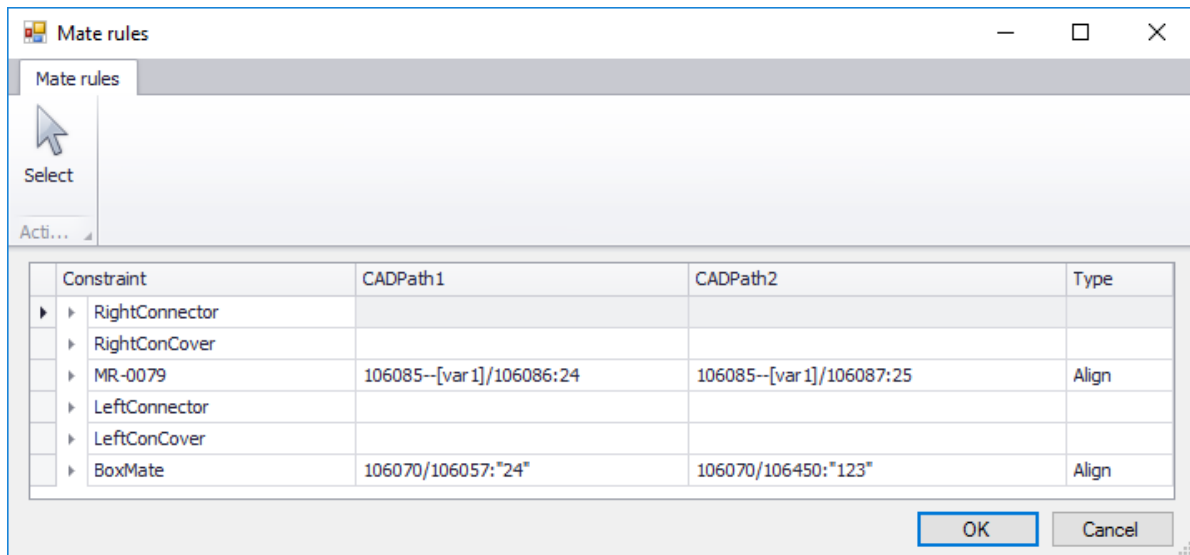
ITM: 124719-[5], Rev: 1, MID: 67, POS: 11, Qty: 1, CUS: ...	True	bracing
for(real A = 0; A < 5; A++)	True	
ITM: 124719-[9], Rev: 1, MID: 61, POS: 5, Qty: 1, ...	True	bracing
ITM: 124719-[8], Rev: 1, MID: 62, POS: 6, Qty: 1, ...	True	something in english
ITM: 124719-[7], Rev: 1, MID: 63, POS: 7, Qty: 1, ...	True	bracing
ITM: 124719-[20], Rev: 1, MID: 68, POS: 12, Qty: 1, Cl...	True	bracing

In the example given above, three items from the “For” construction are loaded five times. In the first cycle $A=0$, and after each cycle A is increased by 1 ($A++$). The cycles are going until A is lower than 5 (cycle with $A=5$ is not running), so the “For” structure will have five cycles (for $A=0, A=1, A=2, A=3, A=4$) and loads three items in each cycle.

Mate rules

The “Mate rules” group consists of an “Add mate rule” button. “Mate rules” are used for defining the relative positions of items in the CAD model.

To add a mate rule to the “Product structure” list, the user should click on the “Add mate rule” button. It opens the “Mate rules” window.



In the list of mate rules, the user should choose a rule and click on the “Select” button, or to double click on the rule. Mate rule appears in the selected structure.

Mate rules are created in “Blue View” and then imported to the Configurator.

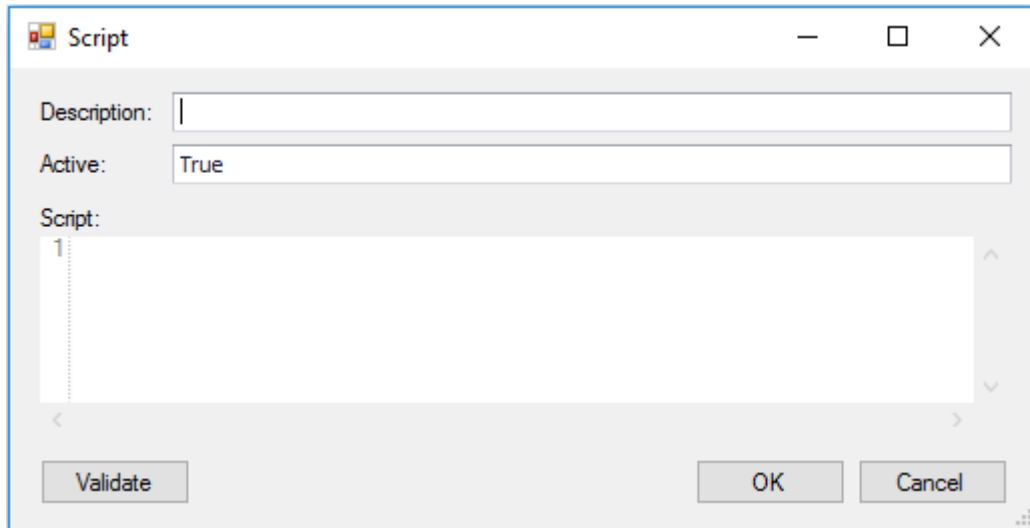
Example:

ITM: 106086, Rev: 1, MID: 188, POS: 2, Qty: 1, Us: Foundation, C...	True	Foundation
Switch on: ConnectorPosition	True	
Condition: "Left"	True	
Mate rule: LeftConnector	True	
Condition: "Right"	True	
Mate rule: RightConnector	True	
ITM: 106087, Rev: 1, MID: 189, POS: 3, Qty: 1, Cl: FrameCover, ...	True	Frame Cover

In this example, the mate rules are in a “Switch” substructure. If the “ConnectorPosition” domain’s value is “Left”, the mate rule named “LeftConnector” is applied. If the “ConnectorPosition” domain’s value is “Right”, the mate rule named “RightConnector” is applied.

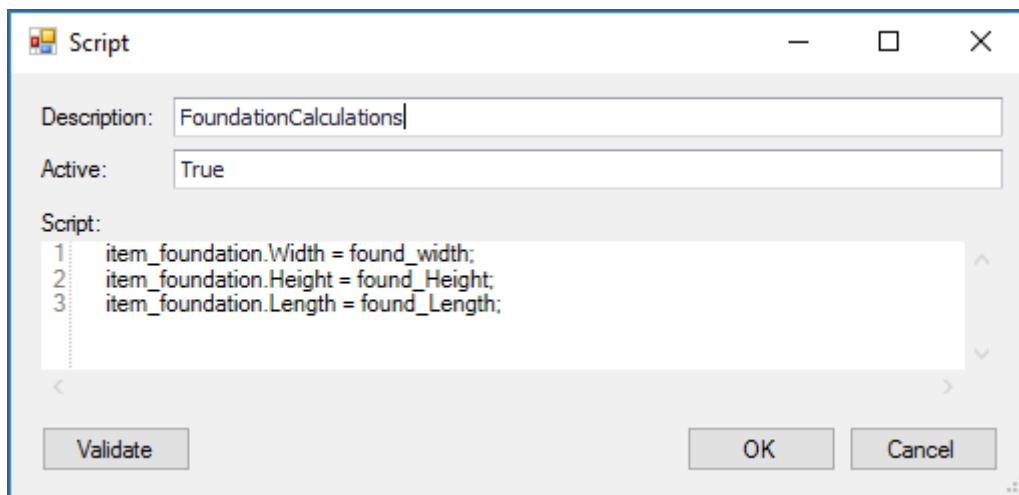
Script

The “Script” group consists of a “Script” button. To add a new script to the “Product structure” list, the user should click on the “Script” button. It opens the “Script” window.



- **“Description”** – can contain comment about the script.
- **“Active”** – defines, if the script is carried out, or not. It can contain True or False values or Boolean expression.
- **“Script”** – must contain the script, that must be carried out.

“Script” can be placed anywhere in the “Product structure” list.

Example:

This script will apply new values to “Width”, “Height” and “Length” item parameters, with “foundation” ModelID.

ITM: 108718, Rev: 2, MID: 62, POS: 12, Qty: 1, Cost: 0.00	True	Lid - 2mm - Blue
ITM: 107513, Rev: 1, MID: 63, POS: 13, Qty: 1, Cost: 0.00	True	Standard Screw PH2x25
Script: item_foundation.Width = found_width;	True	FoundationCalculations
ITM: 106086, Rev: 1, MID: foundation, POS: 2, Qty: 1, Cost: 155.00	True	Foundation
ITM: 106087, Rev: 1, MID: 32, POS: 3, Qty: 1, Cost: 25.00	True	Frame Cover

Figure 9: The previously made script in the “Product structure” list

Model call

The “Model call” group includes an “Add model call” button. It used for defining a call to a sub model. This can be used to reuse an existing model for an item, instead of modeling the rules multiple times.

To create a model call, the user should click on the “Add model call” button. It opens the “Model call” window. It has “ObjectID”, “VariantID”, “Position” and “Description” fields.

- **“ObjectID”** and **“VariantID”** – filled automatically after loading in a model.
- **“Position”** – defines the position of the “Model call” in the BOM.
- **“Description”** – can contain comment about the “Model call” structure.

The user should click on the “Load model definition” button to choose an item to load it to the sub model. In the “Select model” window, the user should input the model’s ObjectID, and optionally a VariantID.

After clicking on the “OK” button, the “Model call” window displays domains of the called model. They are divided into two groups: “Input domains” and “Output domains”.

The “Input domains” list contains domains with “Input” type from the called model.

Input domains		
Sub model domain	Formula	Value
Column_height	ConnectorLength	5
FrameDepth	6+3	9
N_of_blocks	8	8
Panel1Height		1050
Panel2Height		1050
Panel3Height		1050
Panel4Height		1050
Panel5Height		1050

Values of these domains can be changed by inputting new values in the “Value” cell or by using expressions in the “Formula” cell. Expressions can contain domains from the current Configurator model.

The “Output domains” list contains domains with “Output” type from the called model. Values of the output domains of the called model can be used like inputs in domains of current model.

Example:

Output domains	
Sub model domain	Current model domain
I Runout	BoxDistance

In this example, the value of the “Runout” output domain is used as an input for the “BoxDistance” domain.

View

The “View” group includes only a “View as text” button. This button allows to represent information of each row in the “Product structure” list in text form. For this, the user should choose a row in the list and click the “View as text” button.

Example:

IF: ConnectorPosition == "Left"	True	Left or right connector position
Then:	True	
Mate rule: LeftConnector	True	
Mate rule: RightConCover	True	
Else:	True	
Mate rule: LeftConCover	True	
Mate rule: RightConnector	True	

Figure 10: “If” structure is selected

```

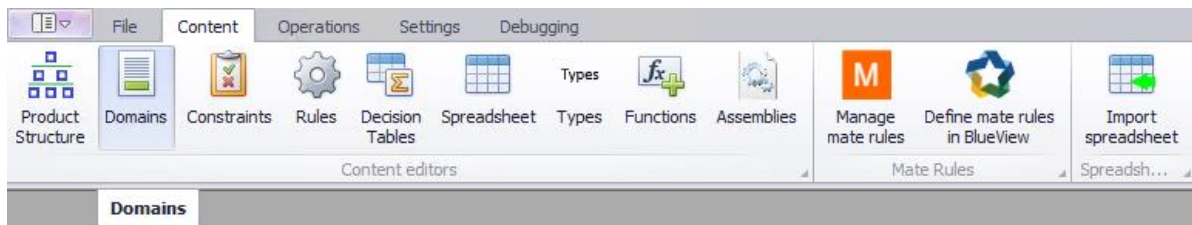
106085 - Notepad
File Edit Format View Help
IF: ConnectorPosition == "Left", Desc: Left or right connector position
  Then:
    Mate rule: LeftConnector
    Mate rule: RightConCover
  Else:
    Mate rule: LeftConCover
    Mate rule: RightConnector
  
```

Figure 11: Notepad with all information about the selected “If” construction

Domains

“Domains” are one of the main tools of interaction between the user and the Configurator. Domains get values from the user, perform calculations, then return information to user.

To open the “Domains” window, the user must choose the “Content” tab in the menu bar and click on the “Domains” button in the toolbar.



The “Domains” window has its own toolbar that allows to edit domain list.

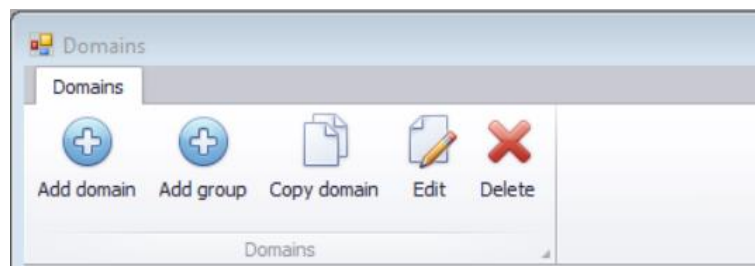


Figure 12: The “Domains” window's toolbar

The toolbar consists of:

- **“Add domain”** – creates a new domain in the domain list.
- **“Add group”** – allows the user to organize domains into groups.
- **“Copy domain”** – allows to create a new domain based on an existing one.
- **“Edit”** – edits the domains in the list. They also can be edited by double clicking on them, or by changing values directly in its domain list.
- **“Delete”** – allows to delete domains from the list with all its information.

Domain

Name:

Display name:

Mode:

Description:

Type:

Items:

Lower bound:

Upper bound:

Value:

Default value:

Value restriction:

Formula:

Visible:

Mandatory:

Sync spreadsheet: ☐

OK Cancel

To create a new domain, it is necessary to input a domain name (other fields can have default settings).

- **“Name”** – can only contain letters, numbers and underscore.
- **“Display name”** – can contain any symbols and is used only to display domain name in the list for the customer.
- **“Mode”** – defines, if the domain gets the value from the user (“IN” mode), it is an output value (“OUT” mode) or makes calculations (“CALC”).
- **“Description”** – can contain comments about the domain.
- **“Type”** – defines the variable type of the domain’s value.
- **“Items”** – can contain values which the customer can choose from. The presence of values in the “Items” field disables manual input into domain and converts it into a drop list form.
- **“Lower bound”** and **“Upper bound”** – define bounds for input values. If the input is more than the upper bound, the value in the cell is automatically changed to the upper bound value. If it is less, than the lower bound, it is changed to the lower bound value.
- **“Value”** – defines the value of the domain.
- **“Default value”** – defines what value will the domain have after the start of the Configurator.
- **“Value restriction”** – forbids adding certain values for the domain. It can also function as a lower and upper bound but instead of changing the invalid values to a corresponding one, “Value restriction” forbids the input of invalid values.
- **“Formula”** – defines rules for calculations of the domain value. This field is used only in “OUT” mode.

- **“Visible”** – defines, if the domain is visible for the customer. Visibility can be defined not only with True or False values, but also by rules, that return True or False values.
- **“Mandatory”** – defines, if the domain must to have value. If the mandatory domain has no value, the Configurator will not start building the model.
- **“Sync spreadsheet”** – defines, if the data from the domain is synchronized with spreadsheet or not.

After clicking on the “OK” button, the domain appears in the list.

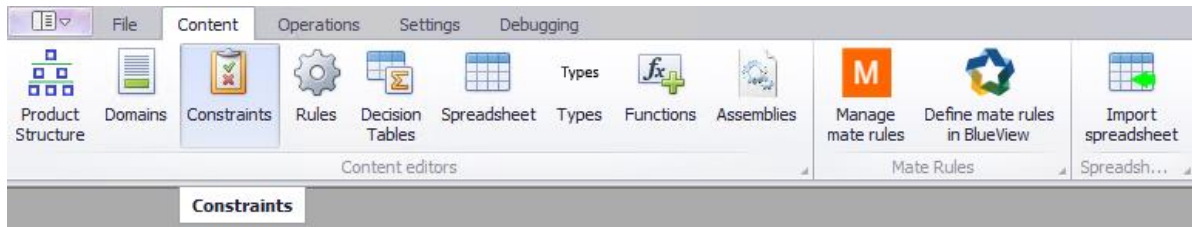
Name	Display name	Description	Type	Items	Lower Bound	Upper Bound	Value	Default Value	Value restriction	Formula	Mandatory	Visible	Spreadsheet	Mode
Color	Color		string	Red; Me...	0	0	Red	Red			True	True	☐	IN
Effect	Effect		integer	1; 2; 5	0	0	2	2			True	True	☐	IN
ConnectorPosition	Connector Position		string	Left; Rig...	0	0	Left	Left			True	True	☐	IN
Lidsize	Lid size		real	2; 4; 6	0	0	6	6			True	True	☐	IN
Connectortype	Connector type		string	Long; Sh...	0	0	Short	Short			True	True	☐	IN
Connectorlength	Connector length		real		0	0	5	5	value <= 100		False	True	☐	IN
Foundation	Foundation		string	No; Yes	0	0	Yes	Yes			True	True	☐	IN
BoxDistance	Box Distance		real		0	0	100	100	value > 0		True	True	☐	IN
IsEngineeringNeeded	Is Engineering N...		string	No; Yes	0	0	No			IF(Conne...	True	True	☐	OUT
Foundation attributes											True	True	☐	

The domain order in the list defines their order in the input form for customer. Changing the order can be done by dragging domains up and down with the mouse.

Constraints

The “Constraints” are used to avoid incorrect input of values by the customer. A constraint can forbid one, or several values for the domain, or allow only chosen values. Decision about prohibition, or resolution is based on other’s domain value.

To open the “Constraints” window, the user must choose the “Content” tab in menu bar and click on the “Constraints” button in the toolbar.



The “Constraints” window has its own toolbar, that allows edition of constraints list.

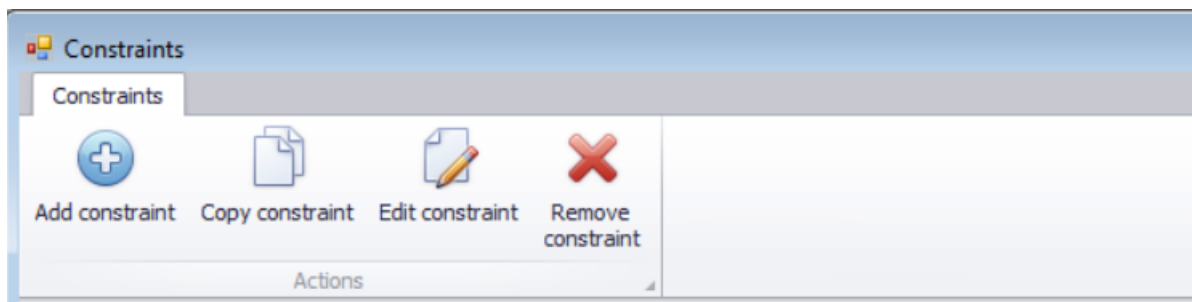


Figure 13: The “Constraints” window’s toolbar

The toolbar consists of:

- **“Add constraint”** – creates a new constraint in the constraint list.
- **“Copy constraint”** – allows to create a new constraint based on an existing one.
- **“Edit constraint”** – allows to edit constraints in the list. Constraints also can be edited by double clicking on them.
- **“Remove constraint”** – allows to delete constraints from the list with all their information.

A new constraint can be created by selecting the “Add constraint” button. It opens the “Constraint” window. The constraint requires a name, at least one premise, and at least one conclusion. The constraint’s name can contain any symbols.

Constraint

Name:

Premise

Add entry Remove entry Conjunction: AND

Domain	Operation	Value
abc abc N_of_blocks ShowWeather Type	EQUALS	

Restriction: IMPLIES

Conclusion

Add entry Remove entry Conjunction: AND

Domain	Operation	Value
--------	-----------	-------

OK Cancel

- **“Add entry”** – creates an empty row in the list of premises.
- **“Domain”** – the user can choose a domain to use in the premise. Available domains are represented in a drop-list that is generated automatically. Only domains with “Boolean”, “string”, “integer” or “real” value type are included in the list, but domains with “string” and “real” value types must have items, domain with “integer” value type can have items or lower and upper bounds and domains with “Boolean” value type are available with default settings.
- **“Operation”** – defines which type of comparison is used (positive or negative).
- **“Value”** – contains value for comparison.

If the premise list contains more, than one row, the user can choose the way premises work together using the “Conjunction” drop list:

- **“AND”** – conclusion is only true, when all premises are fulfilled.
- **“OR”** – conclusion is only true, when at least one premise is fulfilled.
- **“NOT AND”** – conclusion is true, when at least one premise is not fulfilled.
- **“NOT OR”** – conclusion is only true, when all premises are not fulfilled.

User can delete rows in the premise list using the “Remove entry” button.

Restriction drop list defines relations between the “Premise” and the “Conclusion” groups.

- **“IMPLIES”** – if the “Premise” group is true, only values that are defined in the “Conclusion group” are allowed. If the “Premise” group is not true, all values are allowed.
- **“EQUAL”** – if the “Premise” group is true, only values that are defined in the “Conclusion group” are allowed. If the “Premise” group is not true, it forbids values defined in “Conclusion group”.

Example of using “Restriction”:

- **“IMPLIES”**

- When “Type” equals to 4, “Boolean” can be only True.
- When “Type” not equals to 4, “Boolean” can be True or False.

The screenshot shows the 'Constraint' dialog box with the following configuration:

- Name:** Example
- Premise:**
 - Buttons: Add entry, Remove entry
 - Conjunction: AND
 - Table:

Domain	Operation	Value
Type	EQUALS	4
- Restriction:** IMPLIES
- Conclusion:**
 - Buttons: Add entry, Remove entry
 - Conjunction: AND
 - Table:

Domain	Operation	Value
boolean	EQUALS	True
- Buttons: OK, Cancel

- **“EQUAL”**

- When “Type” equals to 4, “Boolean” can be only True.
- When “Type” not equals to 4, “Boolean” can be only False (can’t be True).

The screenshot shows the 'Constraint' dialog box with the following configuration:

- Name:** Example
- Premise:**
 - Buttons: Add entry, Remove entry
 - Conjunction: AND
 - Table:

Domain	Operation	Value
Type	EQUALS	4
- Restriction:** EQUAL
- Conclusion:**
 - Buttons: Add entry, Remove entry
 - Conjunction: AND
 - Table:

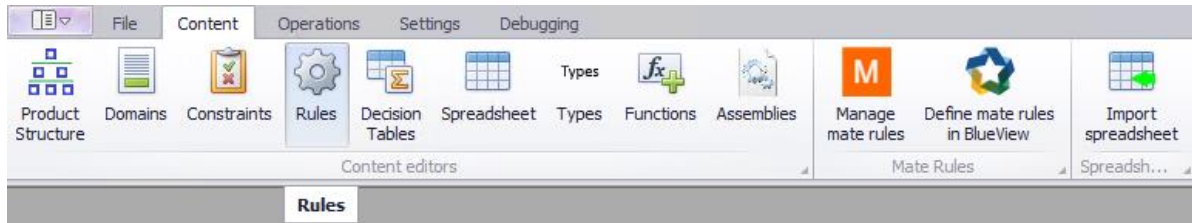
Domain	Operation	Value
boolean	EQUALS	True
- Buttons: OK, Cancel

The “Conclusion” group’s structure is exactly the same as the “Premise” group’s structure.

Rules

“Rules” are used for assigning values to different types of variables, or to perform some other instructions. Decisions are based on “if-then” construction.

To open “Rules” window user must choose “Content” tab in menu bar and click “Rules” button in toolbar.



The “Rules” window has its own toolbar, that allows the edition of the rules list.

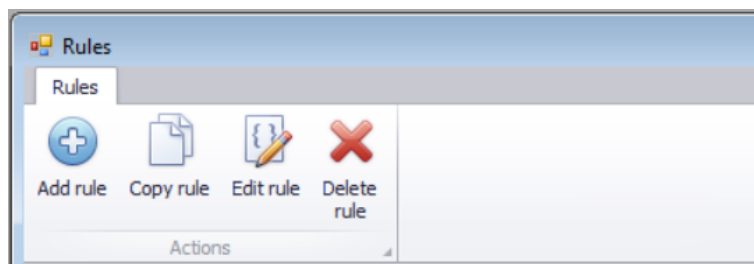
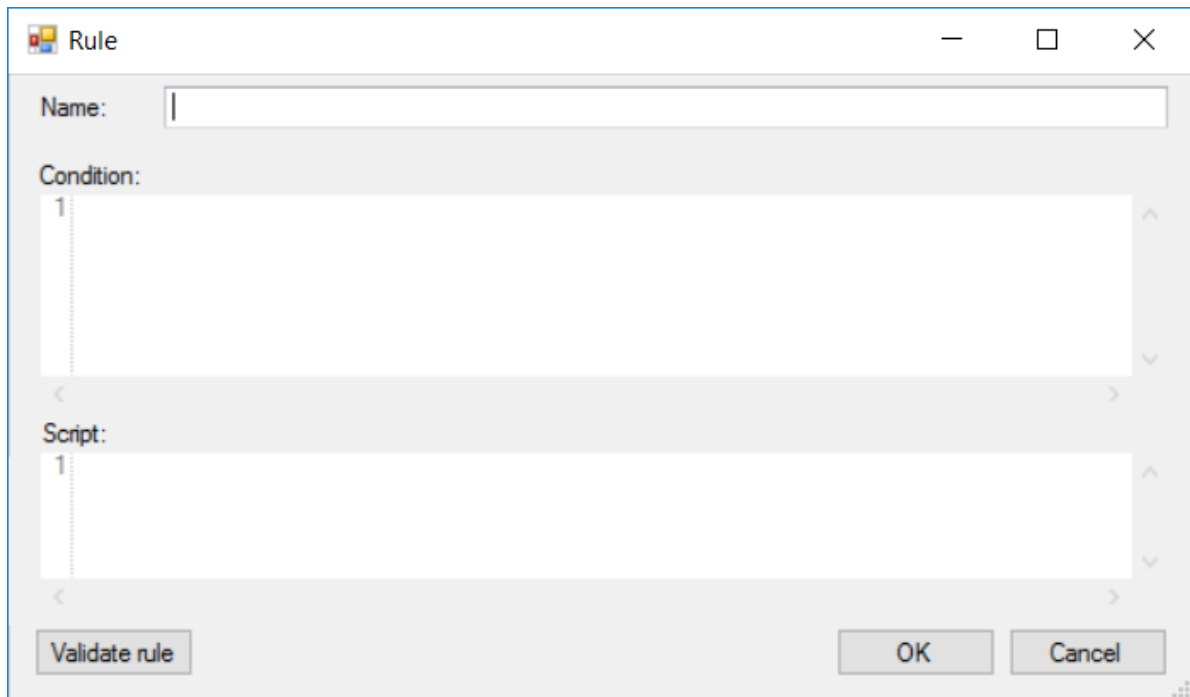


Figure 14: The “Rules” window’s toolbar

The toolbar consists of:

- **“Add rule”** – creates a new rule in the rule list.
- **“Copy rule”** – creates a new rule based on an existing one.
- **“Edit rule”** – edits the rule in the list. Rules can also be edited by double clicking on them.
- **“Delete rule”** – deletes the rule from the list with all its information.

Creating a new rule can be done by clicking on the “Add rule” button. This button opens the “Rule” window. After clicking on the “OK” button, the rule is displayed in the rule list.



Rules require a name, a condition and a script.

- **“Name”** – can contain any symbol.
- **“Condition”** – must contain a Boolean expression.
- **“Script”** – contains logical operations for assigning values of different types to variables or other instructions.
- **“Validate rule”** – checks, if the rule is correct.

The “Rule” is an “if-then” form, where the “Condition” group corresponds to an “if”, and the “Script” group corresponds to a “then” function.

Example:

Rule

Name:

Condition:

```
1 Column_height >= 1500 && Column_height <= 2000
```

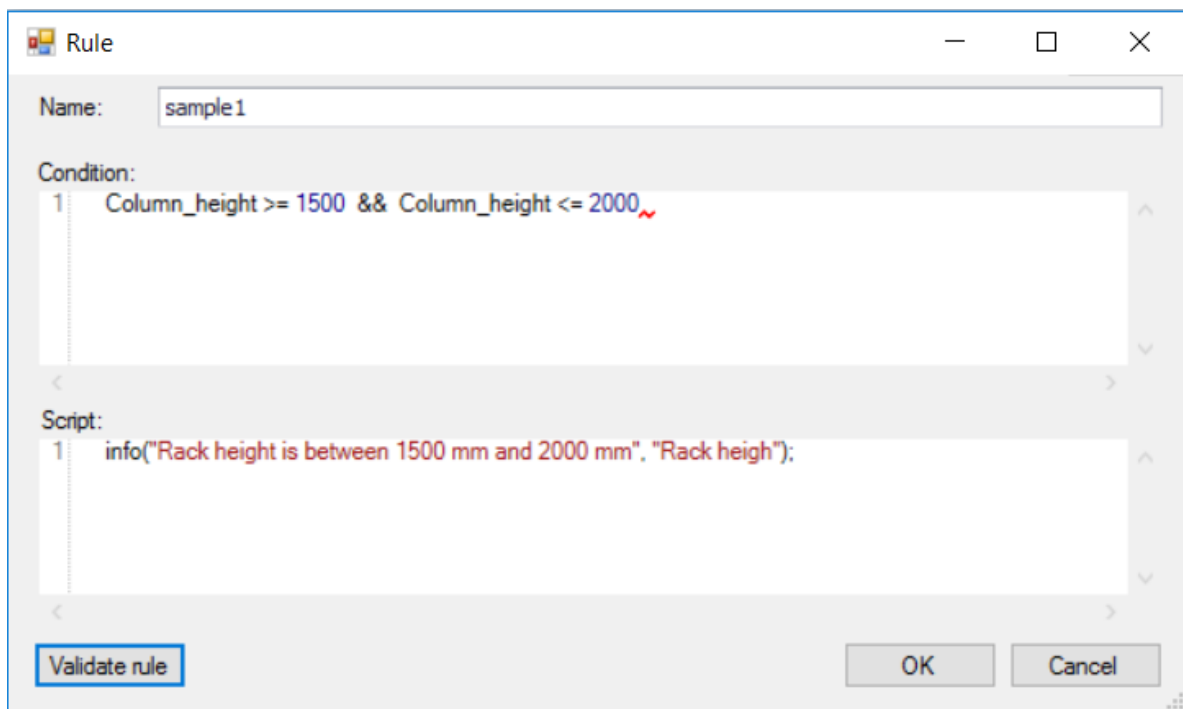
Script:

```
1 N_of_blocks = 2;
2 Panel1Height = 1200;
3 Panel2Height = 1200;
```

The rules in this example define, that if the “Column_height” domain is between 1500 and 2000, then the “N_of_blocks” domain equals to 2, and both the “Panel1Height” and “Panel2Height” domains equal to 1200.

Name		Value
I	Column height	1600
	Number of blocks	2
	Panel 1 Height	1200
	Panel 2 Height	1200

Figure 15: The domain list after applying the rule (condition of the rule is true)

Example:

The rule defines, that if the “Column_height” domain is between 1500 and 2000, then an information message pops up.

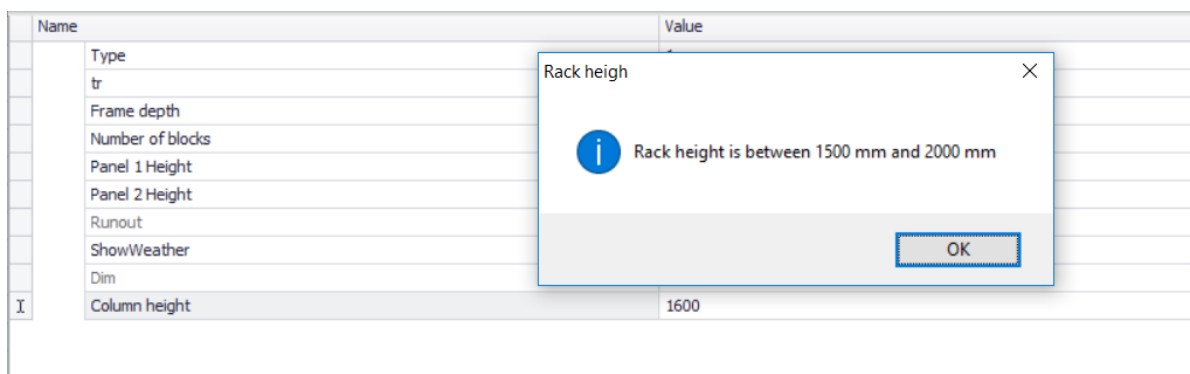
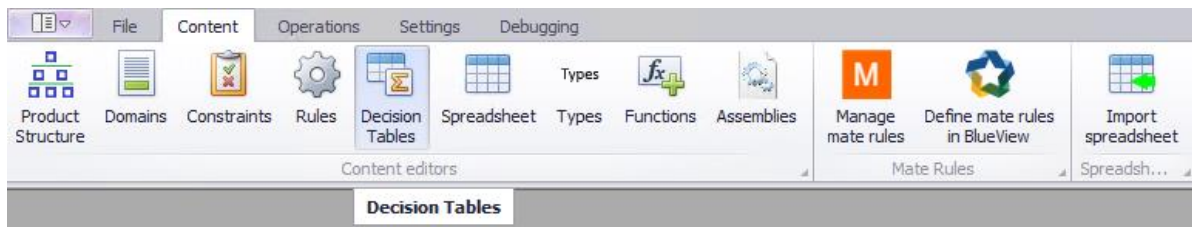


Figure 16: The domain list after applying the rule (condition of the rule is true)

Decision tables

Decision tables are used for defining related data, for example the electrical properties of various items in the model. These values can then be used in rules throughout the model.

To open the “Decision Tables” window, the user must choose the “Content” tab in the menu bar and click on the “Decision Tables” button in the toolbar.



The “Decision Tables” window has its own toolbar, that allows the edition of the tables.

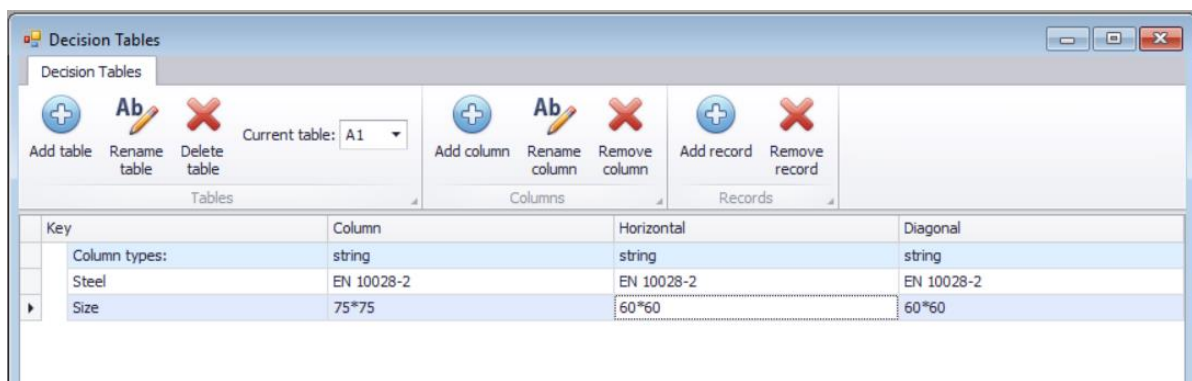


Figure 17: The “Decision Tables” window's toolbar

The toolbar consists of three groups: “Tables”, “Columns” and “Records”.

The “Tables” group includes:

- **“Add table”** – creates a new table.
- **“Rename table”** – changes the table’s name.
- **“Delete table”** – deletes the table with all its information.
- **“Current table”** – allows the user to switch between the tables defined in the model.

The “Columns” group includes:

- **“Add column”** – creates new columns.
- **“Rename column”** – changes column’s name.
- **“Remove column”** – deletes the column with all its information.

The “Records” group includes:

- **“Add record”** – creates new records (rows).

- **“Remove record”** – deletes the record with all its information.

To create a new table, the user should click on the “Add table” button and input the table name.

To create a new column, the user should click on the “Add column” button, input the column name and choose the variable type from the drop-list.

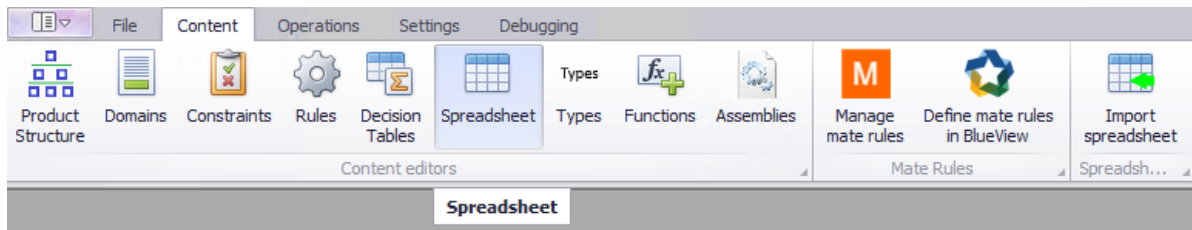
To create a new record, the user should use the “Add record” button and input the record name in the “Key” field.

After these, the table’s cells can be filled with values corresponding to the column variable types.

Spreadsheet

Spreadsheet is used for organizing, structuring and processing the data from the domains.

To open the “Spreadsheet” window, the user must choose the “Content” tab in the menu bar and click on the “Spreadsheet” button in the toolbar.



The “Spreadsheet” window uses Microsoft Excel. For the most stability and best performance, one should use Microsoft Excel 2016.

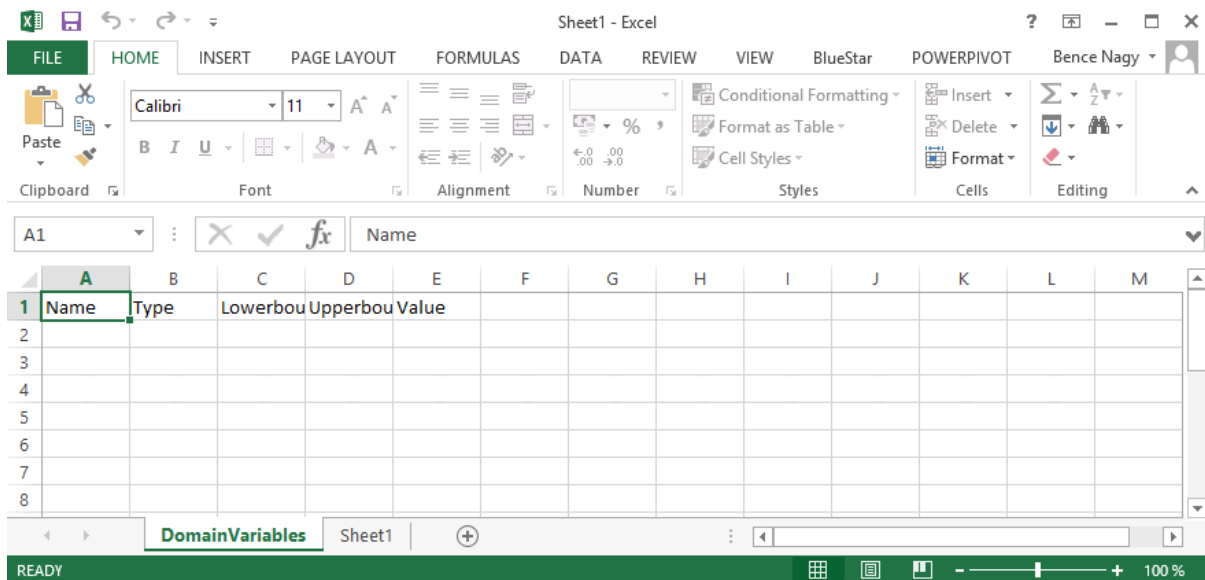


Figure 18: The "Spreadsheet" window's interface

Data from the domains are stored in the “DomainVariables” sheet.

On the image above, the table is empty, because there are no domains linked to the spreadsheet. To create a domain that sends its data to the spreadsheet, the user must check the “Sync spreadsheet” checkbox in the bottom of the window while creating a new domain.

Domain

Name:

Display name:

Mode:

Description:

Type:

Items:

Lower bound:

Upper bound:

Value:

Default value:

Value restriction:

Formula:

Visible:

Mandatory:

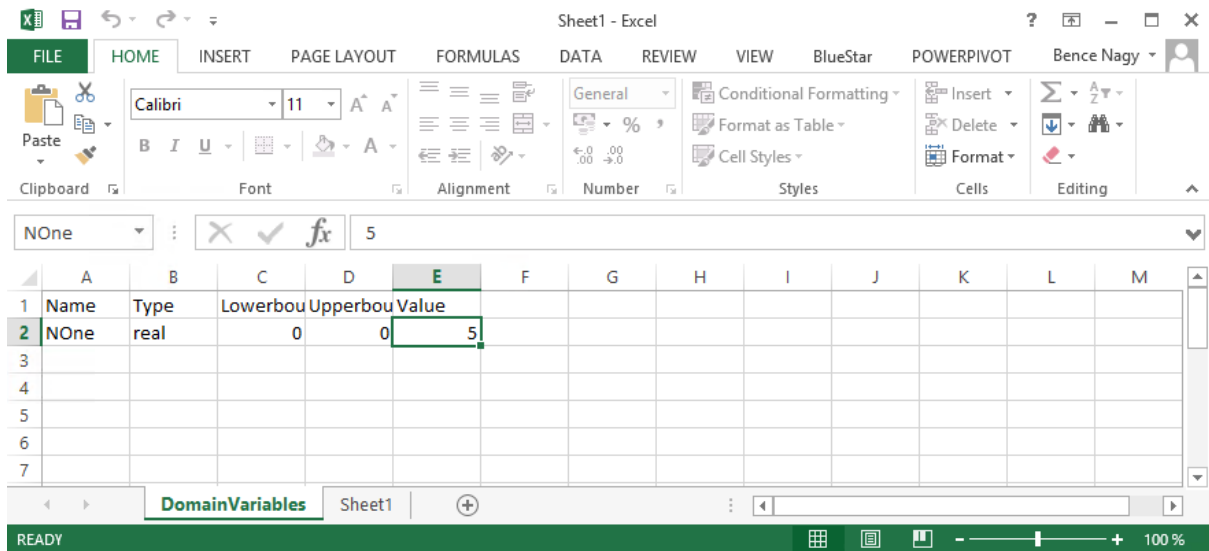
Sync spreadsheet: ☒

OK Cancel

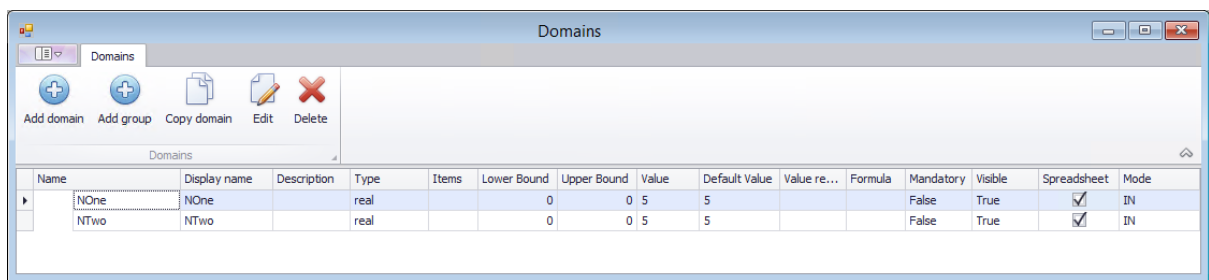
In the domain list, the domains that are synchronized with the spreadsheet have a mark in “Spreadsheet” column.

Name	Display name	Description	Type	Items	Lower Bound	Upper Bound	Value	Default Value	Value re...	Formula	Mandatory	Visible	Spreadsheet	Mode
NOne	NOne		real		0	0	5	5			False	True	☒	IN
NTwo	NTwo		real		0	0	5	5			False	True	☐	IN

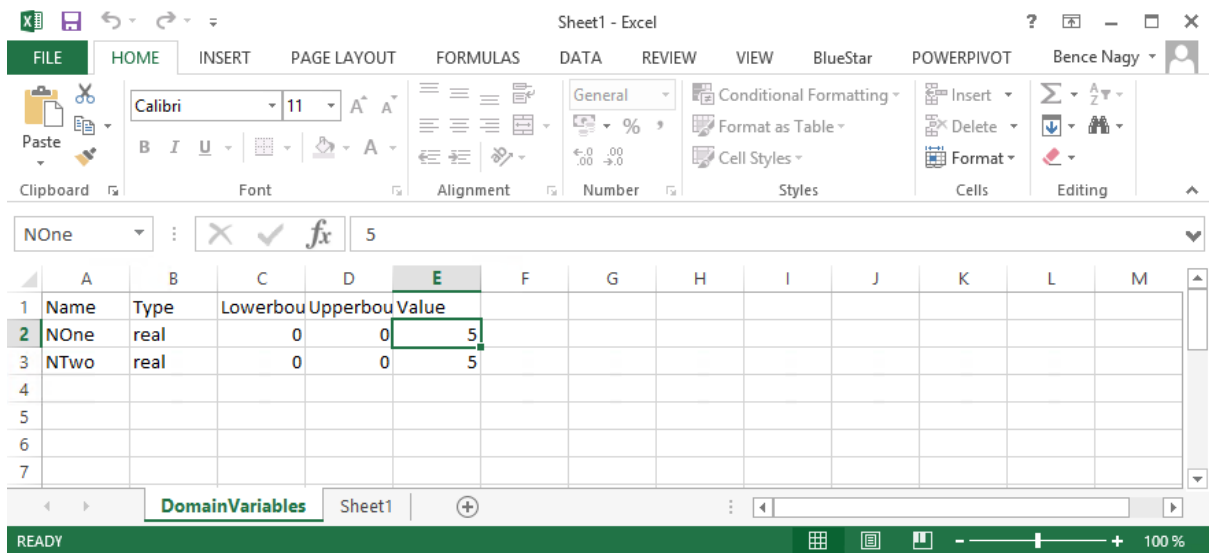
Now the spreadsheet includes data from this domain. After this, the domain’s value can be changed also in the spreadsheet, and it automatically updates the values in the “Domains” window.



The other way to make a domain linked with the spreadsheet is just to check the checkbox in the “Spreadsheet” column in the domain list.



Now the spreadsheet includes data from the two chosen domains.



Data from the spreadsheet can be also outputted to the domain. To do this, the user should create a new domain with “OUT” or “CALC” mode and synchronize it with the spreadsheet.

Domain

Name: NThree

Display name: NThree

Mode: OUT

Description:

Type: real

Items:

Lower bound: 0

Upper bound: 0

Value: 10

Default value:

Value restriction:

Formula: NOne+NTwo

Visible: True

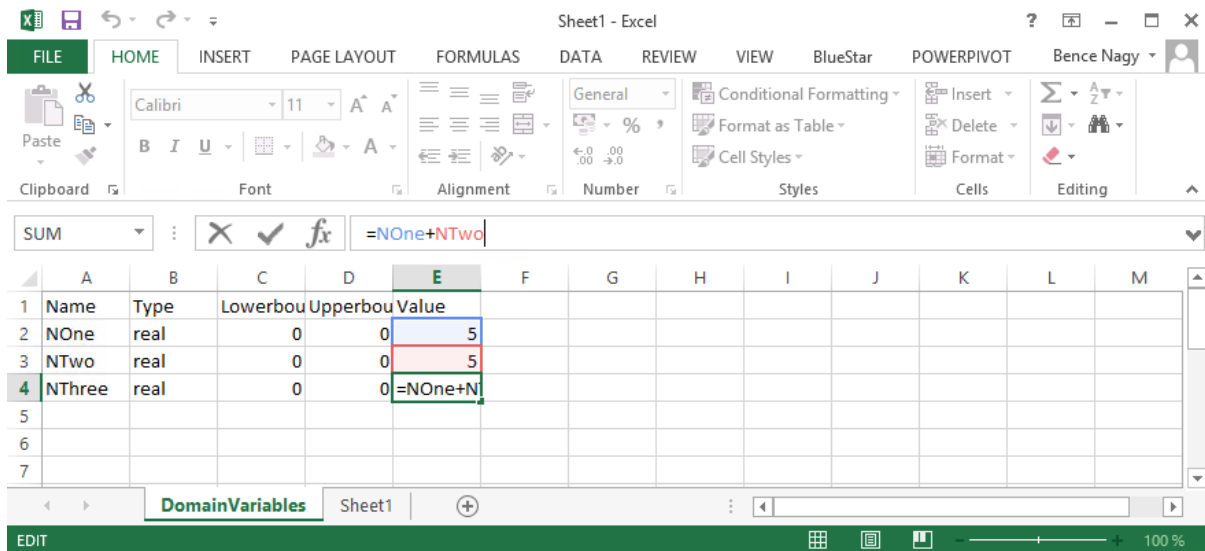
Mandatory: False

Sync spreadsheet: ☒

OK Cancel

Now the spreadsheet has a “NThree” row.

There are two ways of assigning a formula for a domain: the user can either write it in the “Formula” cell in the “Domain” window, or leave that cell empty, and assign a value to the domain using the spreadsheet, where the user should select the “Value” cell in the “NThree” row and write in the wanted Excel formula.



The spreadsheet returns the value to the domain.

Domains														
Domains														
Add domain Add group Copy domain Edit Delete														
Name	Display name	Description	Type	Items	Lower Bound	Upper Bound	Value	Default Value	Value re...	Formula	Mandatory	Visible	Spreadsheet	Mode
NOne	NOne		real		0	0	5	5			False	True	☒	IN
NTwo	NTwo		real		0	0	5	5			False	True	☒	IN
NThree	NThree		real		0	0	10			NOne+NTwo	False	True	☒	OUT

Data from the “DomainVariables” sheet can be used in other sheets of the spreadsheet. It is convenient for complicated calculations or structuring the data, because the “DomainVariables” sheet doesn’t allow any actions.

Example of using data from the domain in another sheet:

The screenshot shows an Excel spreadsheet with the following data:

	A	B	C	D	E	F	G	H	I	J	K	L
1	Name	Type	Lowerbou	Upperbou	Value							
2	NOne	real	0	0	5							
3	NTwo	real	0	0	5							
4	NThree	real	0	0	2							
5												
6												
7												

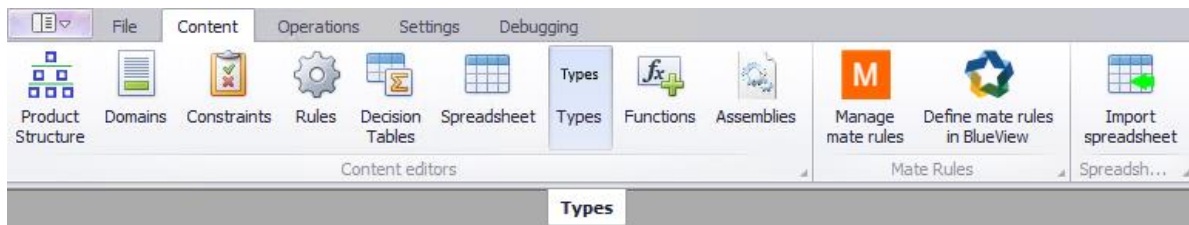
The formula bar shows the formula: `=SQRT(Sheet1!A1:A10)`. The active sheet is 'DomainVariables'.

Values from another sheet can be returned to the “DomainVariables” sheet, and then to domains.

Types

Types are used to group data within the model. For example, a type can represent the dimensions of an item in the model having fields such as height, width and length.

To open the “Types” window, the user must choose the “Content” tab in the menu bar and click on the “Types” button in the toolbar.



The “Types” window has its own toolbar, that allows the edition of the types.

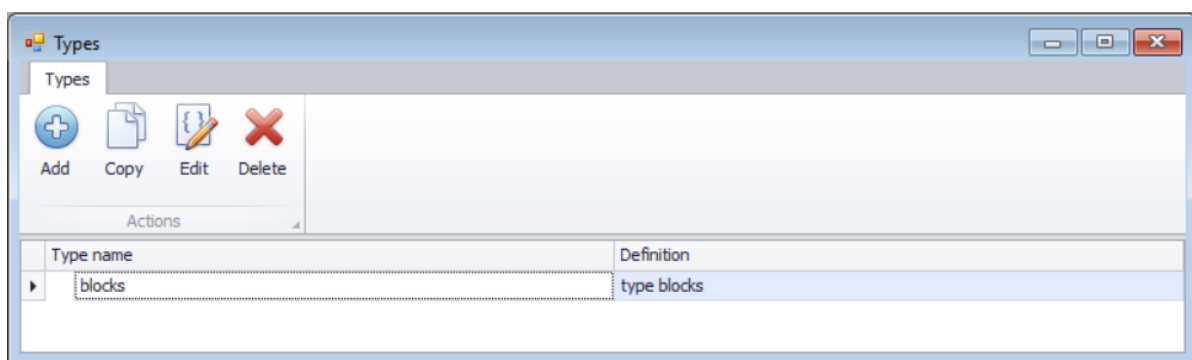
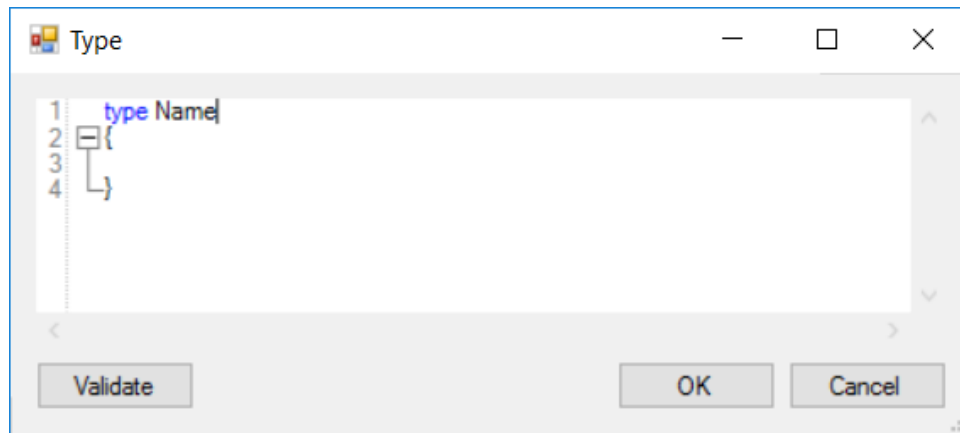


Figure 19: The "Types" window's toolbar

The toolbar consists of:

- **“Add”** – creates a new type in the types list.
- **“Copy”** – creates a new type based on an existing one.
- **“Edit”** – edits the type in the list. They also can be edited by double clicking on them.
- **“Delete”** – deletes the type from the list with all its information.

To create a new type, the user should click on the “Add” button. This button opens the “Type” window.



“Type” requires a name and a body. “Type” name can contain letters, numbers and underscore, while “Type” body contains definition of variables new Square(5,5);

Example:

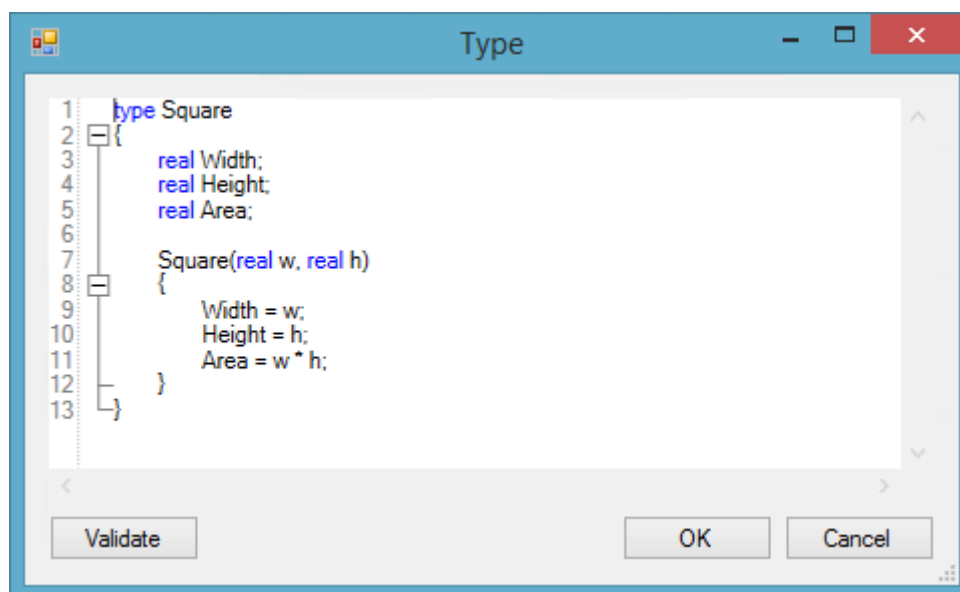


Figure 20: Type calculates square's area from width and height

```
15: new Square(5,5);
```

Functions

“Functions” in the Configurator are used to reuse functionality by encapsulating it in a function. This function can then be called throughout the model.

Example:

Angle calculation in this Configurator model is required 10 times.

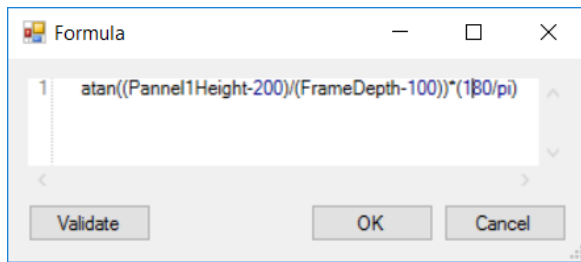


Figure 21: Angle calculation by instruction

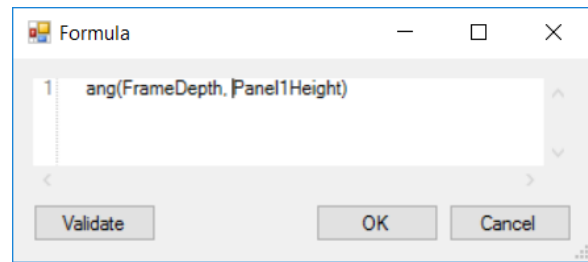
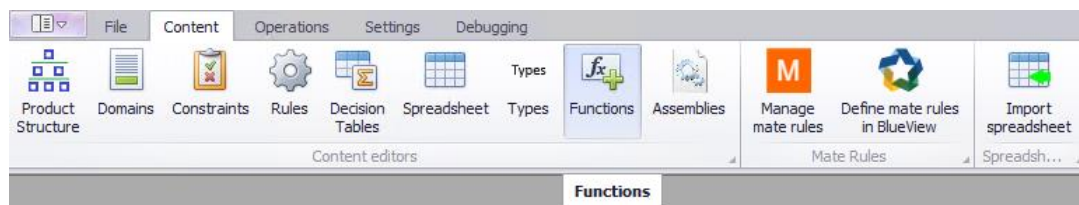


Figure 22: Angle calculation by function

Using functions obviously simplifies the code and saves a lot of time.

Functions can be used in almost any part of the Configurator model. In domains, item tree, rules, etc.

To open the “Global functions” window, the user must choose the “Content” tab in the menu bar and click on the “Functions” button in the toolbar.



The “Global functions” window has its own toolbar, that allows the edition of the functions list and the functions.

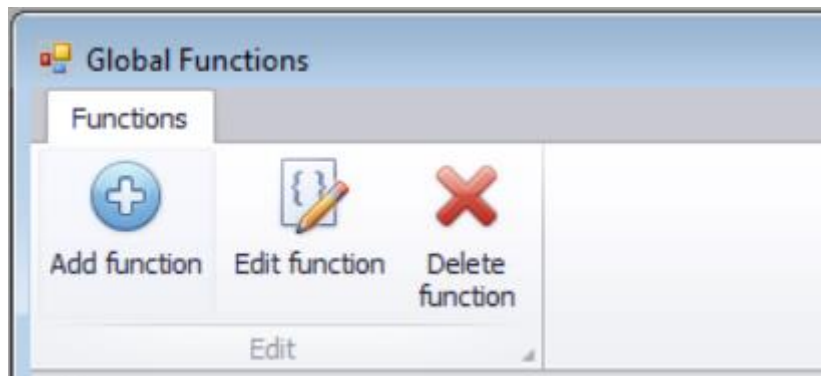
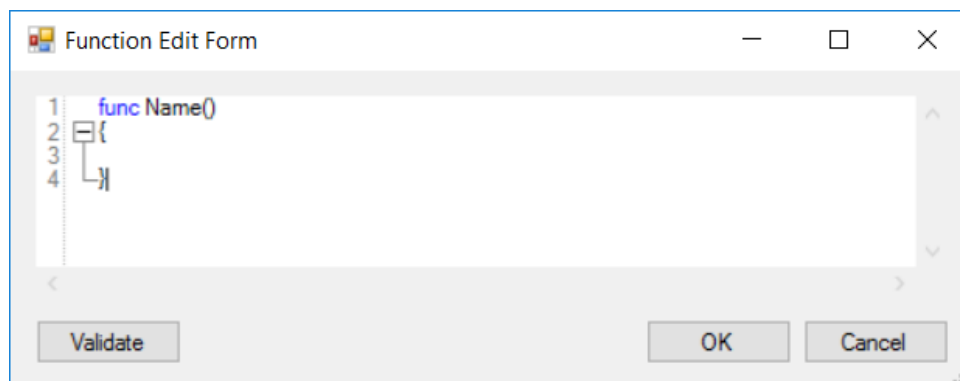


Figure 23: The “Global functions” window’s toolbar

The toolbar consists of:

- **“Add function”** – creates a new function in the function list.
- **“Edit function”** – edits functions in the list. They also can be edited by double clicking on them.
- **“Delete function”** – deletes the function from the list with all its information.

Creating a new function can be done by clicking on the “Add function” button. This button opens the “Function Edit Form” window.



New function requires a name, inputs with their types and a function body.

If the function is used in logical and mathematical calculations (returns values), the user should define the function type. It can be either “string”, “real”, “integer”, “datetime” or “Boolean”.

The keyword for the definition function type must be placed right after keyword “func”.

After that, the user should define the function name. It can contain letters, numbers and underscore.

In the brackets after the function name, the user defines the inputs of the function and their types. The function body consists of logical operations and calculations, that must be in braces.

If the function returns values, the function body must contain the keyword “return”. The returned value must have the same type as the function type.

The “Validate” button checks, if the function is correct.

Example:

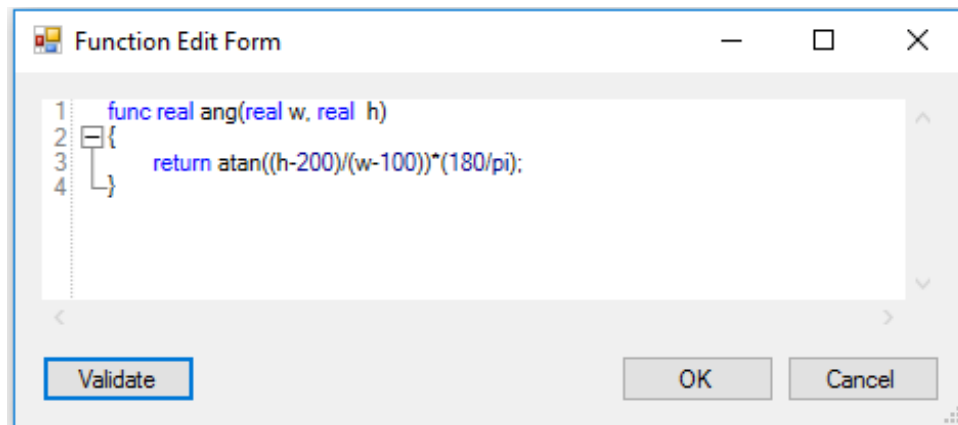
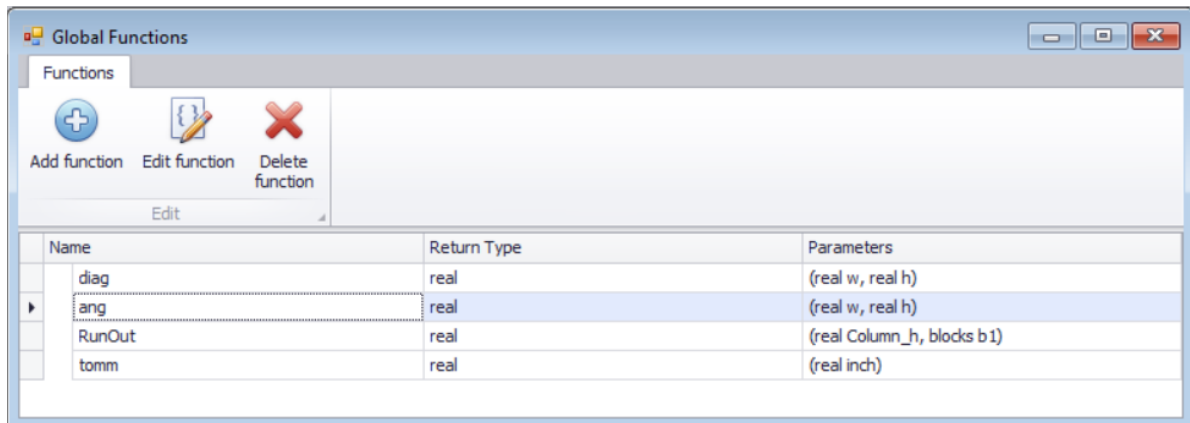


Figure 24: Function calculates arctangent from the frame's width and height and returns it with “real” type

After clicking on the “OK” button, the function appears in list in the “Global Functions” window.



Now the user can call the previously created function in almost any place in the Configurator model, using the command “ang(a, b)” where “a” and “b” are the input values. In this case function, the command is: “ang(FrameDepth, Panel1Height)”.

Example:

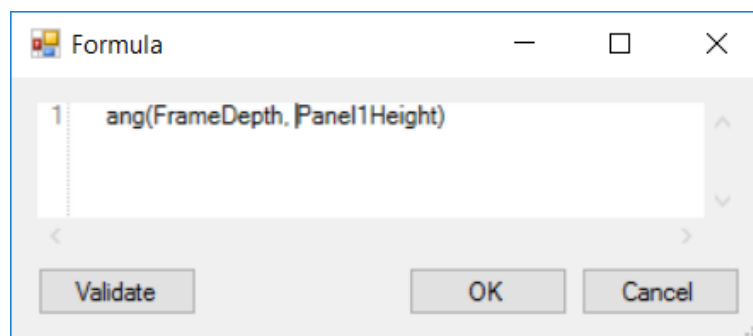


Figure 25: Formula for angle calculation in panel #1

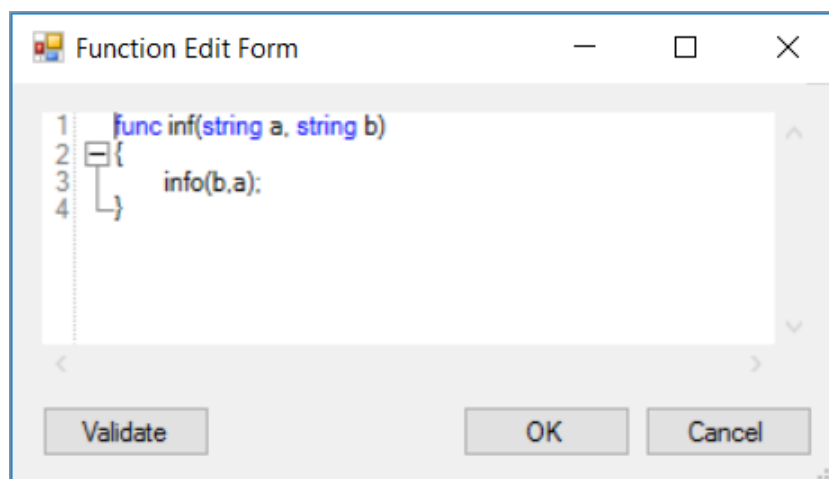
Name	Value	Formula	Value restriction
Panel1Angle	35.9059692492027	ang(FrameDepth, Panel1Height)	

Figure 26: Result of function calculation

Not all functions return values. Some functions just perform some instructions.

Example:

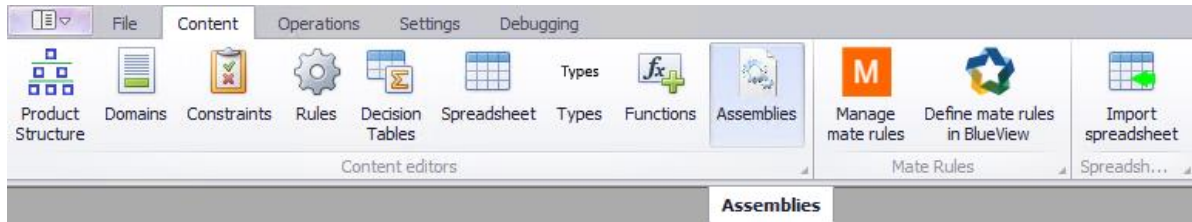
The function “inf” shows a messagebox, but does not return a value.



Assemblies

Assemblies allows the users to load external code written in a .NET language into the model. The functionality defined in this code can then be called from within the model.

To open the “Assemblies” window, the user must choose the “Content” tab in the menu bar and click on the “Assemblies” button in the toolbar.



“Assemblies” window has its own toolbar.

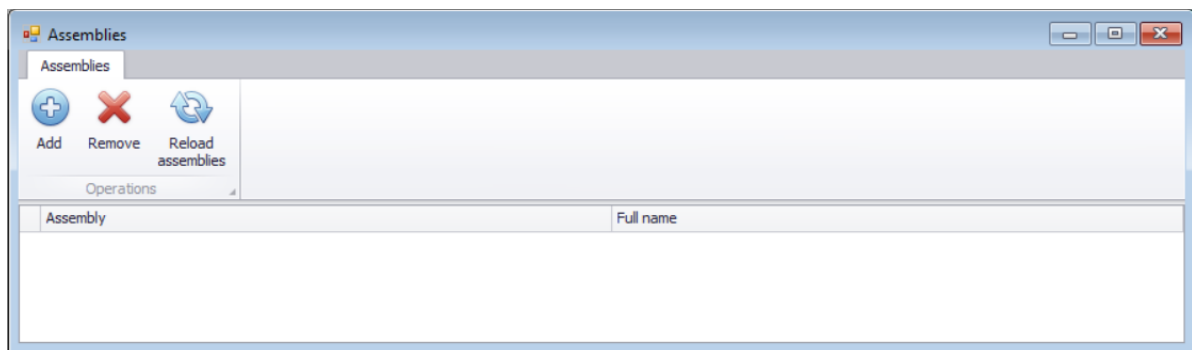


Figure 27: “Assemblies” window's toolbar

The toolbar consists of:

- **“Add”** – creates a new link in the assemblies list.
- **“Remove”** – deletes the link from the list.
- **“Reload assemblies”** – refreshes the links in the list.

To add a new assembly, the user should choose the “Add” button. This button opens the file explorer, where the user must choose the external .DLL or .EXE file.

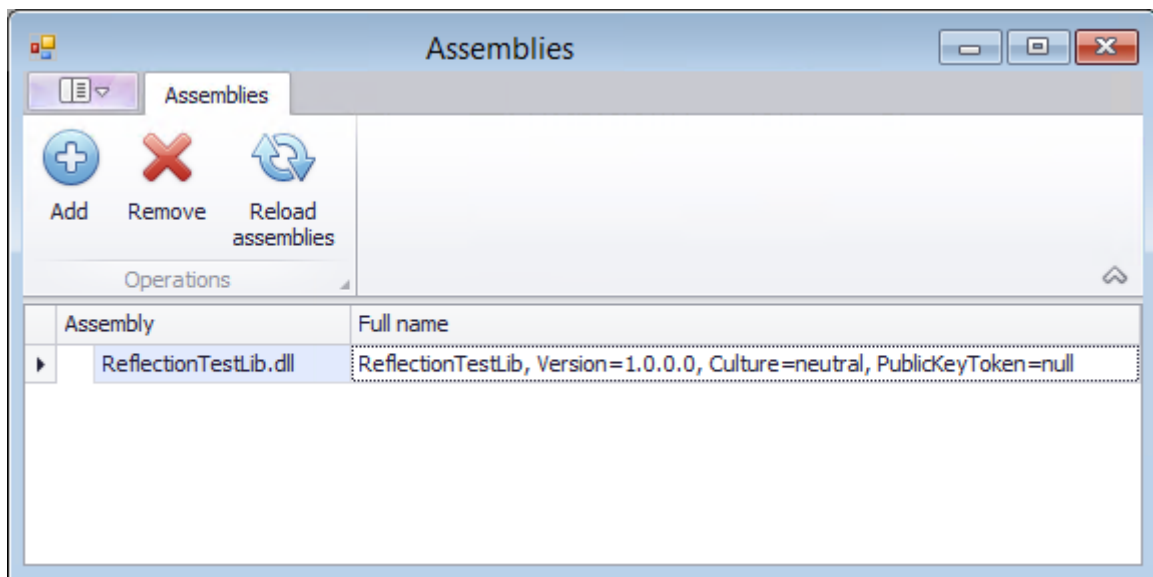
Example:

Figure 28: A called .dll file containing a reflection test

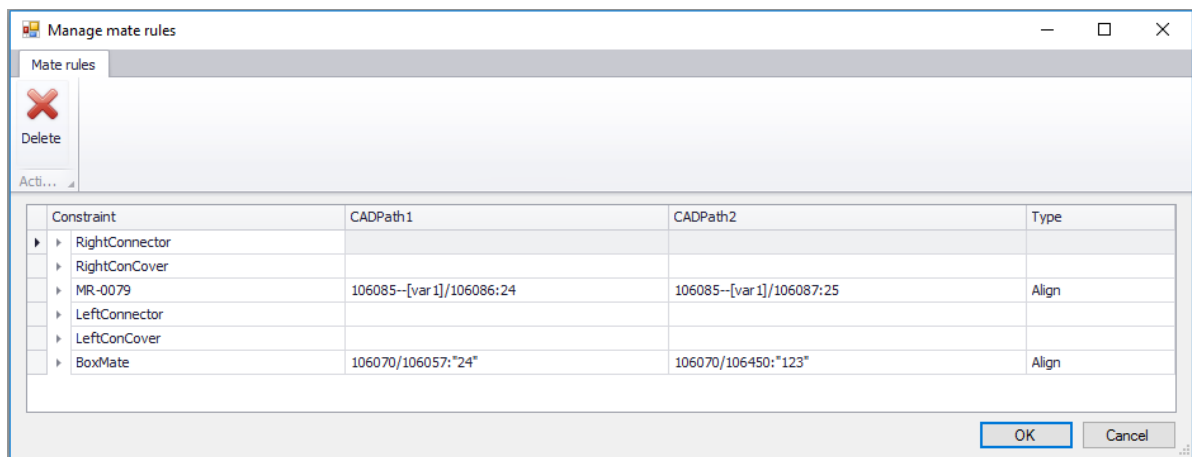
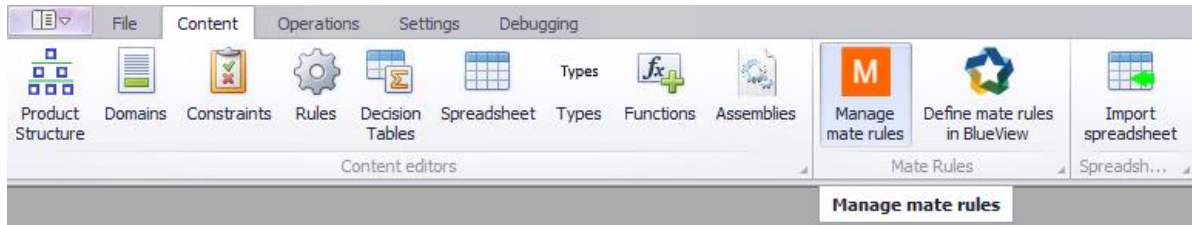


Figure 29: Using the called .dll file in a rule

This is how the script builds up in this given example, where “ReflectionTestLib.dll” is the name of the .dll file, “ReflectionTestLib.ReflectionTestClass” is the fully qualified name of the class, “Product” is the name of the method, and “5” and “7.8” are parameters.

Manage mate rules

The “Manage mate rules” button allows to open the window, where all mate rules of the model are represented.

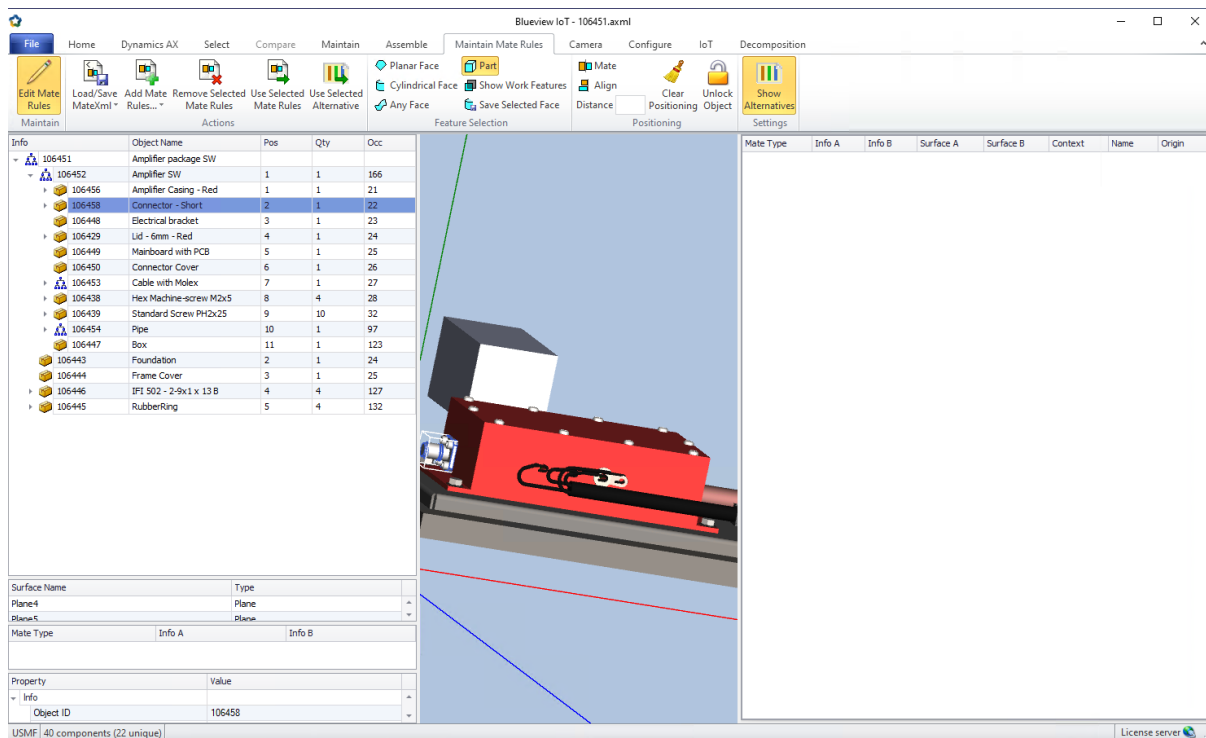


Mate rules can be deleted from the list by the “Delete” button.

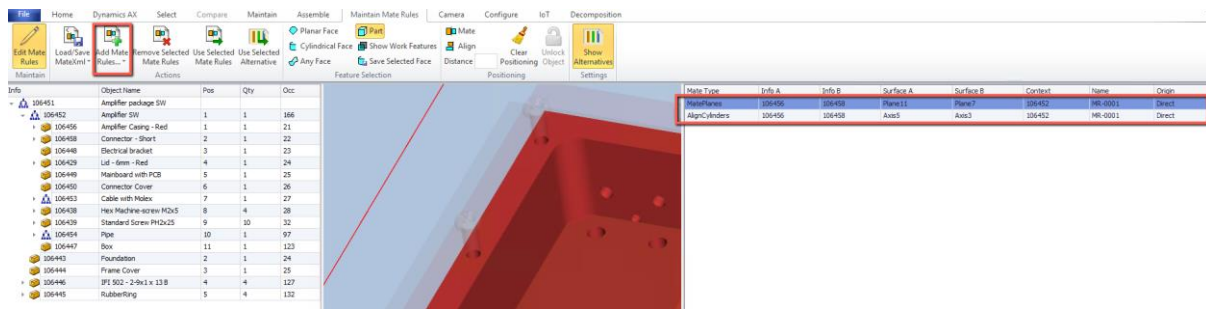
Define mate rules in Blueview

Mate rules can be either written manually in the Constraint grid (see above), or they can be defined in Blueview.

1. Click on the button “Define mate rules in Blueview”. This starts Blueview, opens there a current model and switches Blueview user interface into a mode “Configurator admin”, see below. In this mode, there are three main panels in Blueview.
 - a. A left panel shows a model tree, surfaces of a selected part, mates applied on a selected part and details of selected item.
 - b. A middle panel shows the 3d model itself
 - c. A right panel shows a list of available mate rules. These mate rules were either defined by a user in Blueview, or comes from a touch analysis of the 3d model.



- Now we want to define a new mate rule. Let's say we want to define it for a connector and a casing. A good practice is to select two components, for which a mate rule is to be defined (you can multi-select using Ctrl key), do a right click on one of selected components and click "Isolate selection" (that hides all non-selected components).
- Right click on a component, which should be moved by the new mate rule (the connector in our example) and click "Unlock object". The button is also available in the top ribbon menu, in Maintain Mate rules – Positioning. Note that in Blueview, a component must be unlocked before it can be moved.
- Select two work-features, one on each part to be mated (work-features are defined in a source CAD system and are exported in Blueview files on checkin). A work-feature can be selected in the left panel, in the list of surfaces. Or, work-features can be shown in the 3d model. To show them there, click a button "Show Work features" in the top ribbon menu – Maintaining Mate rules – Feature Selection. Once shown, a work-feature in the 3d model can be selected by a left mouse button (use Ctrl key for multi-select).
- When two work-features are selected, add a constraint – press either a button Mate or Align in the top ribbon menu – Maintain Mate Rules – Positioning.
- Optionally, repeat the steps 4 and 5 for different pairs of work-features to add more constraints on the two components.
- Now when the constraints have been added, let's create a Mate rule from them.
 - Make sure the two components are selected.
 - Expand a drop-down button "Add Mate Rules" in the menu Maintain Mate Rules – Actions and click "From selection". A new mate rule appears in the right panel (see below).



8. Take a look on the new mate rule in the right panel. Each constraint, which was added, corresponds to one line of the mate rule. An automatic name was assigned to the mate rule - notice that it is the same for all lines. The automatic name can be (should be) changed to something meaningful. Just click in the column Name on one of the lines of the new mate rule and edit the name there. Names of other lines will be updated automatically (see below).

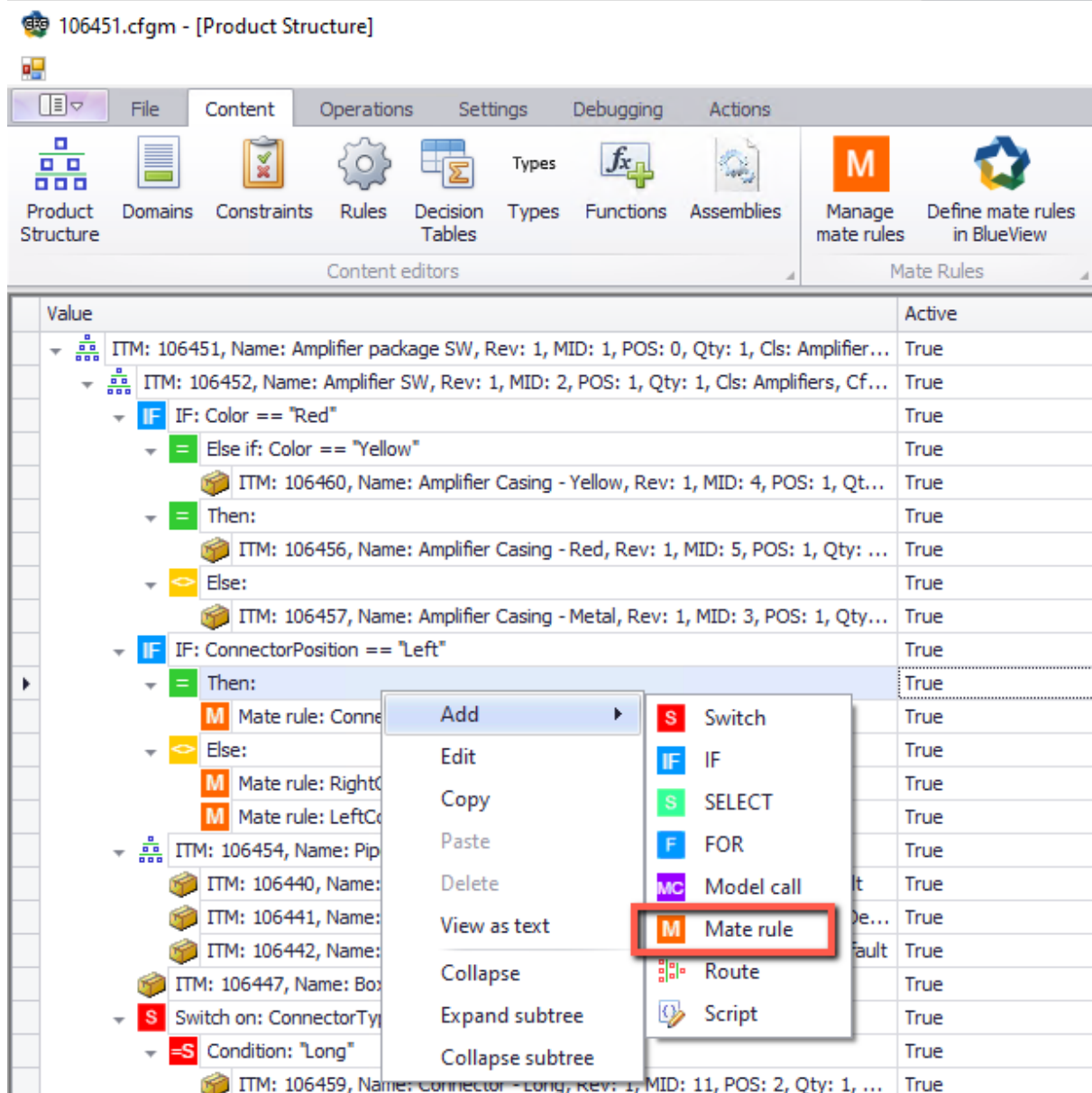
Mate Type	Info A	Info B	Surface A	Surface B	Context	Name	Origin
MatePlanes	106456	106458	Plane11	Plane7	106452	ConnectorTop	Direct
AlignCylinders	106456	106458	Axis5	Axis3	106452	ConnectorTop	Direct

9. A new mate rule in Blueview has been prepared. To use it in the configurator, do the following:
- In 3d model, do a right click and select "Basic view". All components will be shown. Now select again the two components, on which the new mate rule should be applied. This selection is important. Without this, the mate rule might not work correctly.
 - Right click on the new mate rule in the right panel and click "Send to Configurator" (see below).

Mate Type	Info A	Info B	Surface A	Surface B	Context	Name	Origin
MatePlanes	106456	106458	Plane11	Plane7	106452	ConnectorTop	Direct
AlignCylinders	106456	106458	Axis5	Axis3	106452	ConnectorTop	Direct

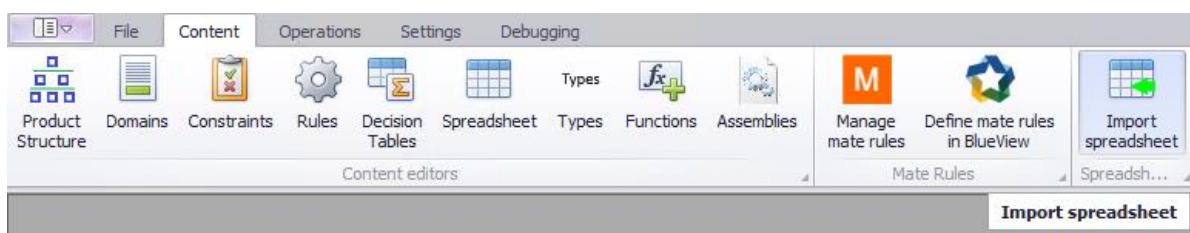
Copy Mate Rule
 Edit Mate Rule
 Send to Configurator

10. If more mate rules are to be defined for already constrained components, their current constraints must be cleared first. Select the components and press a button "Clear positioning" in the menu Maintain Mate Rules – Positioning.
11. Once all necessary mate rules have been defined in Blueview and sent to Configurator, close Blueview and go back to Configurator.
12. In Configurator, press a button "Manage mate rules" in the tab Content. New mate rules are available there.
13. To use a mate rule in the product structure, right click a proper node in the product structure (e.g. IF node) and select Add – Mate rule.



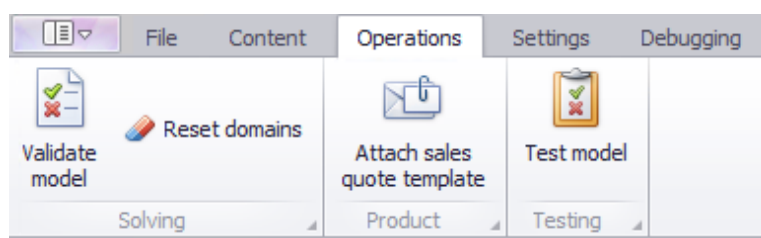
Import Spreadsheet

The “Import Spreadsheet” button allows the user to import spreadsheets to the model from outside the program. After importation, these spreadsheets can be called by the “Spreadsheet” button, as it was already described at the “Spreadsheet” section.



Operations

The “Operations” tab includes three groups: “Solving”, “Product” and “Testing”.



Solving

The “Solving” group consists of two buttons: “Validate model” and “Reset domains”.

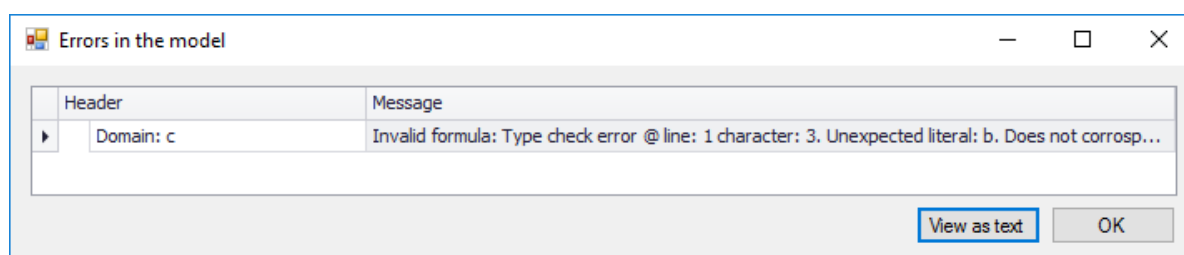
- **“Validate model”** – checks if the model has any errors.
- **“Reset domains”** – resets all domains in the model to default values (makes values in the “value” cells equal to the values in the “default value” cells).

Example for the usage of the “Validate model” button

The model has three domains: “a”, “b” and “c”. Domain “c” returns the sum of “a” and “b”.

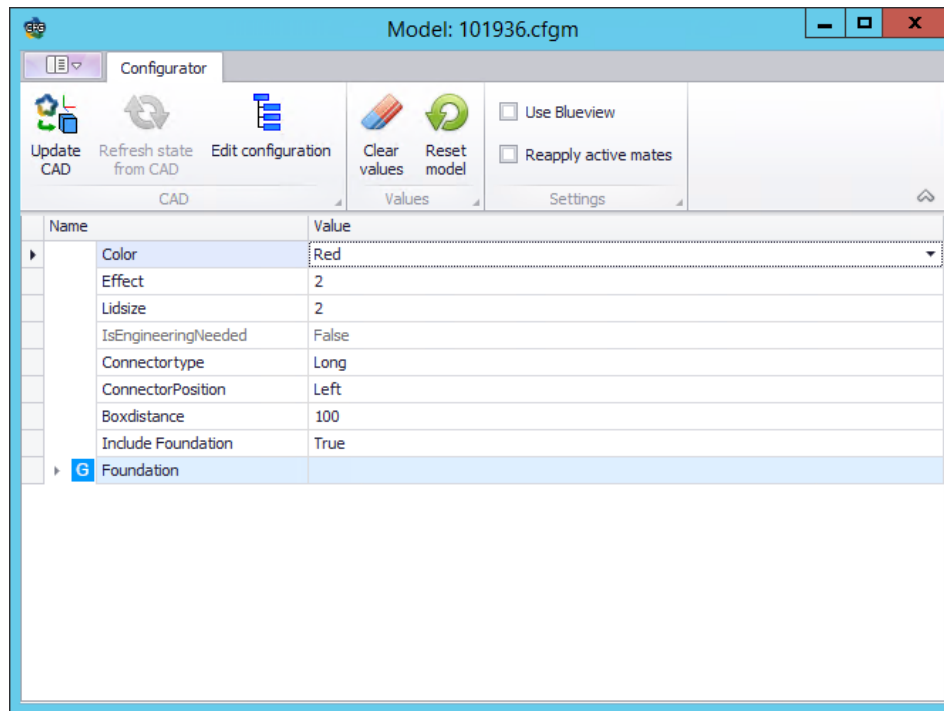
	Name	Formula	Value
	a		1
I	b		5
	c	a+b	6

In this case, clicking on the “Validate model” button causes nothing, but if domain “b” is deleted, then the “Validate model” button will open a window with an error message.



Testing

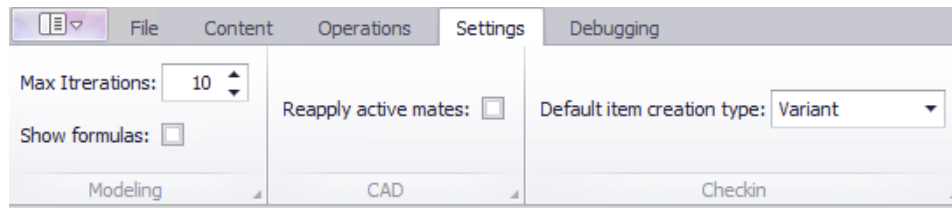
The “Testing” group consists of a “Test model” button. This button opens the “Model” window, that allows the user to test the model.



This window represents all visible domains of the model. As the testing toolbar is nearly identical to the Configurator Interface, more can be found about the testing functions at the section about the “Execution of Configurator model”.

Settings

“Settings” tab includes three groups: “Modeling”, “CAD” and “Checkin”.



In the “Modelling” group, the user can choose the number of iterations for the process of calculating and building the model. Usually, more than one iterations are required to build the CAD model. Some values are changing in the process of calculations, so the Configurator starts the program from the beginning to apply new values. If nothing happened during the iteration, the cycle will be interrupted. If there are some changes in it, each iteration cycle will be interrupted after the number of iterations that is inputted in the “Max Iterations” box.

In the CAD group, the “Show formulas” checkbox allows to display formulas instead of their results.

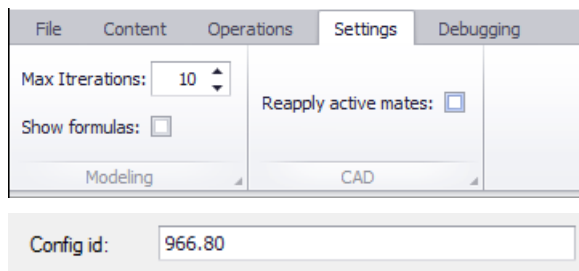


Figure 30: Showing the config ID's result

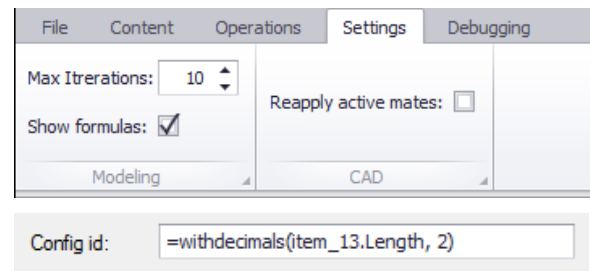


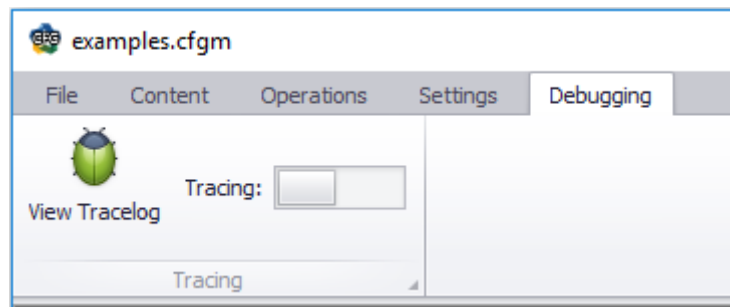
Figure 31: Showing the config ID's formula

The “Reapply active mates” checkbox deletes the mates and reapplies them every time the CAD model is updating. It can be necessary in some cases and in some CAD systems.

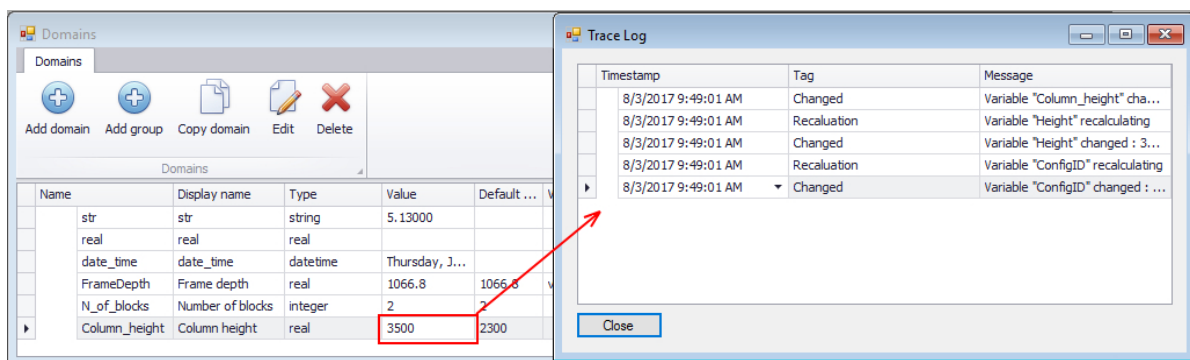
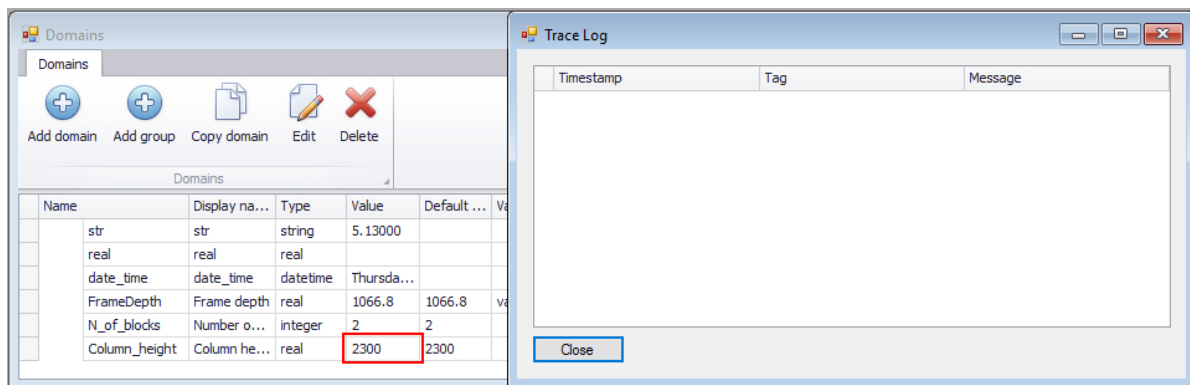
In the “Checkin” group, the “Default item creation type” list defines the default type of the created items, that can be either set to be a “Variant”, or an “Uniqueobject”.

Debugging

The “Debugging” tab includes “View Tracelog” button and “Tracing” switch.



When “Tracing” is switched on, all changes in the domains and item attributes are saved in the trace log.



Configurator programming language

The syntax of the configurator's language is based on C. It allows the user to perform mathematical and boolean operations, operations with text, etc.

Operators

Addition

"+" can be used for the addition of numbers and concatenation (addition) of strings. Addition is performed for the datatypes "integer" and "real", while the "+" operator performs concatenation whenever the left argument is a string.

Name	Type	Formula	Value
I	int	integer	5+6
			11

Figure 32: Integer to integer addition

Name	Type	Formula	Value
I	real	real	5.4+6.6
			12

Figure 33: Real to real addition

Name	Type	Formula	Value
	real	real	5.4+int
I	int	integer	5
			10.4
			5

Figure 34: Real to integer addition

Name	Type	Formula	Value
I	str	string	"five "+"panels"
			five panels

Figure 35: String to string concatenation

Name	Type	Formula	Value
I	str	string	"Panel "+5
			Panel 5

Figure 36: String to number concatenation

Name	Type	Formula	Value
	str	string	tostr(int)+" "+"panels"
	int	integer	5
			5 panels
			5

Figure 37: Integer converted into string to string concatenation

Subtraction

"-" can be used for the subtraction of numbers and for defining negative numbers.

	Name	Type	Formula	Value
I	real	real	5.4- 2.954 	2.446

Figure 38: Subtracting real from real

	Name	Type	Formula	Value
I	real	real	int-2.45 	2.55
	int	integer	5	5

Figure 39: Subtracting real from integer

Multiplication

“*” is used for the multiplication of numbers.

	Name	Type	Formula	Value
I	real	real	5*5 	25

Figure 40: Multiplying real by real

Division

“/” is used for the division of numbers.

	Name	Type	Formula	Value
I	real	real	5/4 	1.25

Figure 41: Dividing real by real

Modulus (integer remainder)

“%” is used to find the remainder after integer division.

	Name	Type	Formula	Value
I	real	real	13%5	3

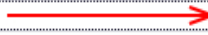


Figure 42: Calculating the modulus of a real by real division

Increment

“++” is used for incrementing the integer value by one. It can be represented by pre-increment (++A) or by post-increment (A++).

“++A” pre-increment increases the value of A by one and then uses A in an expression.

“A++” post-increment uses A in an expression and then increases the value of A by one.

Decrement

“--” is used for decrementing the integer value by one. It can be represented by pre-decrement (--A) or by post-decrement (A--).

“--A” pre-increment decreases the value of A by one and then uses A in an expression.

“A--” post-decrement uses A in an expression and then increases the value of A by one.

Increment and decrement are usually used in “for” construction for counting iterations (cycles).

Arrays

Arrays can be used to store values. They can contain different types of values.

“Arrays” can be created in two ways:

- `ArrayName = new real [5];` – declaration of an empty array of five elements.
- `ArrayName = new real [] {1,2,3,4,5};` – declaration of an array of five elements, with values 1, 2, 3, 4 and 5.

The array's elements can be called with the expression `ArrayName[x]`, where “x” is number of the element in the array. The enumeration of the elements starts from zero.

Example:

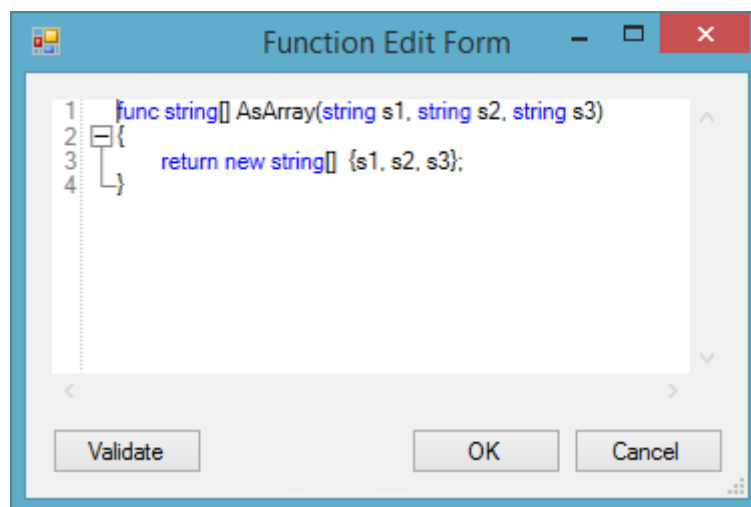


Figure 43: Creating an array

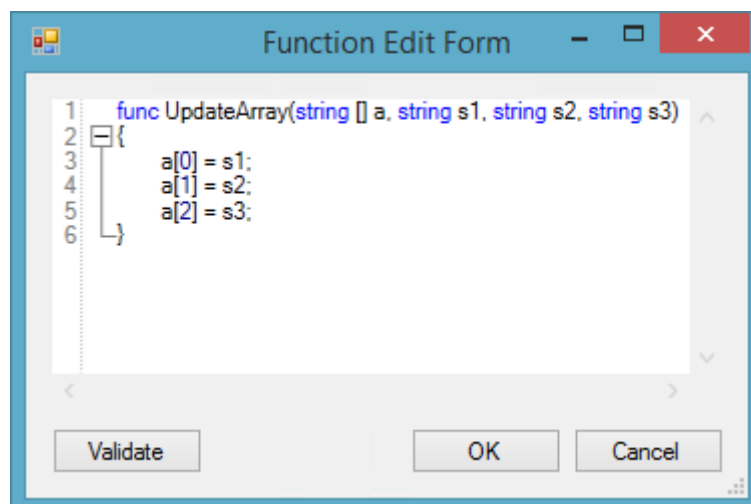


Figure 44: Assigning values to the array

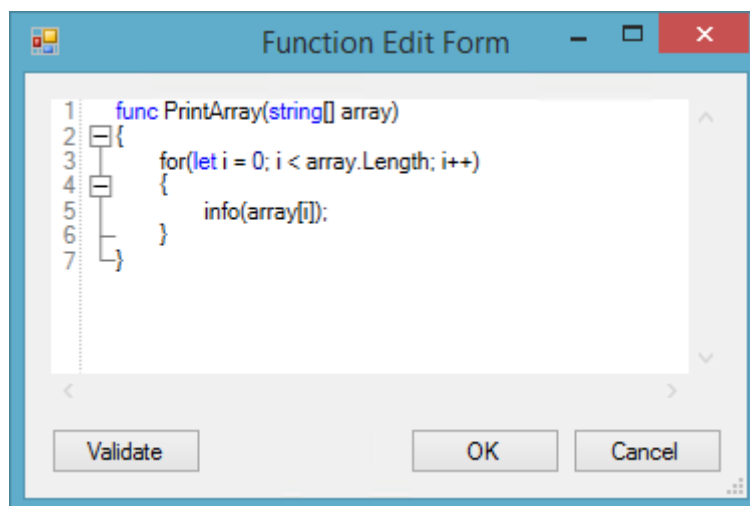


Figure 45: Array iteration

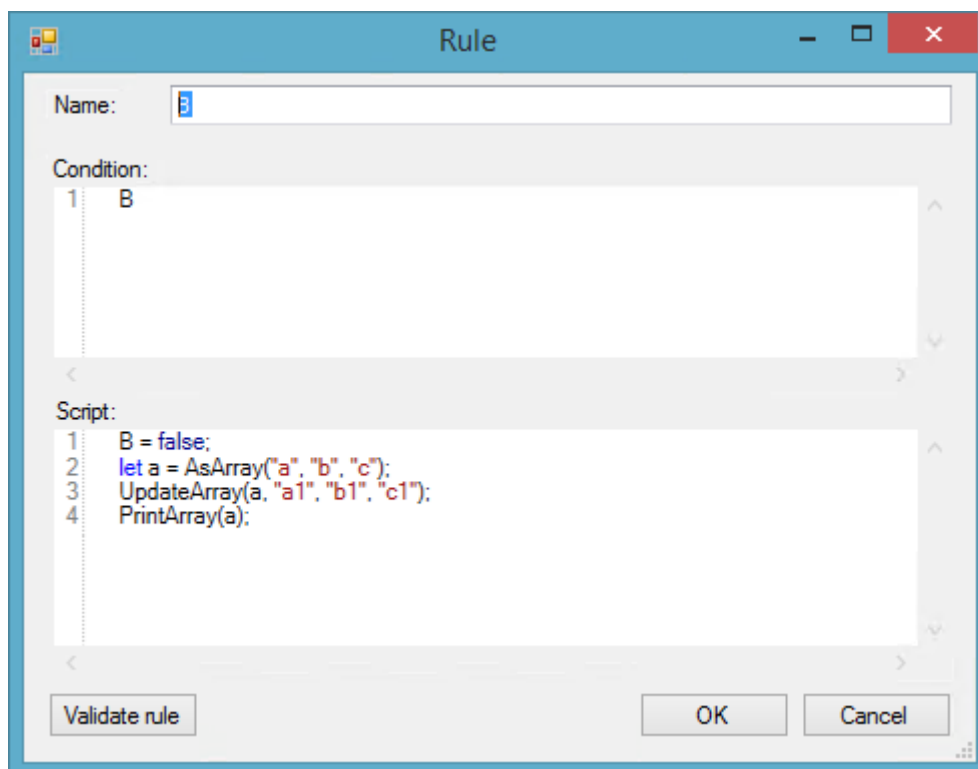


Figure 46: Using the array in a rule

Boolean operations

Boolean expressions are logical expressions, that return only True or False values. Usually, they are used to check, if conditions are satisfied. They are always used in such constructions as “if-else”, “switch-case”, “for”, etc.

Boolean expressions can include different types of values, mathematical and logical operations.

Logical operators

- “&&” – and. Returns true, when the left and right parts of the expression are true.
- “||” – or. Returns true, when the left or right parts of the expression is true.
- “xor” – exclusive or. Returns true, when the left and the right parts of the expression have different Boolean values.
- “->” – implication. Checks the right part of the expression if the left part is true. Returns true, if the checked part is true (both parts are true).

Examples of using logical operators:

Operator “and” (&&):

true	&&	true	=	true
true	&&	false	=	false
false	&&	true	=	false
false	&&	false	=	false

Operator “or” (||):

true		true	=	true
true		false	=	true
false		true	=	true
false		false	=	false

Operator “exclusive or” (xor):

true	xor	true	=	false
true	xor	false	=	true
false	xor	true	=	true
false	xor	false	=	false

Operator “implication” (->):

true	->	true	=	true
true	->	false	=	false
false	->	true	=	false
false	->	false	=	true

Boolean relations

- “==” – equal.
- “<”, “>” – less, more.
- “<=”, “>=” – less or equal, more or equal.
- “!=” – not equal.

Examples of Boolean relations:

Equal (==):

true	==	true	=	true
true	==	false	=	false
5	==	6	=	false
5	==	5	=	true
“abc”	==	“abc”	=	true
“ABC”	==	“abc”	=	true

Not equal (!=):

true	!=	true	=	false
true	!=	false	=	true
5	!=	6	=	true
5	!=	5	=	false
“abcd”	!=	“abc”	=	true
“ABC”	!=	“abc”	=	false

Less, more (<, >):

1	<	2	=	true
5.1	<	5	=	false
1	>	1	=	false
“a”	<	“b”	=	true
“A”	>	“b”	=	false
“B”	<	“a”	=	false

Less or equal, more or equal (<=, >=):

1	<=	2	=	true
5.1	<=	5	=	false
1	>=	1	=	true
“a”	>=	“b”	=	false
“a”	>=	“a”	=	true
“a”	>=	“A”	=	true

Built-in functions

Built-in functions are used for such operations as calculating logarithms, absolute values, exponentiation, changing value types, searching elements in arrays, etc.

Function “ln()”

Calculates natural logarithm.

	Name	Type	Value	Formula
I	real	real	1.94591014905531	ln(4+3)
I	real	real	1.94591014905531	ln(7)

Function “log()”

Calculates decimal logarithm.

	Name	Type	Value	Formula
I	real	real	0.778151250383644	log(6)
I	real	real	0.778151250383644	log(4+2)

Function “abs()”

Returns absolute value of a number.

	Name	Type	Value	Formula
I	real	real	3	abs(-3)
I	real	real	4	abs(4)
I	real	real	8	abs(4 - 12)

Function “toint()”

Changes value type to integer.

	Name	Type	Value	Formula
I	int	integer	4	toint(4.5)
I	int	integer	5	toint(4.5001)
I	int	integer	123	toint("123")
I	int	integer	-123	toint("-123")
I	int	integer	123456	toint("123"+"456")
I	int	integer	-123	toint("-" + "123")
I	int	integer	4	toint(11/3)

Function “tostring()”

Changes value type to string.

	Name	Type	Value	Formula
I	str	string	45	tostring(45)
I	str	string	45.847	tostring(45.847)
I	str	string	2.14285714285714	tostring(15/7)

Function “tochr()”

Changes value type to char. If function argument consists of more, than one character function, it returns the first.

Name	Type	Value	Formula
I	str	string	2
			tochr(24)
Name	Type	Value	Formula
I	str	string	a
			tochr("a")
Name	Type	Value	Formula
I	str	string	e
			tochr("example")
Name	Type	Value	Formula
I	str	string	3
			tochr(5+27)
Name	Type	Value	Formula
I	str	string	3
			tochr(10/3)
Name	Type	Value	Formula
I	str	string	-
			tochr(-5.2)

Function “substr()”

Extracts part of the string. Arguments of the function are the string, index of the char in the string that will be the first char of the extracted part, length of the extracted part.

Name	Type	Value	Formula
I	str	string	abc
			substr("abcdefgh", 0, 3)
Name	Type	Value	Formula
I	str	string	cdef
			substr("abcdefgh", 2, 4)
Name	Type	Value	Formula
I	str	string	5-8
			substr("45-86", 1, 3)

Function “lwc()”

Makes all string characters lowercase.

Name	Type	Value	Formula
I	str	string	exampletext
			lwc("exampletext")
Name	Type	Value	Formula
I	str	string	exampletext
			lwc("ExampleText")
Name	Type	Value	Formula
I	str	string	exampletext
			lwc("EXAMPLETEXT")

Function “upc()”

Makes all string characters uppercase.

	Name	Type	Value	Formula
I	str	string	EXAMPLETEXT	upc("exampleText")
I	str	string	EXAMPLETEXT	upc("ExampleText")
I	str	string	EXAMPLETEXT	upc("EXAMPLETEXT")

Function “toreal()”

Changes value type to real.

	Name	Type	Value	Formula
I	real	real	5	toreal(5)
I	real	real	5.1	toreal("5.1")
I	real	real	0	toreal(" ")

Function “len()”

Returns the length of the string.

	Name	Type	Value	Formula
I	int	integer	8	len("abcde"+"fgh")
I	int	integer	5	len("abcde")
I	int	integer	4	len("12.4")

Function “arraylength()”

Returns the length of array (number of array elements).

	Name	Type	Value	Formula
I	int	integer	12	arraylength(new int[12])
I	int	integer	4	arraylength(new int[] {1,2,3,4})

Function “date()”

Inputs date and time. The date and time must be inputted as arguments of the function in (YYYY, MM, DD, hh, mm, ss) form.

	Name	Type	Value	Formula
I	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “year()”

Extracts the year value from the date.

	Name	Type	Value	Formula
I	real	real	2017	year(date_time)
	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “month()”

Extracts the month value from the date.

	Name	Type	Value	Formula
▶	real	real	6	month(date_time)
	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “day()”

Extracts the day value from the date.

	Name	Type	Value	Formula
I	real	real	29	day(date_time)
	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “hour()”

Extracts the hour value from the time.

	Name	Type	Value	Formula
I	real	real	16	hour(date_time)
	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “minute()”

Extracts the minute value from the time.

	Name	Type	Value	Formula
I	real	real	35	minute(date_time)
	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “second()”

Extracts the second value from the date.

	Name	Type	Value	Formula
I	real	real	15	second(date_time)
	date_time	datetime	Thursday, June 29, 2017 4:35:15 PM	date(2017, 6, 29, 16, 35, 15)

Function “withdecimals()”

Defines how many decimal signs of the real number will be displayed. This function returns string values.

	Name	Type	Value	Formula
I	str	string	5.123	withdecimals(5.123456789, 3)
I	str	string	5.12346	withdecimals(5.123456789, -5)
I	str	string	5.13000	withdecimals(5.13, 5)

Function “strsw()”

Checks, if the beginning of the string in the first argument equals to the string in the second argument. Returns True or False values.

	Name	Type	Value	Formula
I	bool	boolean	True	strsw("abcdef", "")
I	bool	boolean	False	strsw("abcdef", " ")
I	bool	boolean	True	strsw("abcdef", "abc")
I	bool	boolean	True	strsw("abcdef", "abcdef")
I	bool	boolean	False	strsw("abcdef", "abcdefgh")
I	bool	boolean	False	strsw("abcdef", "def")

Function “strew()”

Checks, if the end of the string in the first argument equals to the string in the second argument. Returns True or False values.

	Name	Type	Value	Formula
I	bool	boolean	True	strew("abcdef", "")
I	bool	boolean	False	strew("abcdef", " ")
I	bool	boolean	True	strew("abcdef", "def")
I	bool	boolean	True	strew("abcdef", "abcdef")
I	bool	boolean	False	strew("abcdef", "aabcdef")
I	bool	boolean	False	strew("abcdef", "abc")

Function “strcontains()”

Checks, if the string in the first argument contains the string in the second argument. Returns True or False value.

	Name	Type	Value	Formula
I	bool	boolean	False	strcontains("abcdef", " ")
I	bool	boolean	True	strcontains("abc def", " ")
I	bool	boolean	True	strcontains("abcdef", "abc")
I	bool	boolean	True	strcontains("abcdef", "cd")
I	bool	boolean	False	strcontains("abcdef", "xcd")

Function “match()”

Returns the index of the searched element in array. The first argument must contain the searched element, and the second argument must contain the array. If the searched element cannot be found in the array, the function returns -1 as a value.

Name	Type	Value	Formula
I	int	2	match(3, new real []{1, 2, 3, 4, 5})
Name	Type	Value	Formula
I	int	0	match(1, new real []{1, 2, 3, 4, 5})
Name	Type	Value	Formula
▶	int	2	match("c", new string[]{"a", "b", "c", "d", "e"})
Name	Type	Value	Formula
I	int	0	match("a", new string[]{"a", "b", "c", "d", "e"})
Name	Type	Value	Formula
I	real	-1	match(55, new real []{1, 2, 3, 4, 5})
Name	Type	Value	Formula
I	int	-1	match("x", new string[]{"a", "b", "c", "d", "e"})

Function “sign()”

Used to check if the given number is positive or negative. For positive number, it returns 1, for negative -1, and for null returns 0.

Name	Type	Value	Formula
I	int	1	sign(3.141592)
Name	Type	Value	Formula
I	int	-1	sign(-2)
Name	Type	Value	Formula
I	int	0	sign(0)

Function “IF()”

Function “IF()” is a short version of the “if - else” construction. It has three arguments: the first is the condition (Boolean expression), the second contains the value that is returned if the condition is true, and the value from the third argument is returned if the condition is false.

Name	Type	Value	Formula
I	str	yes	IF(1=1, "yes", "no")

Name	Type	Value	Formula
I	str	no	IF(1=2, "yes", "no")

Name	Type	Value	Formula
I	real	7	IF(7>5, 7, 5)

Name	Type	Value	Formula
I	real	5	IF(false, 7, 5)

Name	Type	Value	Formula
I	real	7	IF(true, 7, 5)

Function “dbl()”

Extracts data from the decision tables. The first argument defines the decision table, the second defines the record, and the third defines the column.

The screenshot shows the 'Decision Tables' window. The 'Current table' dropdown is set to 'A1'. The table has two columns: 'Column' and 'Horizontal'. The 'Records' are 'Steel' and 'Size'. The 'Columns' are 'Column', 'Horizontal', and 'Diagonal'. The values are: Steel: EN 10028-2, Horizontal: EN 10028-2, Diagonal: EN 10028-2; Size: 75*75, Horizontal: 60*60, Diagonal: 60*60.

Name	Type	Value	Formula
I	str	EN 10028-2	dbl("A1", "Steel", "Horizontal")

Name	Type	Value	Formula
I	str	60*60	dbl("A1", "Size", "Diagonal")

Statements

Statements are the smallest standalone elements of the programming language. Statements are instructions for some special actions.

Declaration

To declare a variable, the user should use the keyword “let” before the variable name and define the variable value. The variable type will be inferred automatically from the initial value.

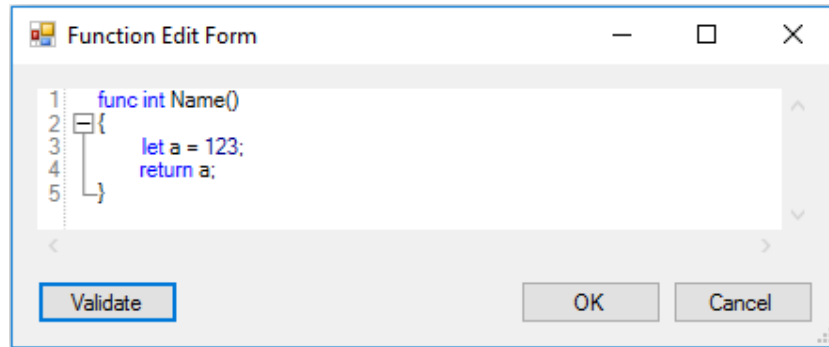


Figure 47: Variable “a” is declared as an integer and returned

	Name	Type	Value	Formula
I	int	integer	123	Name()

Figure 48: Variable “a” is returned by function Name () as an integer

To declare an array, the user must define the array length or each elements of the array.

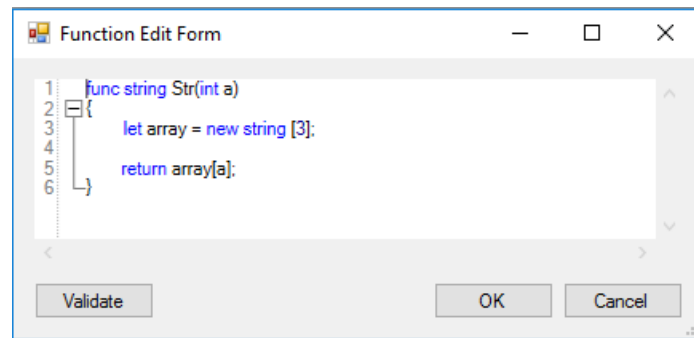


Figure 49: Declaration of the array with defining its length. Elements are not defined and have no values

	Name	Type	Value	Formula
	str	string		Str(0)
	str2	string		Str(1)
▶	str3	string		Str(2)

Figure 50: Elements of the array

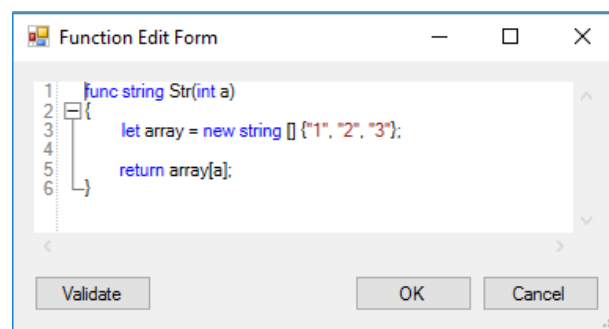


Figure 51: Declaration of the array with defining its elements

	Name	Type	Value	Formula
▶	str	string	1	Str(0)
	str2	string	2	Str(1)
	str3	string	3	Str(2)

Figure 52: Elements of the array

Assignment

To assign a value to a variable, the character “=” can be used.

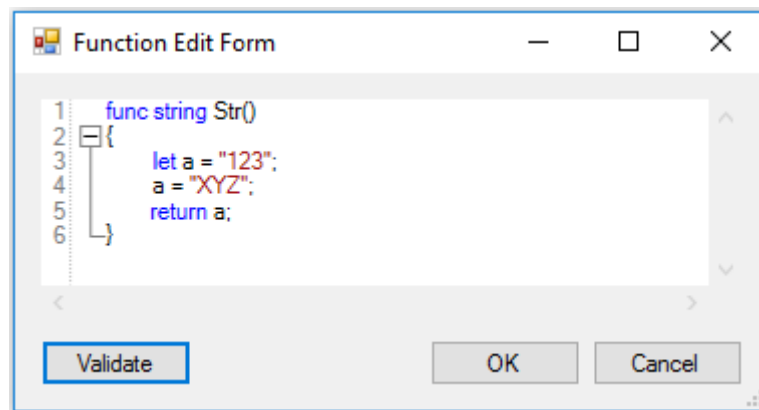


Figure 53: Variable “a” is declared with value “123”, value “XYZ” is assigned to “a”, then returned

	Name	Type	Value	Formula
I	str	string	XYZ	Str()

Figure 54: Variable “a” is returned by function Str()

Array access

Extracting an element of an array is possible with the array's name and an index of the element in square brackets.

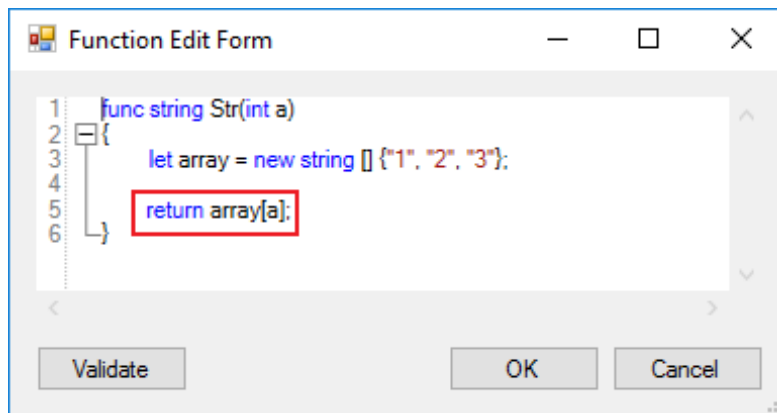
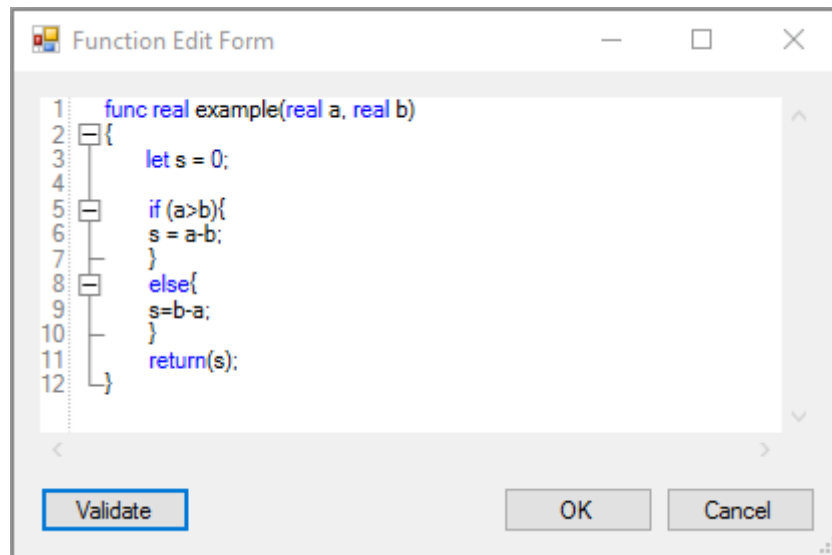


Figure 55: Declaration of an array with defining its elements. Function returns element with inputted index

“If-else”

The “If-else” construction checks if the condition is fulfilled, and depending on this, decides which instruction must be carried out. If the condition is true, then the “if” group’s instructions (instruction after “if” keyword and condition in brackets) are carried out, and if the condition is false, then the “else” group’s instructions (instruction after “else” keyword) are carried out.



The function gets two numbers as arguments, and the “if” statement decides which operation will be carried out based on which number is bigger. If “a” is bigger than “b”, “b” will be subtracted from “a”. In every other case “a” will be subtracted from “b”.

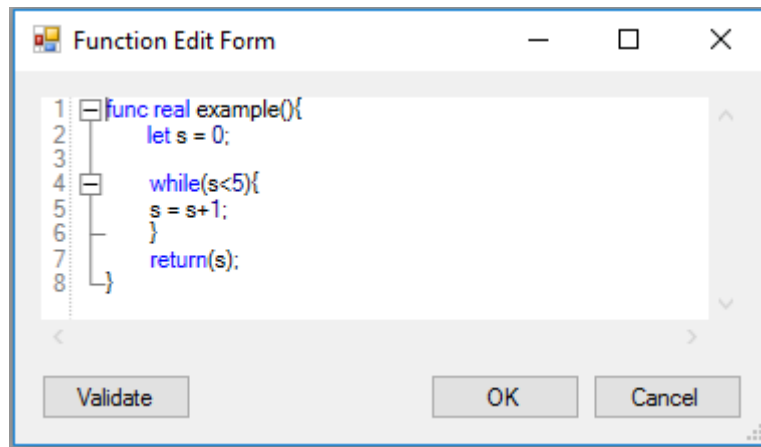
	Name	Type	Value	Formula
I	real	real	5	example(2,7)

	Name	Type	Value	Formula
I	real	real	1	example(8,7)

Figure 56: The function's output

“While”

The “While” construction repeats a set of instructions while the condition is fulfilled.



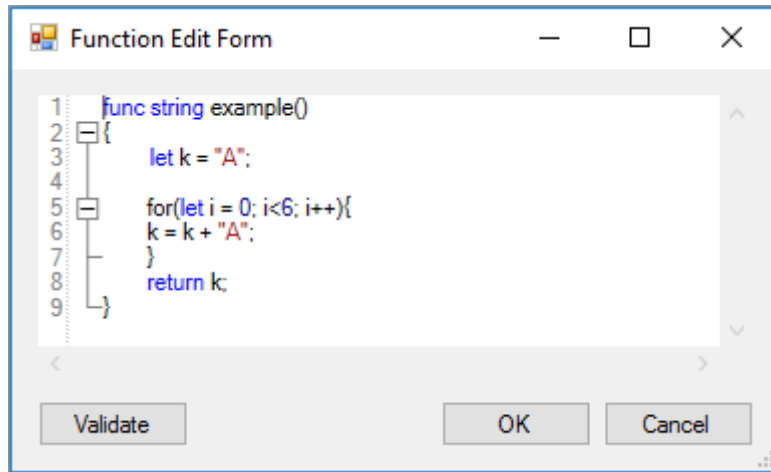
In this example, the “While” variable “s” is less, than 5, the program repeats the cycles. In each cycle “s” will be increased by 1. The cycle stops, when “s” equals 5.

	Name	Type	Value	Formula
▶	real	real	5	example()

Figure 57: The function's output

“For”

The “For” construction repeats some instructions while the condition is fulfilled. In contrary to the “While” construction, “For” has a special variable-counter that can be declared right in a “For” construction. Declaration of the variable-counter, the condition and the action with counter are placed right after keyword “For” in brackets.



The “For” construction has “i” as a variable-counter, and in each cycle the letter “A” is added to the string named “k”. After each cycle, “i” increases by 1.

	Name	Type	Value	Formula
►	str	string	AAAAAAA	example()

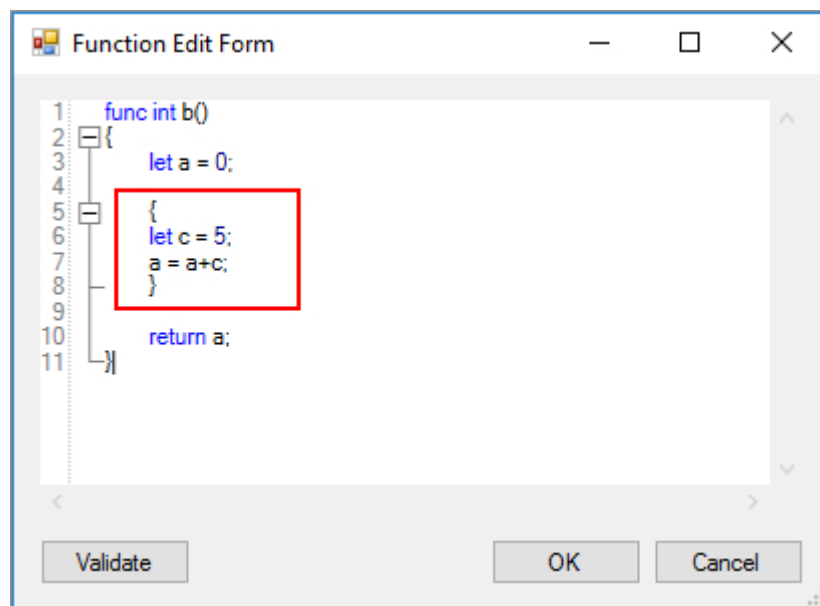
Figure 58: The function's output

Scope rules

“Scope rules” allow to create parts of the code with local variables. It can be used for temporary changing of the global variables’ value or for memory saving (after the end of the scope structure, all variables, that have been declared in it, are deleted). Scope in code is denoted by braces.

Example

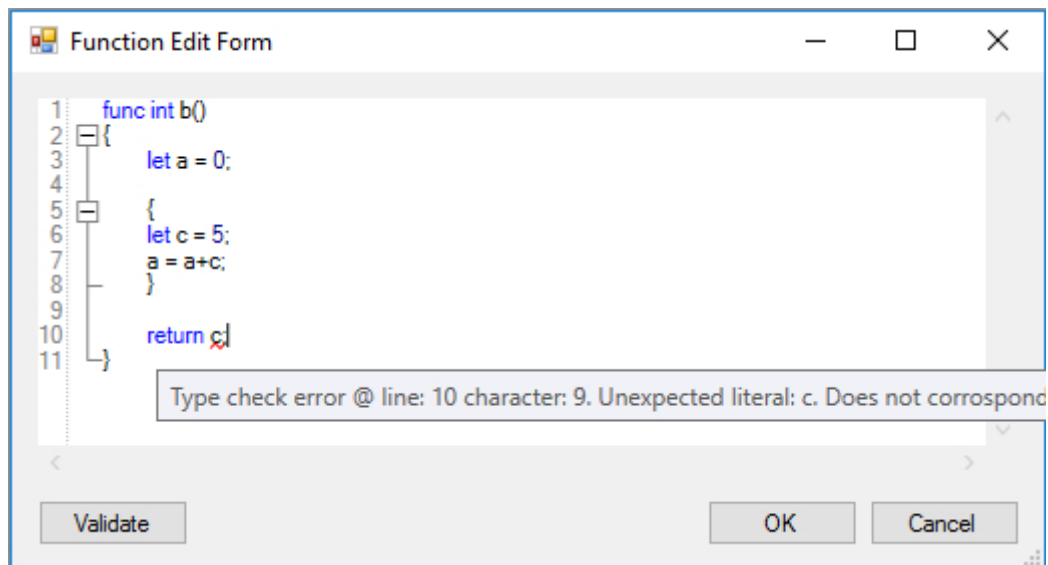
In the function “b()” the variable “a” was declared with the value 0. In the inner scope (marked with red frame) the variable “c” was declared with the value 5. Within the inner scope, the value of “c” was added to the value of “a” was added to “a”, as seen on line 7. Because of the scope rules, the inner scope marked with red can access the value of the variable “a”, which is declared in the outer scope.



	Name	Type	Value	Formula
I	int	integer	5	b()

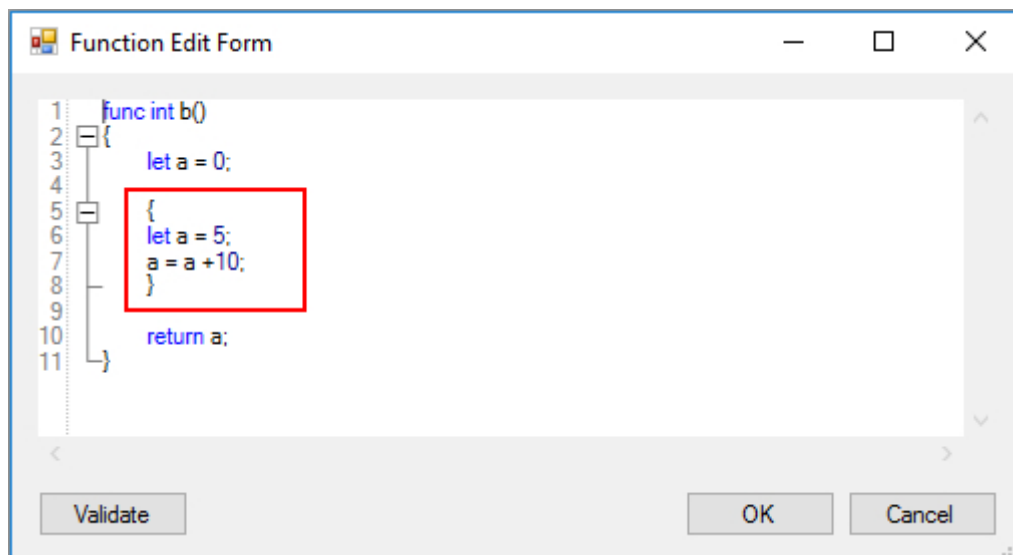
Figure 59: The function's output

But the function can't return the value of the "c", because "c" is declared in the inner scope.



Example

In the function "b()" the variable "a" was declared with the value 0. In the inner scope (marked with red frame) another variable "a" was declared with the value 5. Within the inner scope, 10 is added to the value "a". Because of the scope rules, the only the value of the inner "a" is changed, giving it the value 15, while the value of the outer "a" remains 0. Therefore, the function returns 0, as the return statement uses the value of the outer "a".



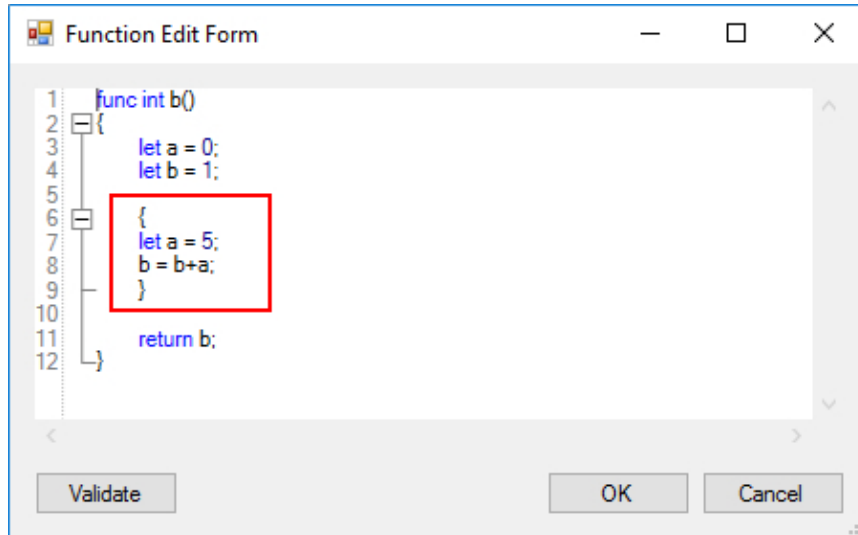
	Name	Type	Value	Formula
▶	int	integer	0	b()

Figure 60: The function's output

Example

In function “b()” was declared variables “a” with value 0 and “b” with value 1. In local scope (marked with red frame) variable “a” was added to “b” and then was declared with value 5. In the moment when “a” was added it wasn’t declared in the local scope so its value was taken from global scope (“a” with value 0 was taken).

After the end of the local scope variable “a” from it was deleted and function returns value “b”.



	Name	Type	Value	Formula
▶	int	integer	1	b()

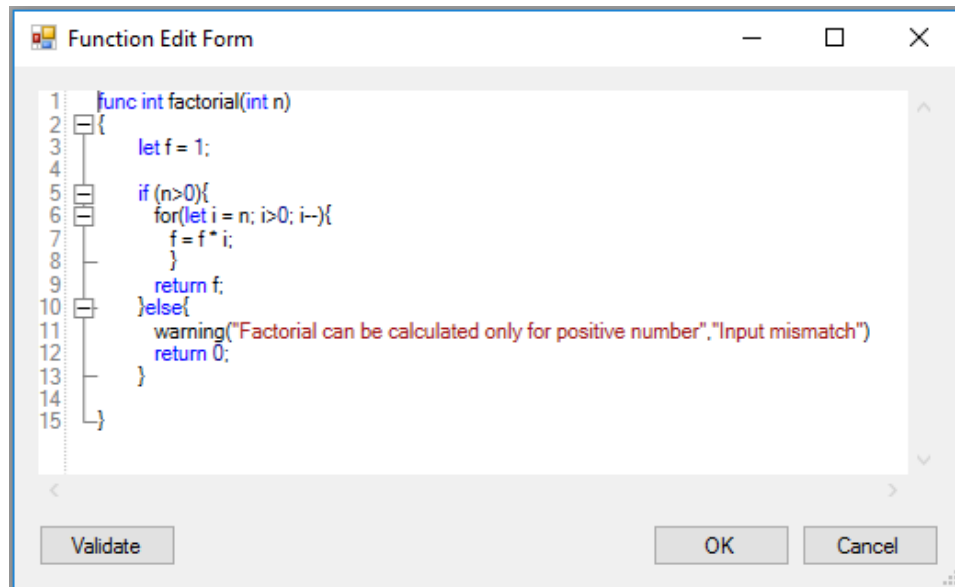
Figure 61: The function's output

In the example, when variable “a” haven’t been declared before carrying out an operation with it, an error message pops up.

Language examples

Calculation function for factorial.

This function has one input, “n”, that defines the number which factorial is searched. If the “n” is positive, the program starts cycle “For” to calculate the factorial. After the last iteration, the function returns the result. If “n” is zero, or negative, the function shows a warning window with inputted text and returns 0.

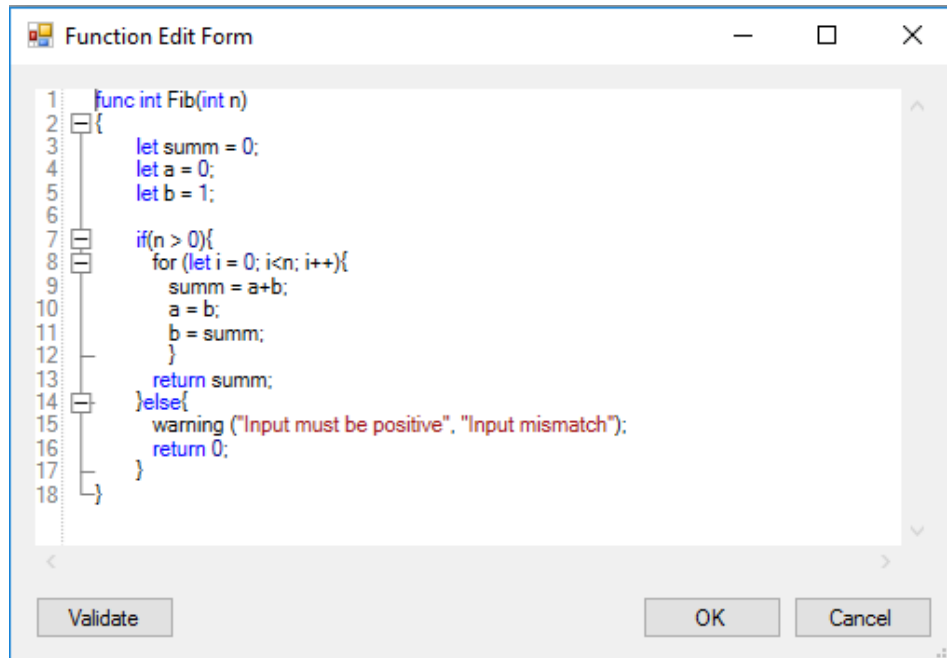


	Name	Display name	Value	Formula
I	int	int	120	factorial(5)

Figure 62: The function's output

Calculation function for element of the Fibonacci sequence.

The function has one input, “n”, that defines the index of the searched element of the Fibonacci sequence. If “n” is positive, the program starts cycle “For” to calculate the element. After the last iteration, the function returns the result. If “n” is zero, or negative, the function shows a warning window with inputted text and returns 0.



	Name	Display name	Value	Formula
I	int	int	13	Fib(6)

Figure 63: The function's output

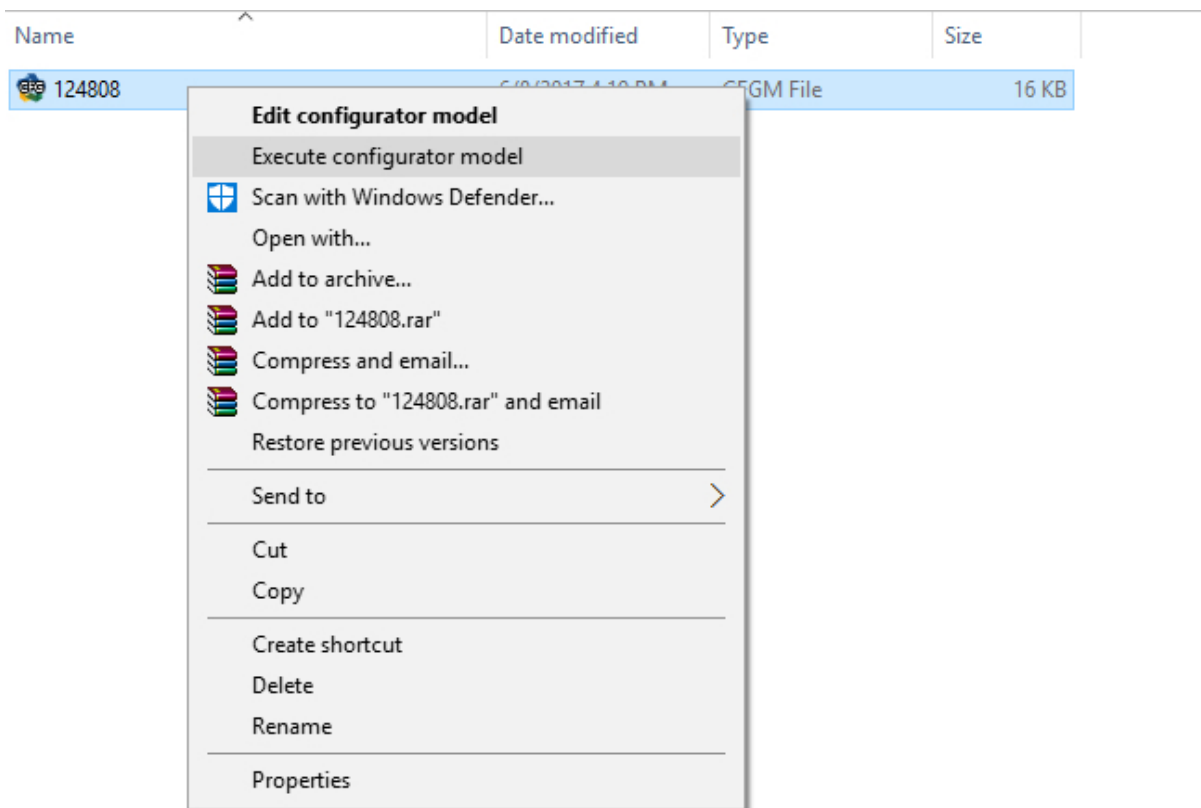
Execution of Configurator model

After the configurator model is created, it can be used in CAD systems and in Bluestar.

Name	Date modified	Type	Size
 124808	6/8/2017 4:19 PM	CFGM File	16 KB

Figure 64: Configurator model in folder

To execute the configurator model, the user should right click on it in the folder and choose "Execute configurator model". This opens the configuration interface of the model.



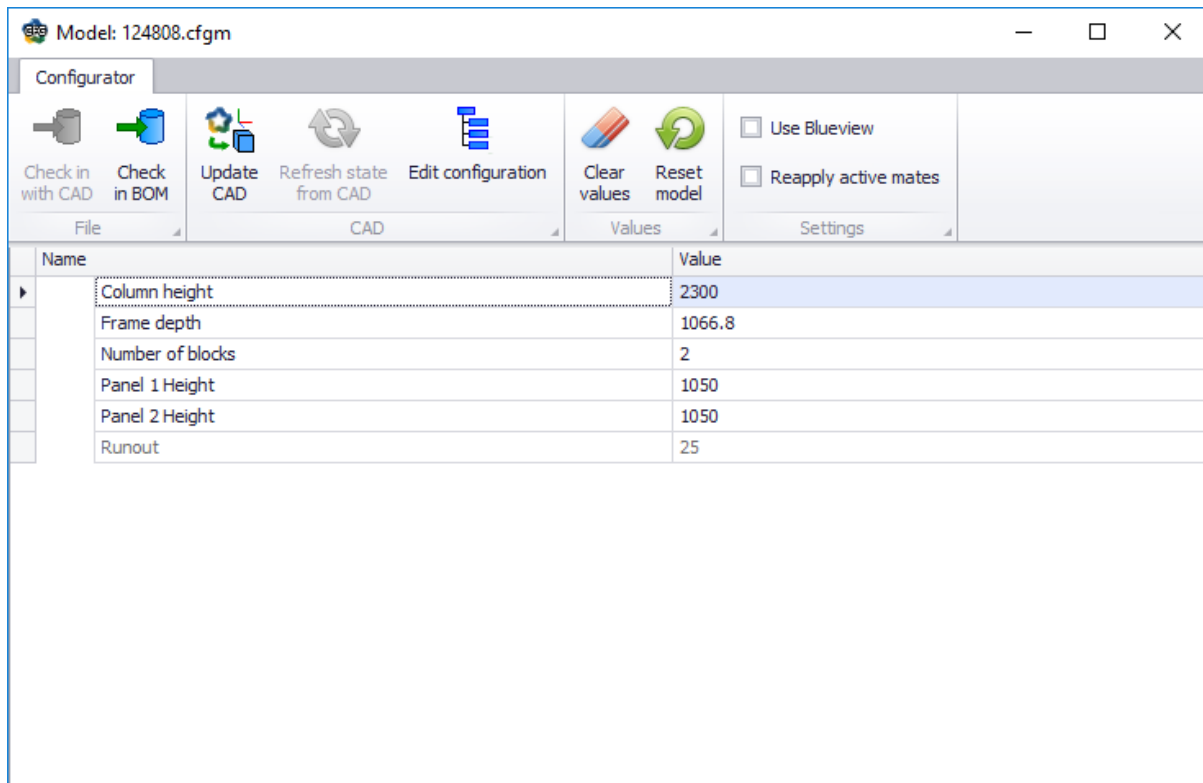


Figure 65: Configurator interface

The configuration interface window has a toolbar and a list of domains.

The toolbar of the “Model” window includes four groups: “File”, “CAD”, “Values” and “Settings”.

File

The “File” group includes two buttons: “Check in with CAD” and “Check in BOM”.

- **“Check in with CAD”** – checks the Configurator model into Bluestar from the CAD system. This button is only available when the CAD is running.
- **“Check in BOM”** – checks the configured BOM into Bluestar.

CAD

The “CAD” group consists of “Update CAD”, “Refresh state from CAD” and “Edit configuration” buttons.

- **“Update CAD”** – updates the model in CAD system.
- **“Refresh state from CAD”** – refreshes the values from the CAD model.
- **“Edit configuration”** – opens the “BOM editor” window.

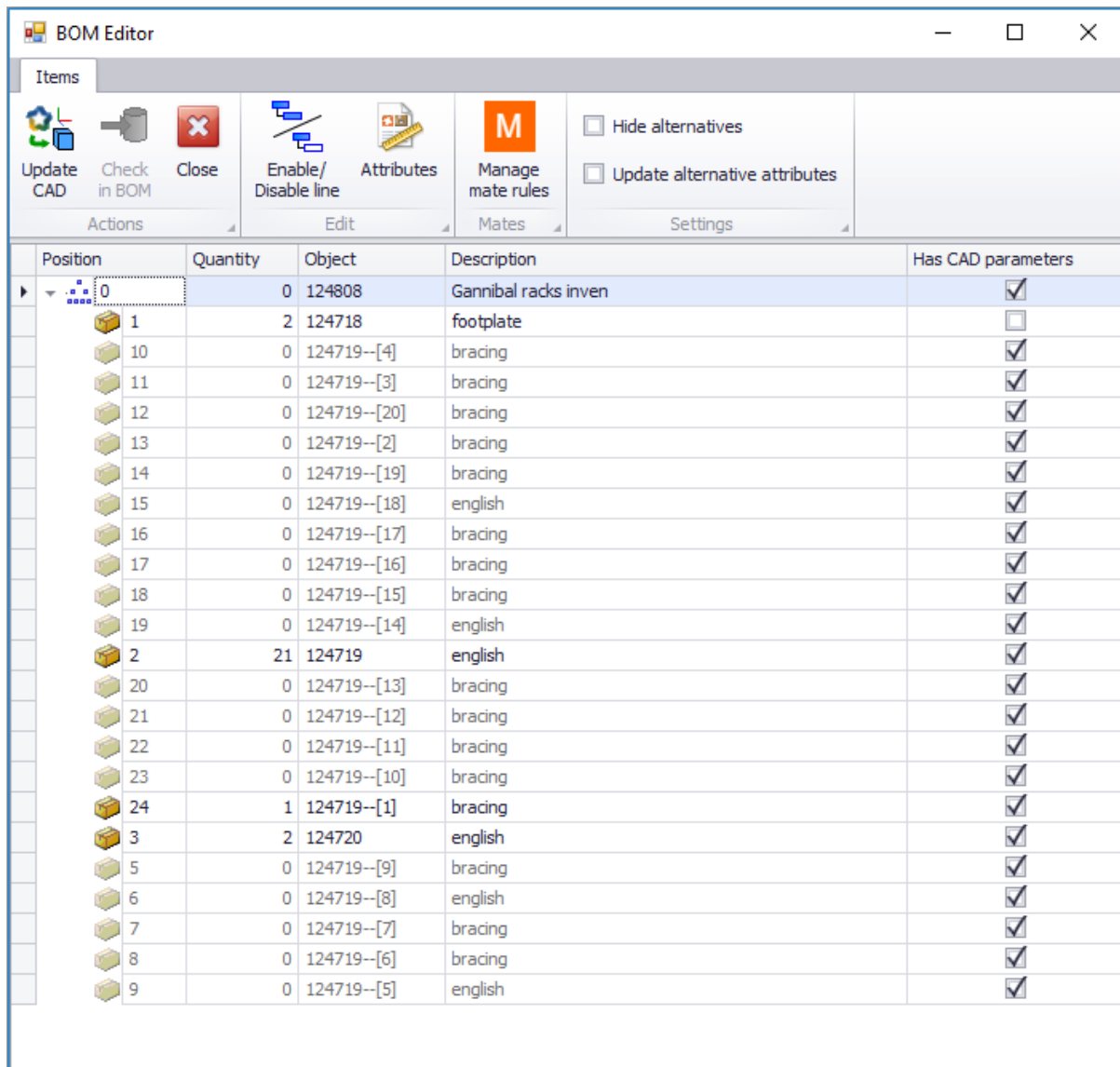


Figure 66: The “BOM editor” window’s toolbar

- This window represents the structure of the model.
- The toolbar’s buttons are:
 - **“Update CAD”** – updates the CAD model.
 - **“Check in BOM”** – checks the BOM into the Bluestar.
 - **“Close”** – closes the window.
 - **“Enable/Disable line”** – manually enables or disables the item in the list.
 - **“Attributes”** – opens the list of the attributes of the selected item.

- **“Manage mate rules”** – manually activates and deactivates the mate rules.
- **“Hide alternatives”** – hides all unused items in the list.
- **“Update alternative attributes”** – updates the attributes of inactive items.

Values

“Values” group includes “Clear values” and “Reset model” buttons.

- **“Clear values”** – allows to reset domains to default values.
- **“Reset model”** – allows to completely reset the Configurator model without saving of any changes.

Settings

“Settings” group includes “Use Blueview” and “Reapply active mates” checkboxes.

- **“Use Blueview”** – allows Configurator to use Blueview instead of any other CAD.
- **“Reapply active mates”** – allows to delete mates and reapply them every time the CAD model is updating. It can be necessary in some cases and in some CAD systems.

Releasing Configurator model

When the user has completed the steps above for creating a configurable model in Bluestar the configurable model can be released to D365 ERP.

- In this example, the configurable ‘Amplifier Package SW’ product needs to have an item release template attached to it before it can be released to Dynamics. The template is based on an item that has already been released to Bluestar. When we release the ‘amplifier’ with this template it will make sure that we get the meta-data we are interested in from the product over to ERP.

+ New Delete Search Check out View files Engineering object Advanced **Dynamics ERP** External ERP

Release View Dynamics Configuration model

Release item Release by excel Released product Inventory transactions Release product configuration model
 Release trace Templates Prices Net requirements View configuration model
 Instruction template

Templates

TEMPLATES

Dynamics item template

200017

Item ↑	Product name
100173	Amplifier package
110905	Multirelease
200015	BS Default Template
200016	BS Manu Template
200017	BS CFG Template
200018	test assembly
200019	BS FC Template

Work area filter: On

Signed on	Status	Class
	Design	
CHARLIE	Design	Amplifie
CHARLIE	Design	Amplifie
CHARLIE	Design	Amplifie
CHARLIE	Design	Amplifie

- To release the item, select it in the engineering grid and press 'release item' in the 'Dynamics ERP' tab.

+ New Delete Search Check out View files Engineering object Advanced **Dynamics ERP**

Release View Dynamics Configuration model

Release item Release by excel Released product Inventory transactions Release product configuration model
 Release trace Templates Prices Net requirements View configuration model
 Instruction template On-hand

- This will open a dialogue that allows you to setup the release, and decide what companies – legal entities – in the Dynamics system you want to release to. Configure the setup and press ok to continue.

Release item

Settings Release companies

Object ID
106451

Variant ID
000356

Object name
Amplifier package SW

Revision
0-0

Item type
BOM

Item group

BOM SETUP

Approved
☐ No

Active
☐ No

ROUTE SETUP

Release routes
☐ No

Approve routes
☐ No

Activate routes
☐ No

SETUP

Release type
Full release

STOPPED

Inventory
☐ No

Purchase order
☐ No

Sales order
☐ No

Activate Windows

Go to Settings to activate Windows

OK

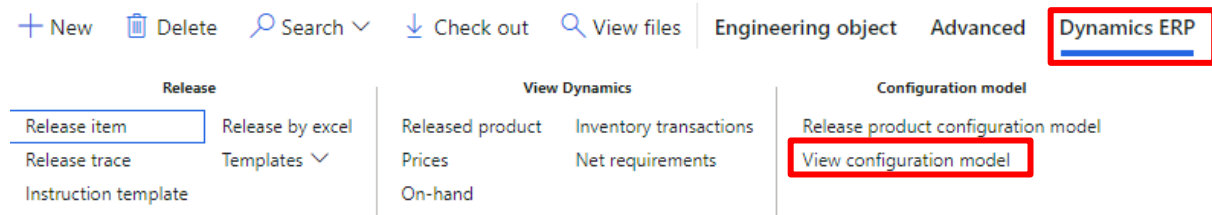
Cancel

- Now the product has been released to D365 ERP. However the configuration model is still not released. Press the 'Release product configuration model' to release it to ERP as well.

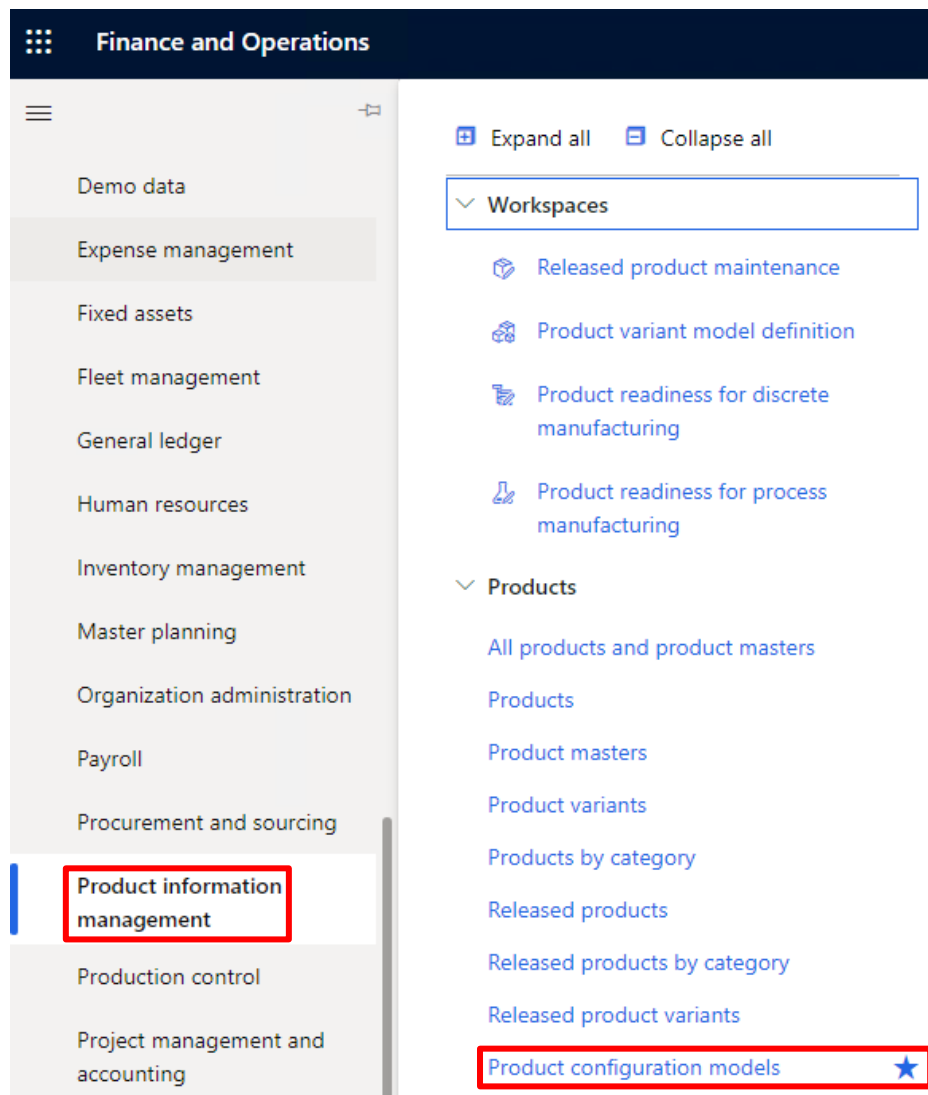
+ New Delete Search Check out View files Engineering object Advanced **Dynamics ERP**

Release		View Dynamics		Configuration model
Release item	Release by excel	Released product	Inventory transactions	Release product configuration model
Release trace	Templates	Prices	Net requirements	View configuration model
Instruction template		On-hand		

Once the product configuration model has been released, there is a quick link – view configuration model - in Bluestar to find the released model:



An overview of all released product configuration models can be accessed through the 'product information management' module:



[+ New](#)
[Delete](#)
[Model](#)
[Options](#)

Maintain
 Duplicate

Product model details
 Versions Model properties
 User interface Configuration templates

Set up
 Components Price models
 Attribute types Price model criteria
 Table constraints

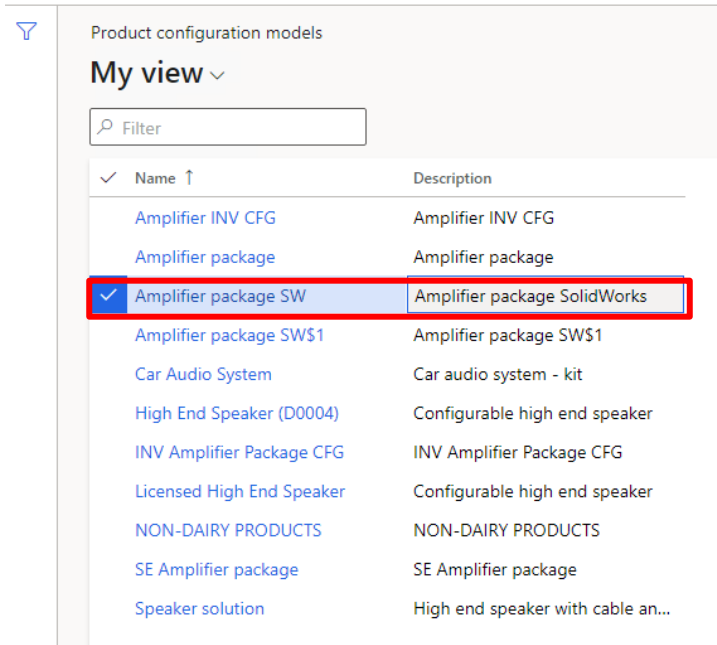
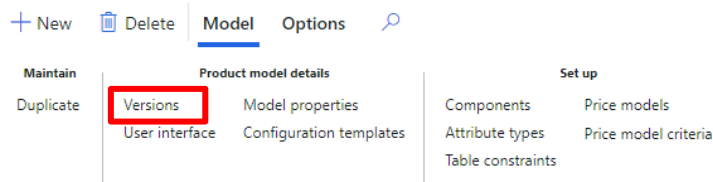
Product configuration models

My view ▾

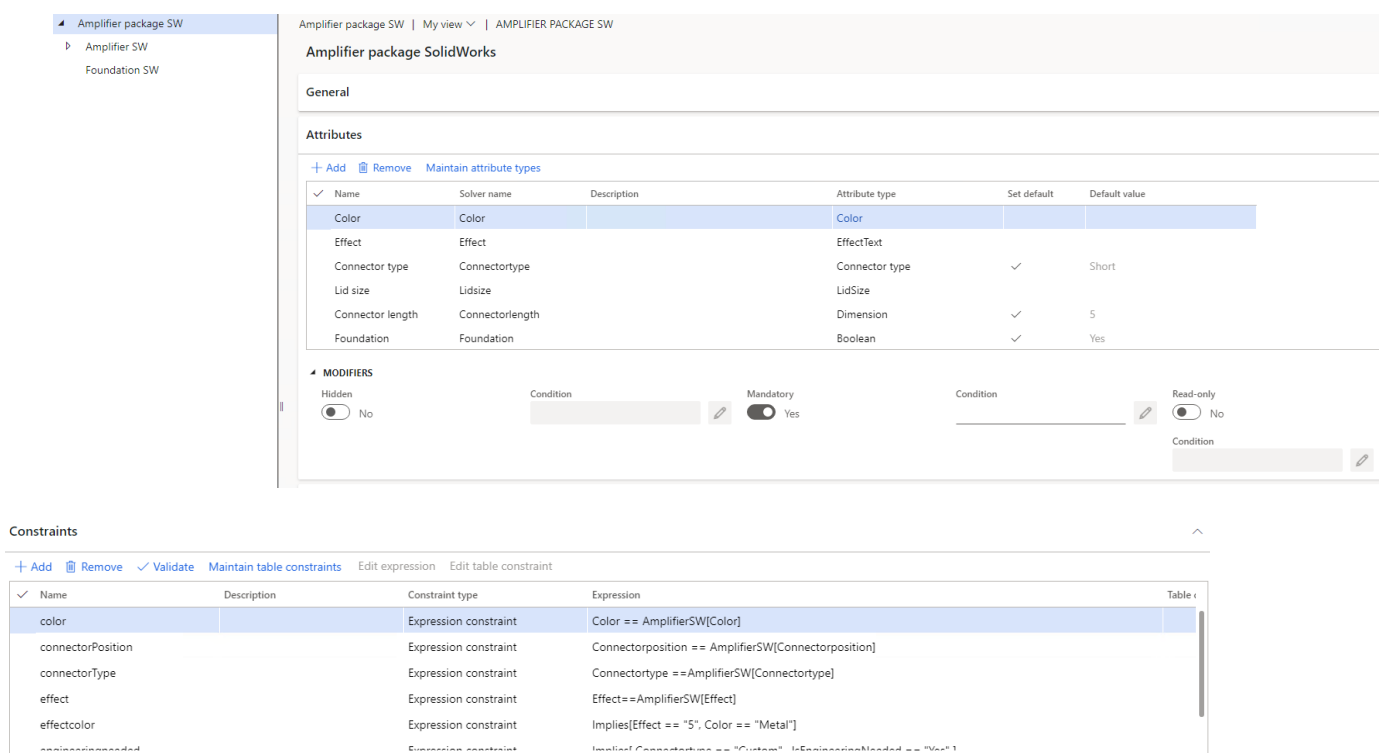
Filter

✓ Name ↑	Description
Amplifier INV CFG	Amplifier INV CFG
Amplifier package	Amplifier package
✓ Amplifier package SW	Amplifier package SolidWorks
Amplifier package SW\$1	Amplifier package SW\$1
Car Audio System	Car audio system - kit
High End Speaker (D0004)	Configurable high end speaker
INV Amplifier Package CFG	INV Amplifier Package CFG
Licensed High End Speaker	Configurable high end speaker
NON-DAIRY PRODUCTS	NON-DAIRY PRODUCTS
SE Amplifier package	SE Amplifier package
Speaker solution	High end speaker with cable an...

- From the product model details we can access the released item related to the selected configuration model



- Selecting the configured product – Amplifier package SW – opens a detail screen that shows the data from Bluestar on the configured model now visible in the AX configurator.
- The release have brought over the attributes, BOM lines and constraints from Bluestar to AX that was set up on the Bluestar side for the configured product.



Calculations

+ Add Remove Validate Edit expression

✓ Name	Description	Target attribute	Expression
length		AmplifierSW[Connectorlength]	Connectorlength

Subcomponents

+ Add Remove BOM line details Maintain components

✓ Name	Solver name	Description	Component	Item number
Amplifier SW	AmplifierSW	Amplifier SW	Amplifier SW	106452
Foundation SW	FoundationSW	Foundation SW	Foundation SW	106443

MONIEFERC

BOM lines

+ Add Remove BOM line details Edit condition

✓ Name	Description	Condition	Item number
POS3_106444	Frame Cover		106444
POS4_106446	IFI 502 - 2-9x1 x 13 B		106446
POS5_106445	RubberRing		106445

- With this data from Bluestar, we can continue working with the model by doing cost calculations and deliveries on the AX side.

Bluestar Sales configurator

Once the product configuration model has been released to FSC, the product can be added to a sales order, from which the product can be configured with Bluestar and Blueview.

This is used for sales personnel to collect requirements for a configurable product from the customer, add it to a sales order, and then transfer the configuration to Bluestar for engineering to review.

- Press the 'Open all sales order' from the 'Sales and marketing' module

Expand all Collapse all

Workspaces

- Sales order processing and inquiry
- Sales return processing

Customers

- All customers
- Recalculate credit limit

Relationships

Marketing

Sales quotations

Sales orders

- All sales orders
- Open orders
- Intercompany orders
- Sales journal
- Order confirmation
- Order shipping
- Sales agreements
- Sales returns

Sales and marketing

- Create a new sales order or open an existing one.

+ New Delete Sales order Sell Manage Pick and pack Invoice Retail General Warehouse

All sales orders

My view

Filter

Sales order	Customer account	Customer name	Order type
000752	US-027	Birch Company	Sales order
000753	US-003	Forest Wholesales	Sales order
000754	US-001	Contoso Retail San Diego	Sales order
000768	US-001	Contoso Retail San Diego	Returned order
000769	US-002	Contoso Retail Los Angeles	Returned order
000770	US-004	Cave Wholesales	Returned order
000771	US-004	Cave Wholesales	Returned order
000772	US-006	Contoso Retail Portland	Returned order
000773	DE-001	Contoso Europe	Sales order
000776	US-027	Birch Company	Sales order
000783	US-001	Contoso Retail San Diego	Sales order
000784	US-003	Forest Wholesales	Sales order
000786	US-027	Birch Company	Sales order
000787	US-027	Birch Company	Sales order
000788	US-003	Forest Wholesales	Sales order
000789	US-004	Cave Wholesales	Sales order

- Once the sales order is open, you can add a new line and select a product based on a configurable model – in this case the 'Amplifier package SW'.

Sales order details | My view ▾

000788 : Forest Wholesales

Sales order header

DELIVERY ADDRESS

Name: Forest Wholesales

Delivery address: Forest Wholesales

Address: 123 White Road
Los Angeles, CA 90004
USA

DELIVERY DATE

Requested ship date: 3/21/2017

Requested receipt date: 3/22/2017

Confirmed ship date:

Confirmed receipt date:

[Simulate delivery dates](#)

REFERENCES

Customer reference:

Customer requisition:

Item number ↑	Search name ▾	Product name	Item group	Item type
100681	AmplifierPackageCFG	Amplifier Package CFG Inventor	Audio	Item
106451	AmplifierpackageSW	Amplifier package SW	Audio	Item
200106	Amplifierpackage	Amplifier package	Audio	Item
200125	Amplifierpackage	Amplifier package	Audio	Item
200175	Amplifierpackage	Amplifier package2	Audio	Item
200206	Amplifierpackage	Amplifier package	Audio	Item
220002	Amplifierpackage	SE Amplifier package	Audio	Item

Sales order lines

[+ Add line](#) [+ Add lines](#) [Add products](#) [Remove](#)

✓	Type	Variant number	Item number	Configuration	Product name	As built revision	Sales category	CW quantity	CW unit	Quantity	Unit
✓		VN-000447	106451	424	Amplifier package SW / Amplifier package SW						
✓		VN-000431	106451	405	Amplifier package SW / Amplifier package SW						

The configurable product is now added to the sales order, and with Bluestar sales can now configure the product in multiple ways.

In this example we choose to configure with the Blueview application that allows us to configure the product, while we see our configurations visualized in real-time in the Blueview 3D product viewer.

- Select the line, and press the 'Bluestar PLM' button. In the dropdown select 'Configure line using Blueview'.

Sales order details | My view ▾

000788 : Forest Wholesales

Sales order header

DELIVERY ADDRESS

Name: Forest Wholesales

Delivery address: Forest Wholesales

Address: 123 White Road
Los Angeles, CA 90004
USA

DELIVERY DATE

Requested ship date: 3/21/2017

Requested receipt date: 3/22/2017

Confirmed ship date:

Confirmed receipt date:

[Simulate delivery dates](#)

REFERENCES

Customer reference:

Customer requisition:

DISCOUNTS

Total discount %: 0.00

WAREHOUSE

Release status: Open

TRANSPORTATION

Routes:

Sales order lines

[+ Add line](#) [+ Add lines](#) [Add products](#) [Remove](#) [Sales order line](#) [Financials](#) [Inventory](#) [Product and supply](#) [Update line](#) [Warehouse](#) [Retail](#) [Bluestar PLM](#) [Attachments](#)

✓	Type	Variant number	Item number	Configuration	Product name	As built revision	Sales category	CW quantity	CW unit	Quantity	Unit
✓		VN-000431	106451	405	Amplifier package SW / Amplifier package SW					1.00	Pcs
✓		VN-000433	106451	406	Amplifier package SW / Amplifier package SW					1.00	Pcs
✓		VN-000435	106451	407	Amplifier package SW / Amplifier package SW					1.00	Pcs
✓		VN-000173	106451	Default	Amplifier package SW / Amplifier package					1.00	Pcs
✓		VN-000173	106451	Default	Amplifier package SW / Amplifier package					1.00	Pcs

CONFIGURATION

[Configure line using Blueview](#)

[Configure product in Bluestar PLM](#)

[Configure product in Bluestar PLM \(Web\)](#)

[Transfer configuration to Bluestar PLM](#)

[Check out](#)

[Smart export](#)

VIEW

[View files](#)

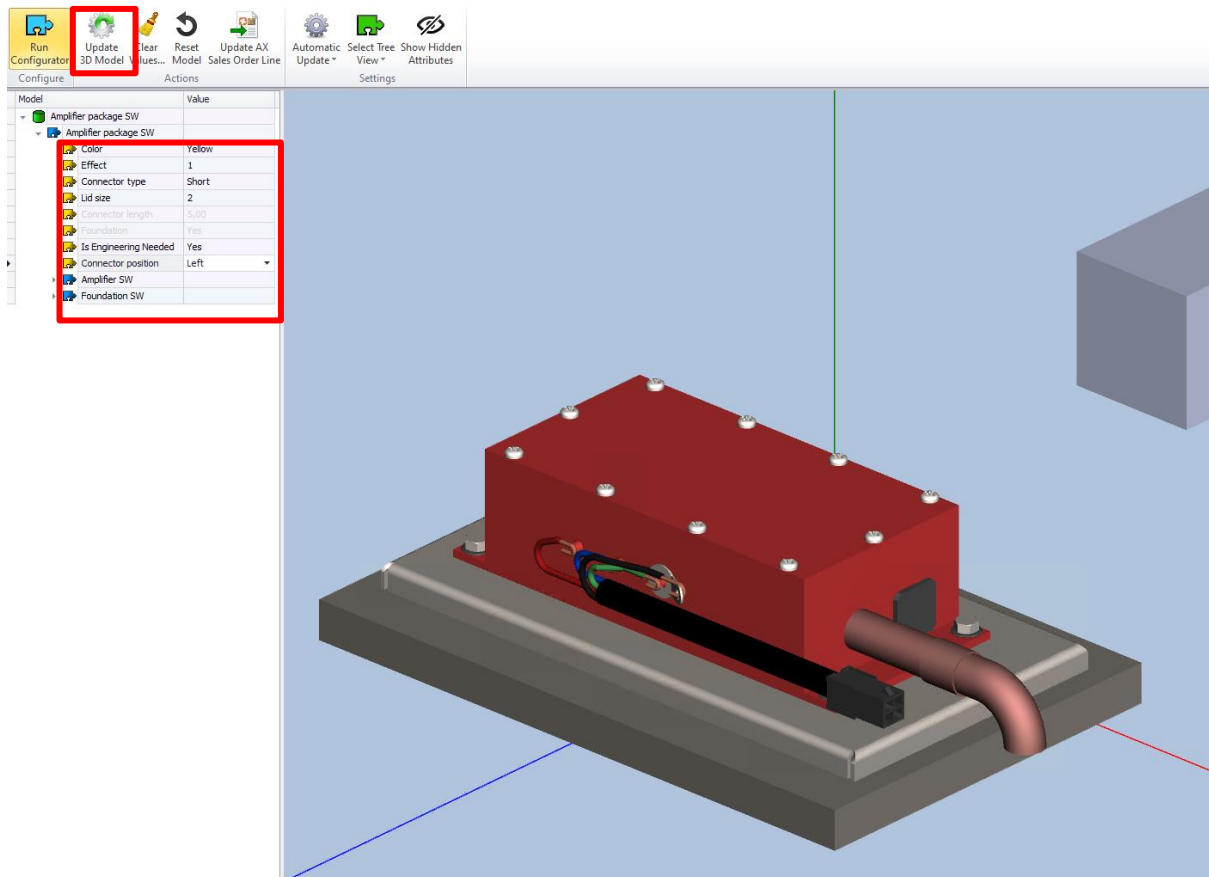
[Engineering object](#)

ATTACH

[New SmartObject](#)

[New change](#)

- This will open the Blueview application with the amplifier model. From here you can configure the model from the predefined configuration options.
- Chose one of the available options for each of the product attributes. If an attribute is marked in red text, this means the configuration option is not possible due to constraints.
- Once you are happy with the configuration press 'Update 3D Model'



- You will then see the result immediately in the model

